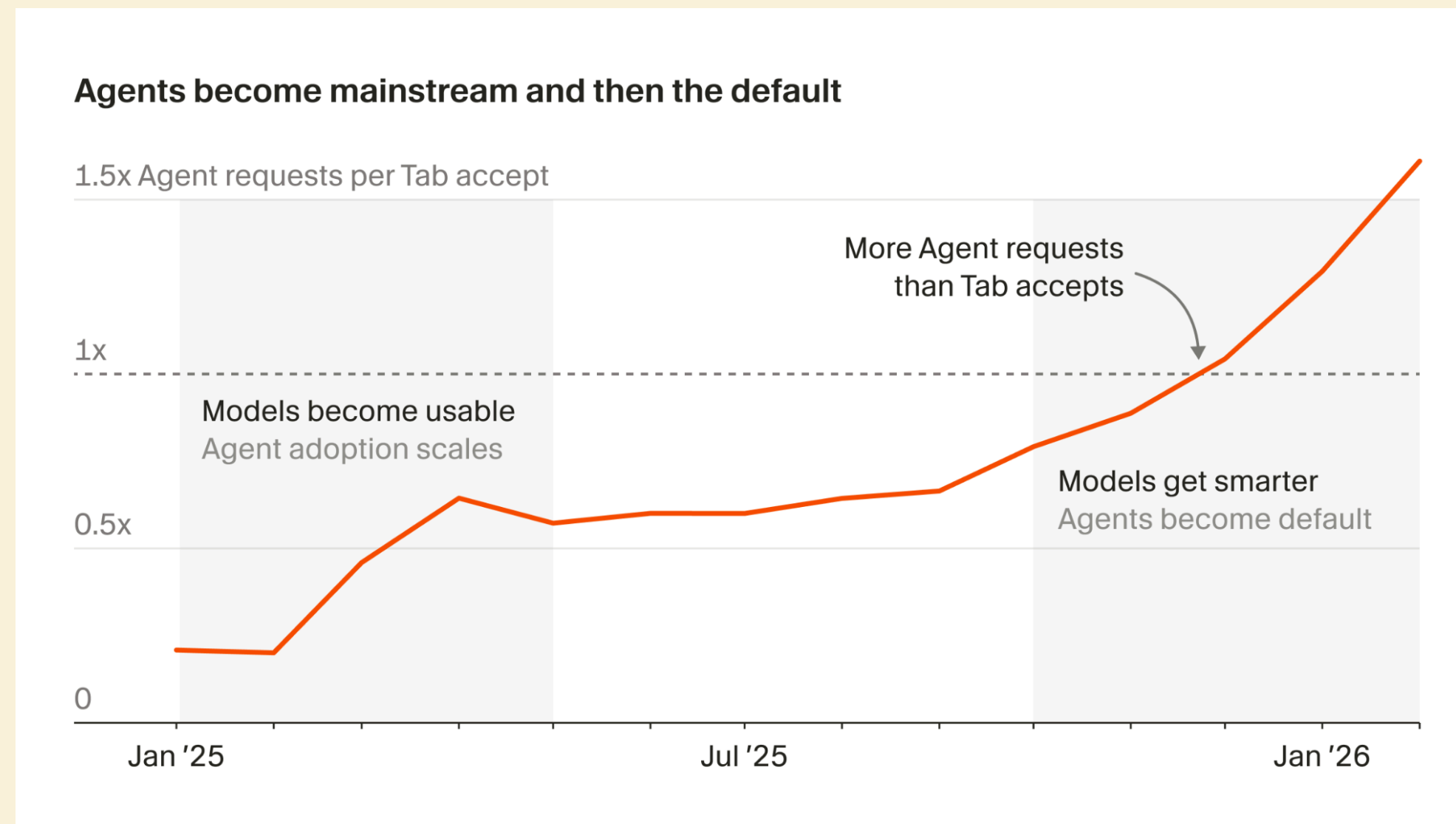


Engineering the Harness: A Practical Workshop

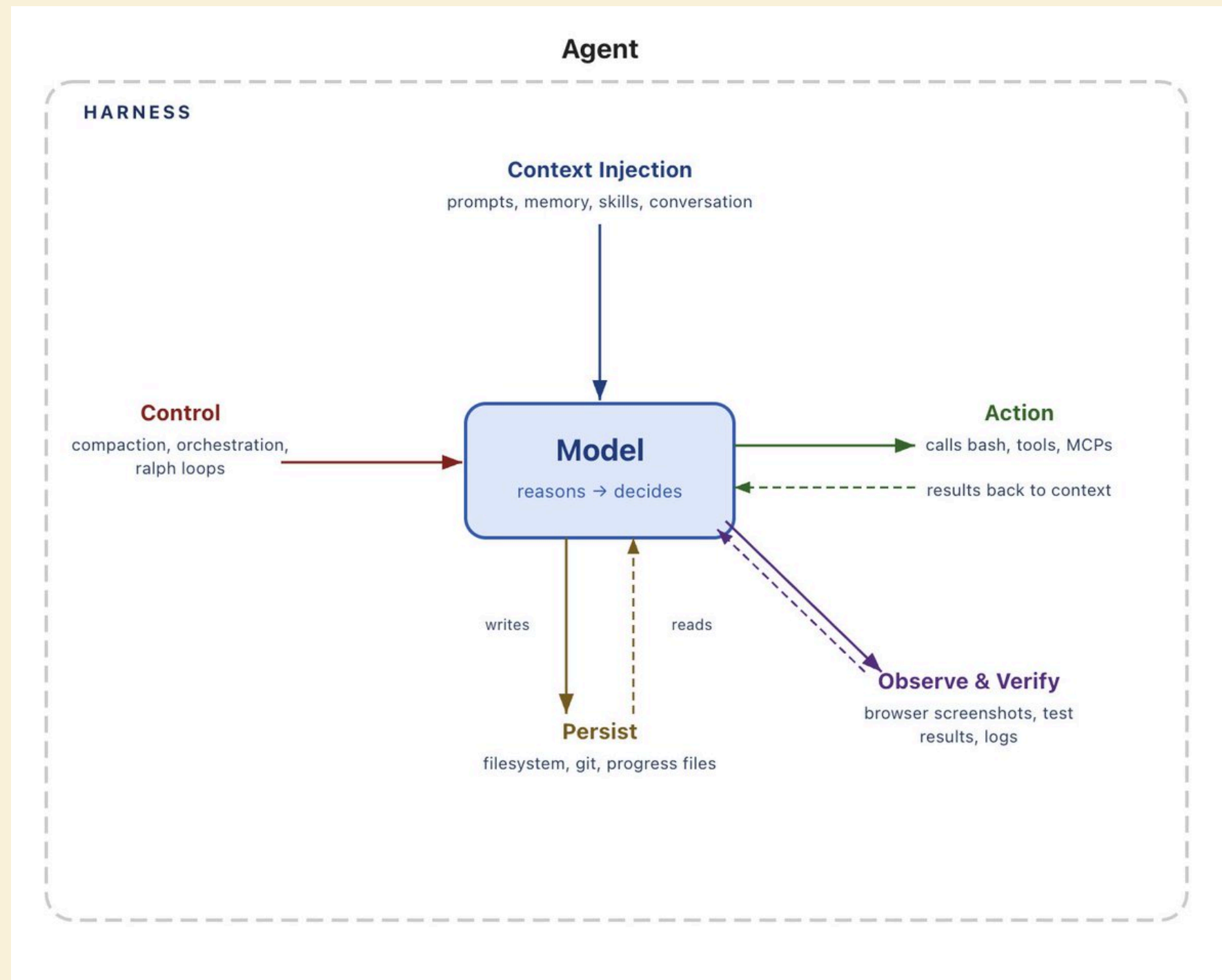
Rajiv Shah · OpenHands

We have moved beyond tab completion

Cursor users made more agent requests than autocomplete accepts: developers were delegating tasks, not just accepting code suggestions.



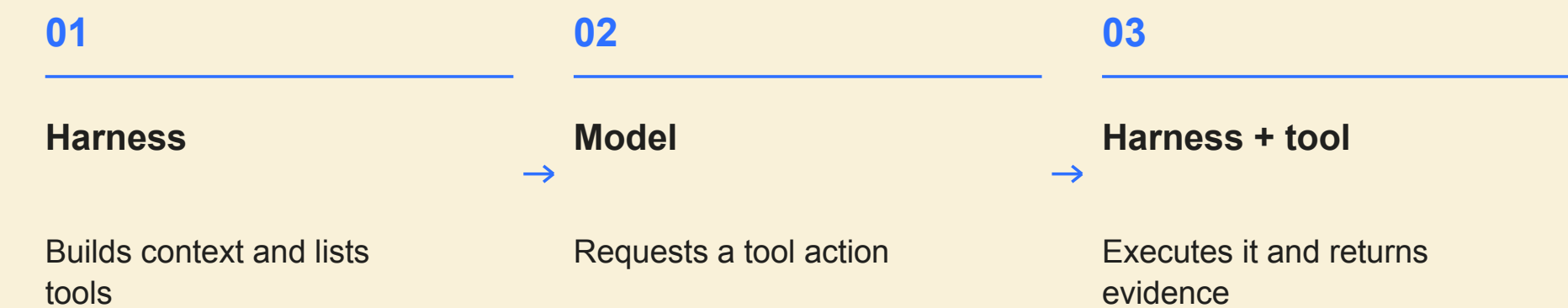
Agent = model + harness



The model requests an action. The harness supplies context, executes the tool, and returns the result.

What an agent does

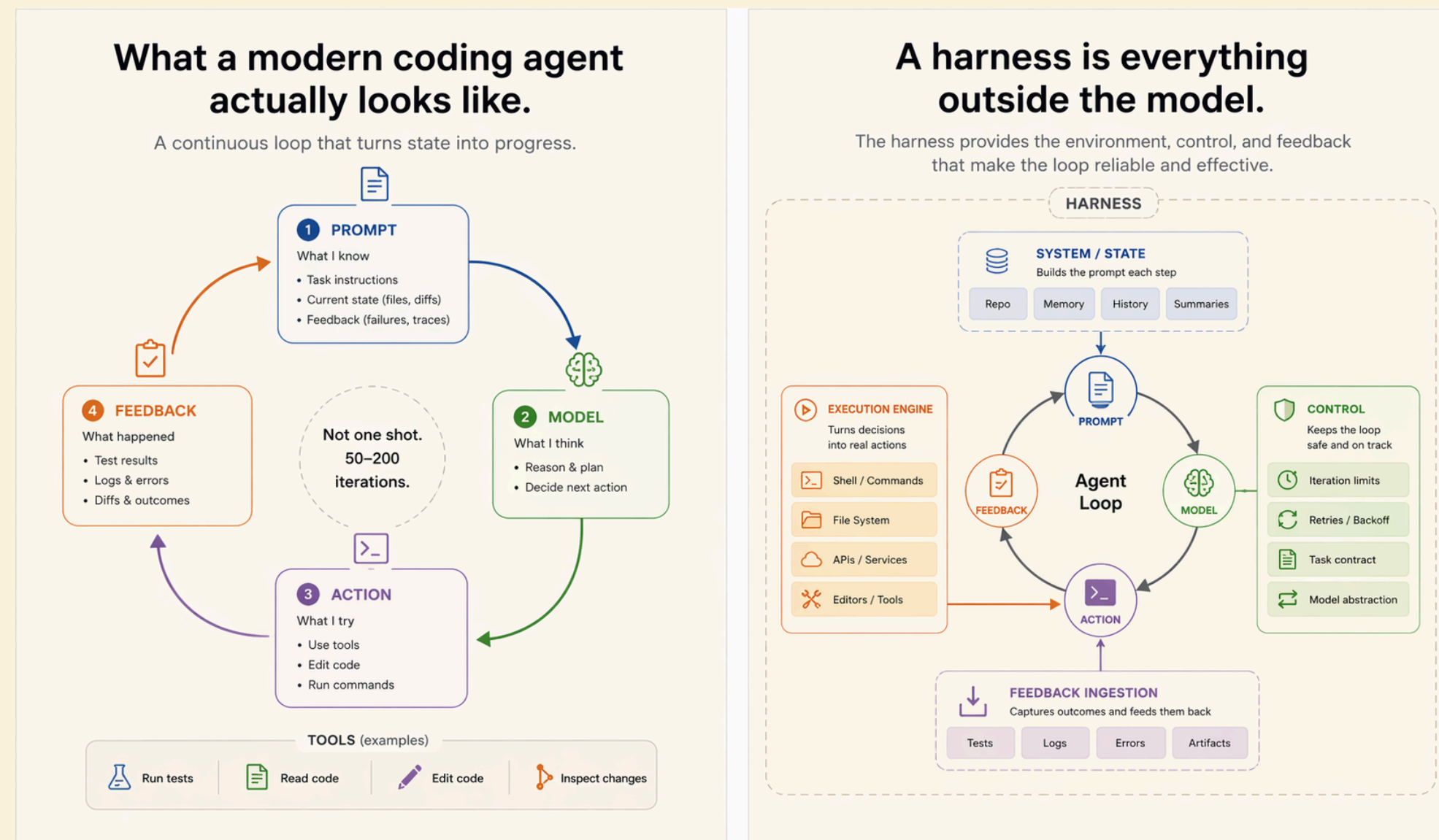
A tool result becomes input to the next model call. The trace records that exchange.



The returned observation becomes context for the next model call. Illustrative loop, not a new trace.

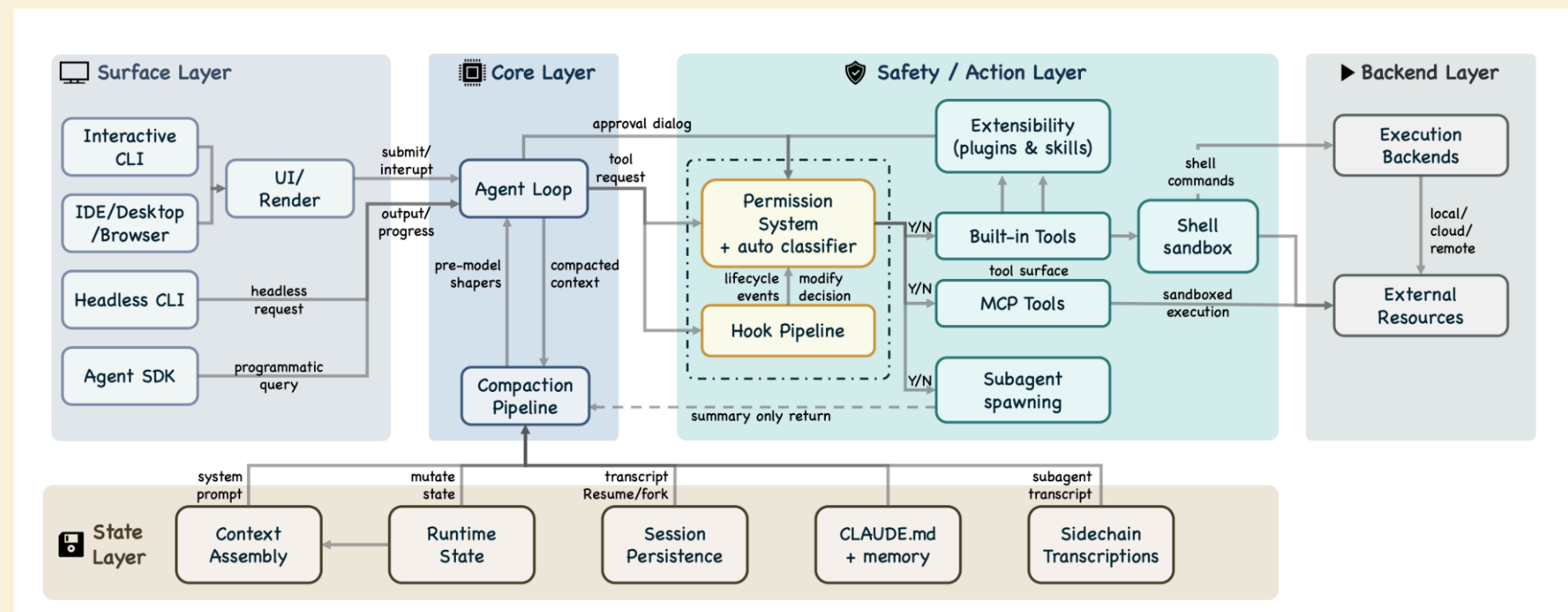
What is in a harness

Instructions, tools, context management, execution, and stopping rules all sit outside the model.



Harnesses get complicated quickly

A modern coding agent includes much more than the model and a tool-call loop.



Rise of Claude Code and Codex

Multiple selections allowed; survey percentages are not additive.

Tool	Earlier survey	Later survey
Claude Code	18%	39%
Codex	3%	16%
Copilot	29%	21%
Cursor	18%	12%

Working-deck survey figures, pending source recheck. Multiple tools allowed; shares are not additive.

Review source: verify the dated survey figures before presenting.

Harnesses from model providers

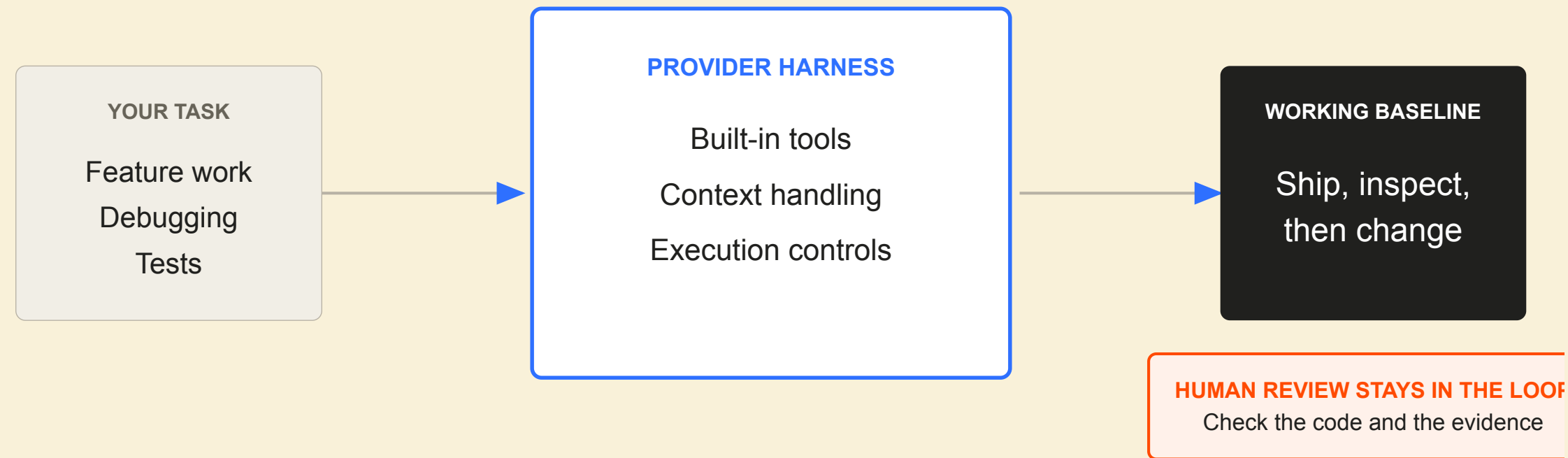
Each provider ships a coding harness with its own tools, context policies, and execution controls.

Model provider	Harness
Anthropic	Claude Code
OpenAI	Codex
DeepSeek	DeepSeek Harness

Provider-owned harnesses. GitHub counts in the working notes need a dated refresh.

Review source: archived GitHub star counts need a dated check.

Provider harnesses are a good starting point

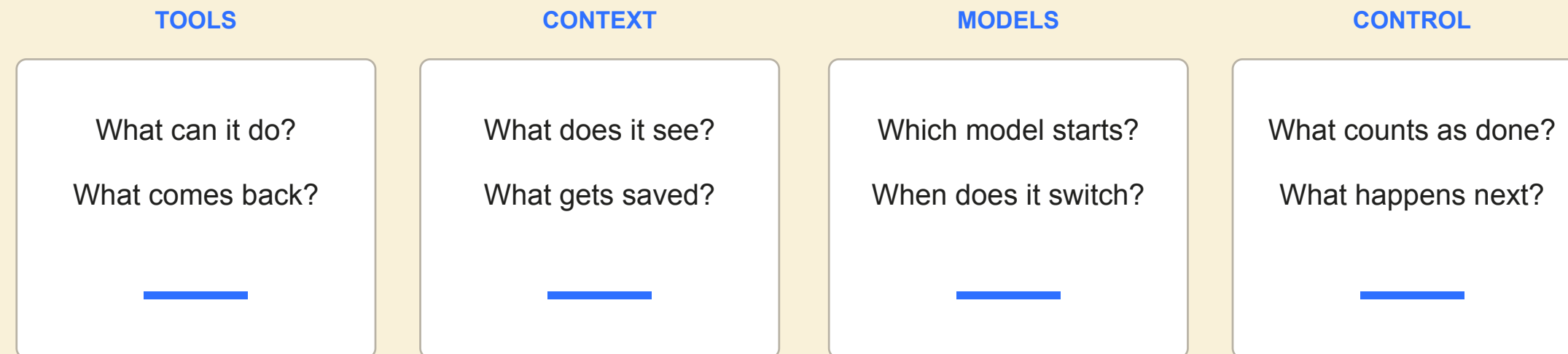


Claude Code and Codex handle feature work, debugging, refactoring, and tests.

Anthropic observed 67% more merged PRs per engineer per day after adopting Claude Code. Internal observation, with human review.

What you can change in a harness

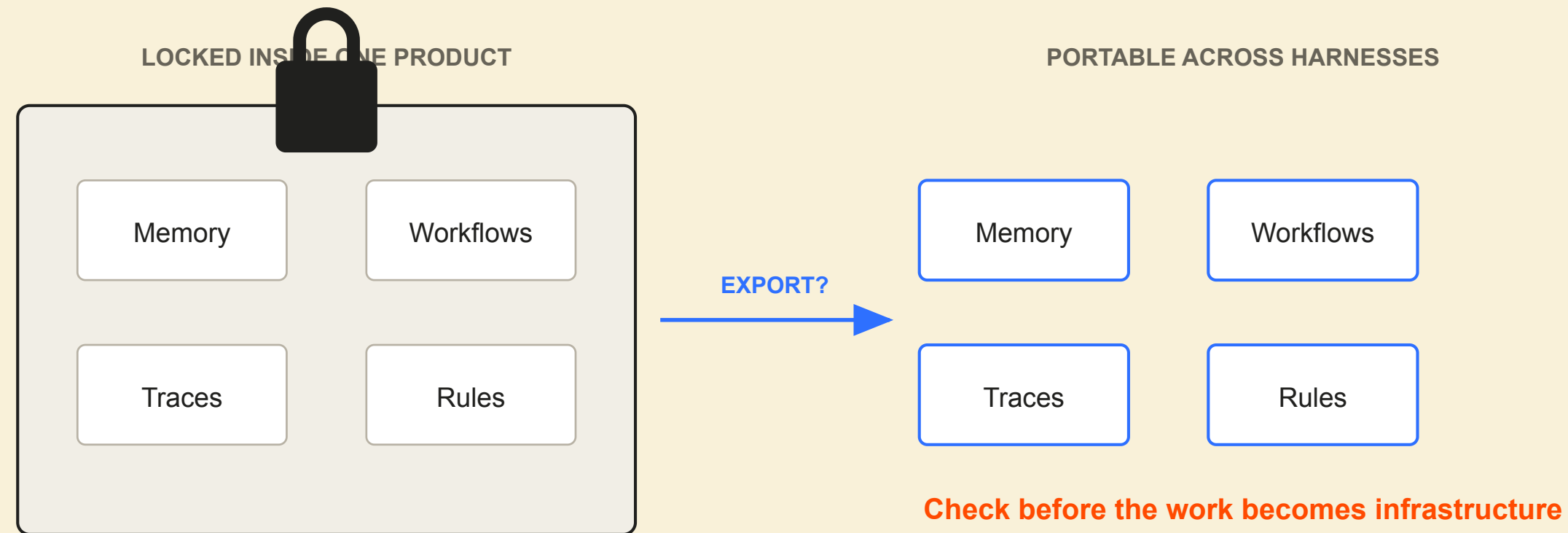
THE MODEL STAYS FIXED. THE HARNESS CHANGES WHAT SURROUNDS IT.



These are the four engineering surfaces

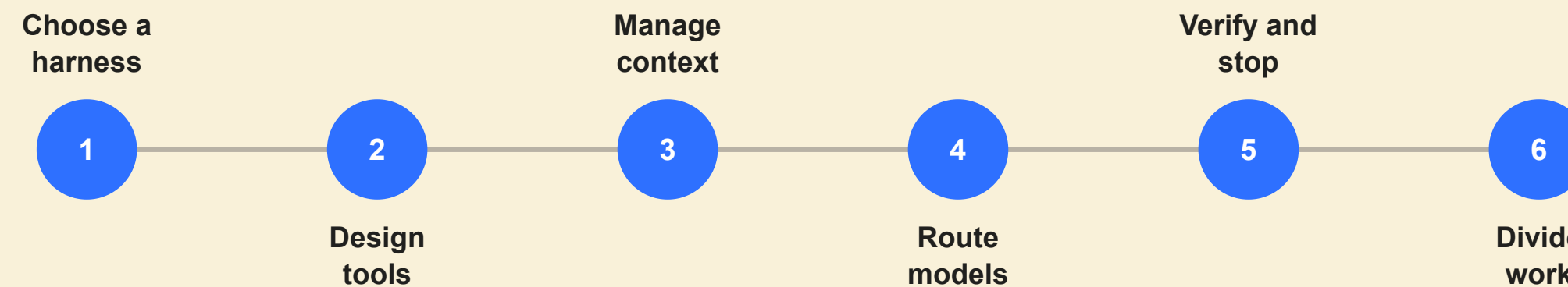
Provider defaults can miss requirements or repeat work. Tools, context, model choice, and control are the four places we can change the behavior.

Consequences of a closed harness



A closed harness can lock you into its memory and workflows. Check whether you can export and reuse them elsewhere.

Workshop roadmap: inside the harness



Each workshop changes one part of the same agent loop.

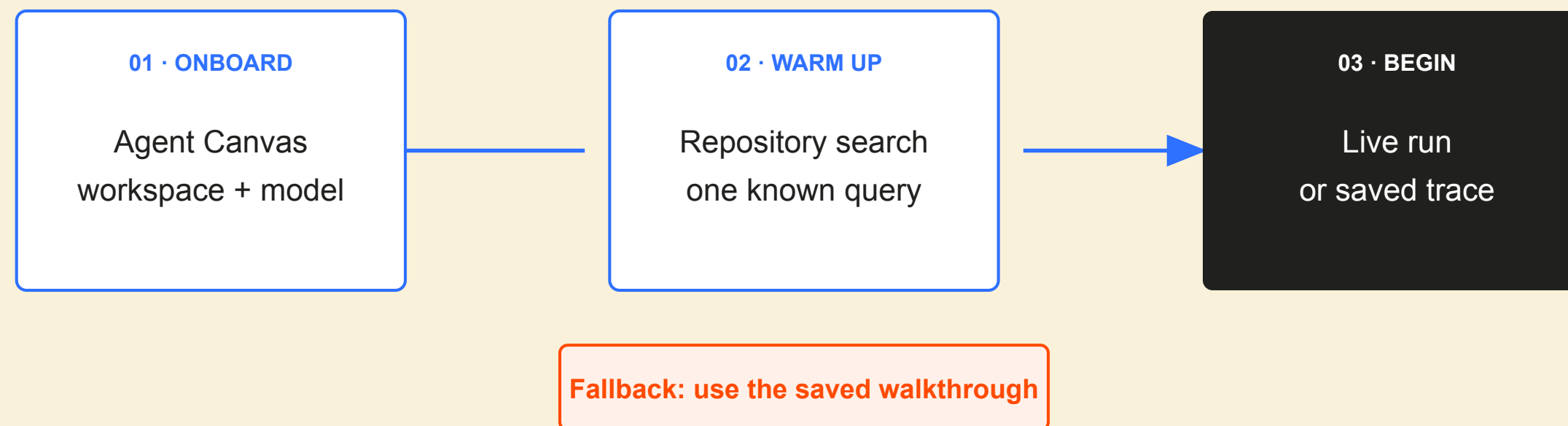
Each workshop changes one part of the same agent loop, then checks what changed in the trace and result.

Workshop 1: Harness choice

01

We will hold the task fixed and inspect what each harness changes.

Workshop environment and fallback path

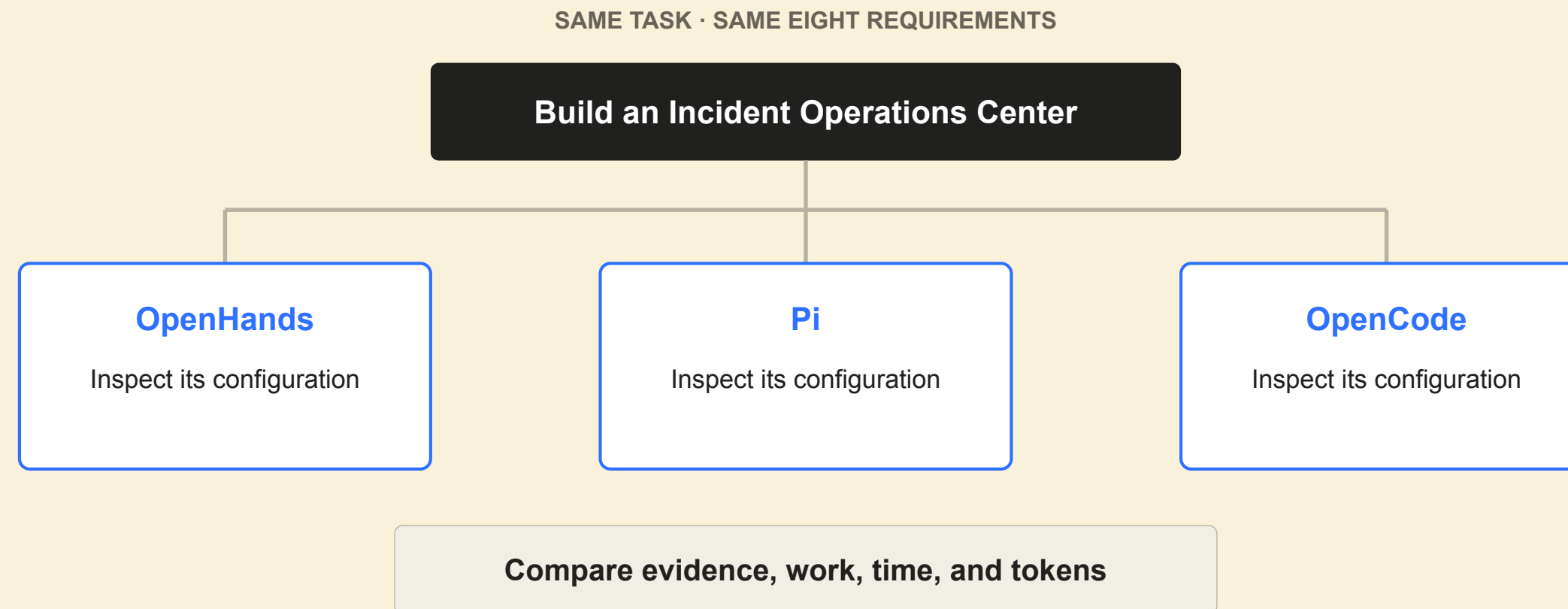


Open the quickstart and complete Agent Canvas onboarding.

Check the workspace, model, and provider; run the repository-search warmup.

If setup is incomplete, follow the saved-trace walkthrough.

Harness comparison: the same task across three systems



Build an Incident Operations Center against eight requirements.

Compare the saved OpenHands, Pi, and OpenCode runs. Check their configurations before attributing differences to the harness.

Trace one decision to the harness

Trace one request through tool execution and the returned result. Identify the configuration responsible for that behavior.

The model chooses actions inside a harness

Selected events from a real Agent Canvas run · Incident Operations Center · GLM-5.2

HARNESS	Supplies system instructions and available tools	00000
USER	Requests a durable incident-operations system	00001
MODEL	Requests a terminal call: list repository files	00004
HARNESS / TOOL	Executes the command and returns file names	00005
MODEL	Requests a full test run	00121
HARNESS / TOOL	Returns the test result: 41 passed in 11.14s	00122
MODEL	Proposes completion through the finish tool	00286

Editing and intermediate events omitted. Passing tests establish only the behavior those tests cover.

Completion scores hid large differences in work

context on each call and finished with higher token use and cost.

Same full-stack project with Sonnet 4.5

We repeated the incident project with Sonnet to test whether the GLM-5.2 pattern was a property of the harness alone. It was not.

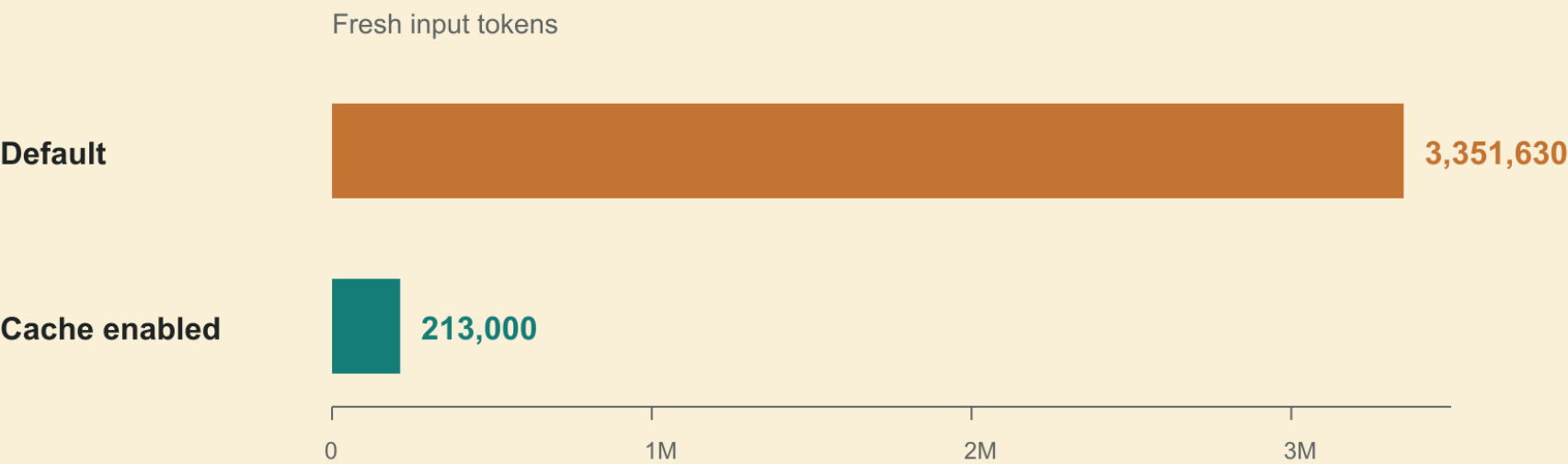
Harness	Quality	Time	Model calls	Tool calls	Input	Cached	Fresh input	Output
OpenHands	7/8	12m 24s	66	65	4.26M	97.7%	98K	42.5K
Pi, cache enabled	6/8	16m 49s	89	88	3.84M	94.5%	213K	58.3K
OpenCode	6/8	15m 52s	75	82	4.15M	97.9%	87K	82.6K

Similar completion scores can require very different amounts of work.

Provider caching cut Pi's fresh input from 3.35M to 213K tokens

Cache markers reduced fresh input in the Pi rerun

Same model and task: Sonnet 4.5 · Incident Operations Center · one trial per configuration



THE CHANGE `cacheControlFormat: "anthropic"`

Both runs: 89 model calls. External checks: 7/8 before, 6/8 after.

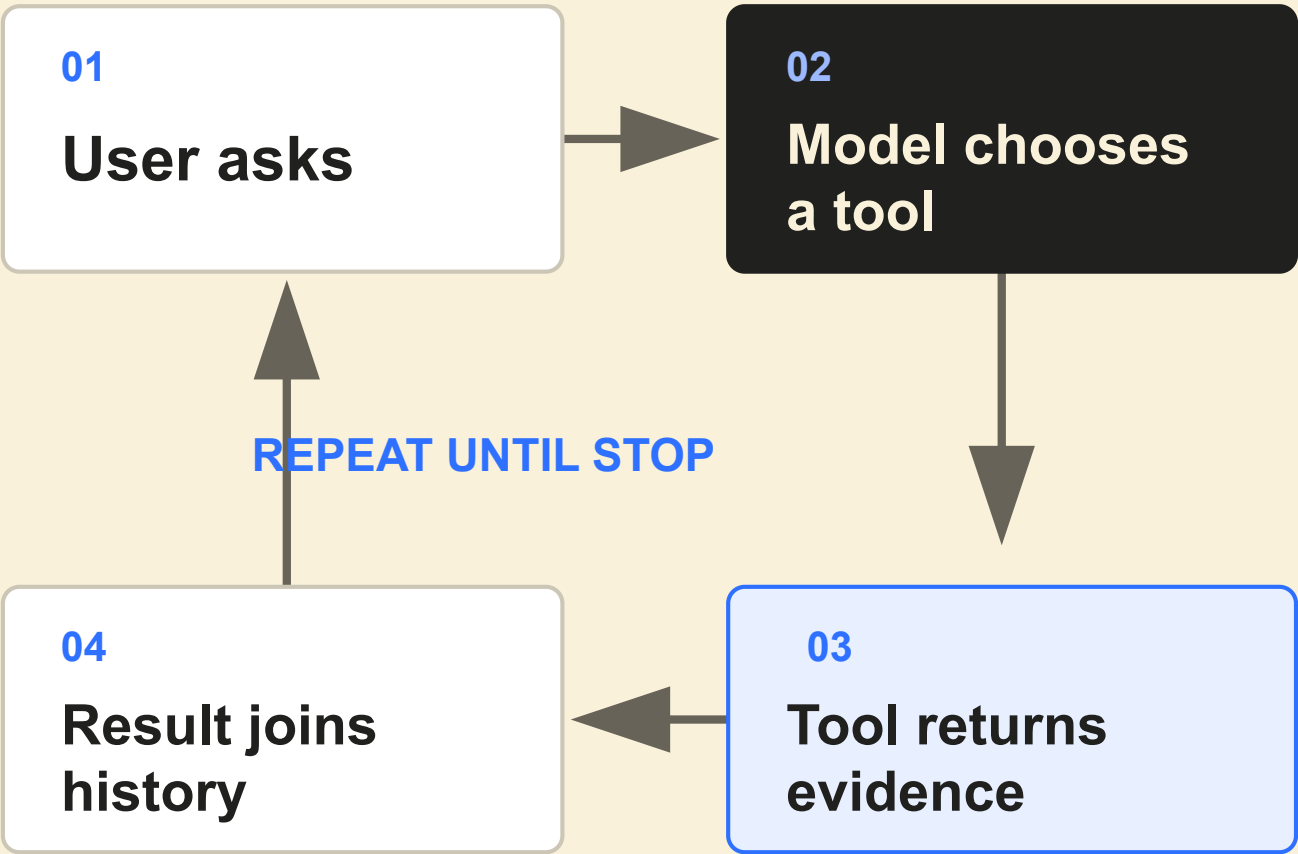
The caching fix did not establish a correctness gain. Dollar cost was unavailable.

Setting `cacheControlFormat:`
"anthropic" enabled provider cache reads.
Both runs made 89 model calls.

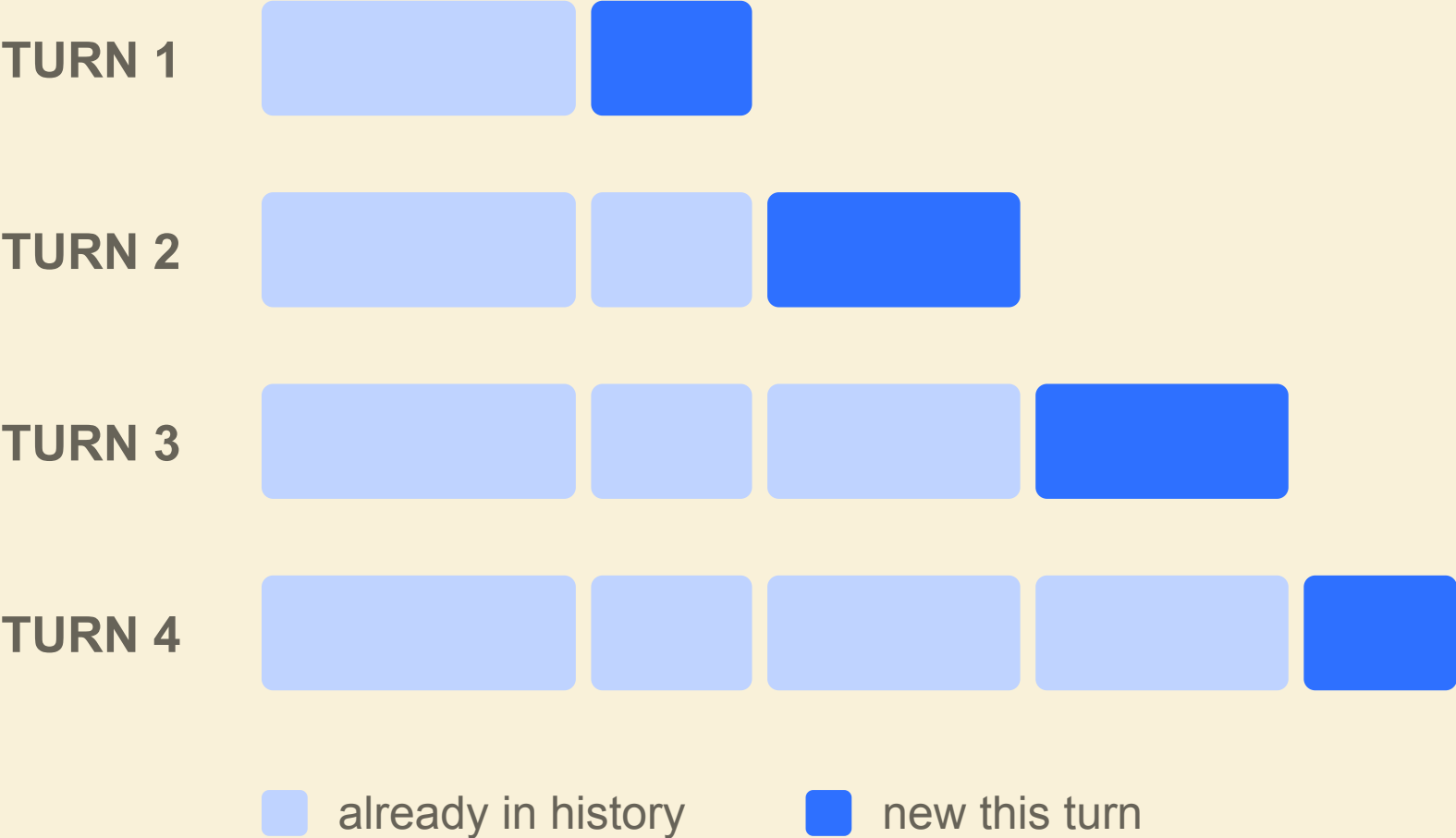
The checks differed, 7/8 versus 6/8. These separate trajectories do not establish a quality effect from caching.

Every agent turn resends the transcript so far

ONE TURN THROUGH THE LOOP



WHAT THE NEXT MODEL CALL RECEIVES

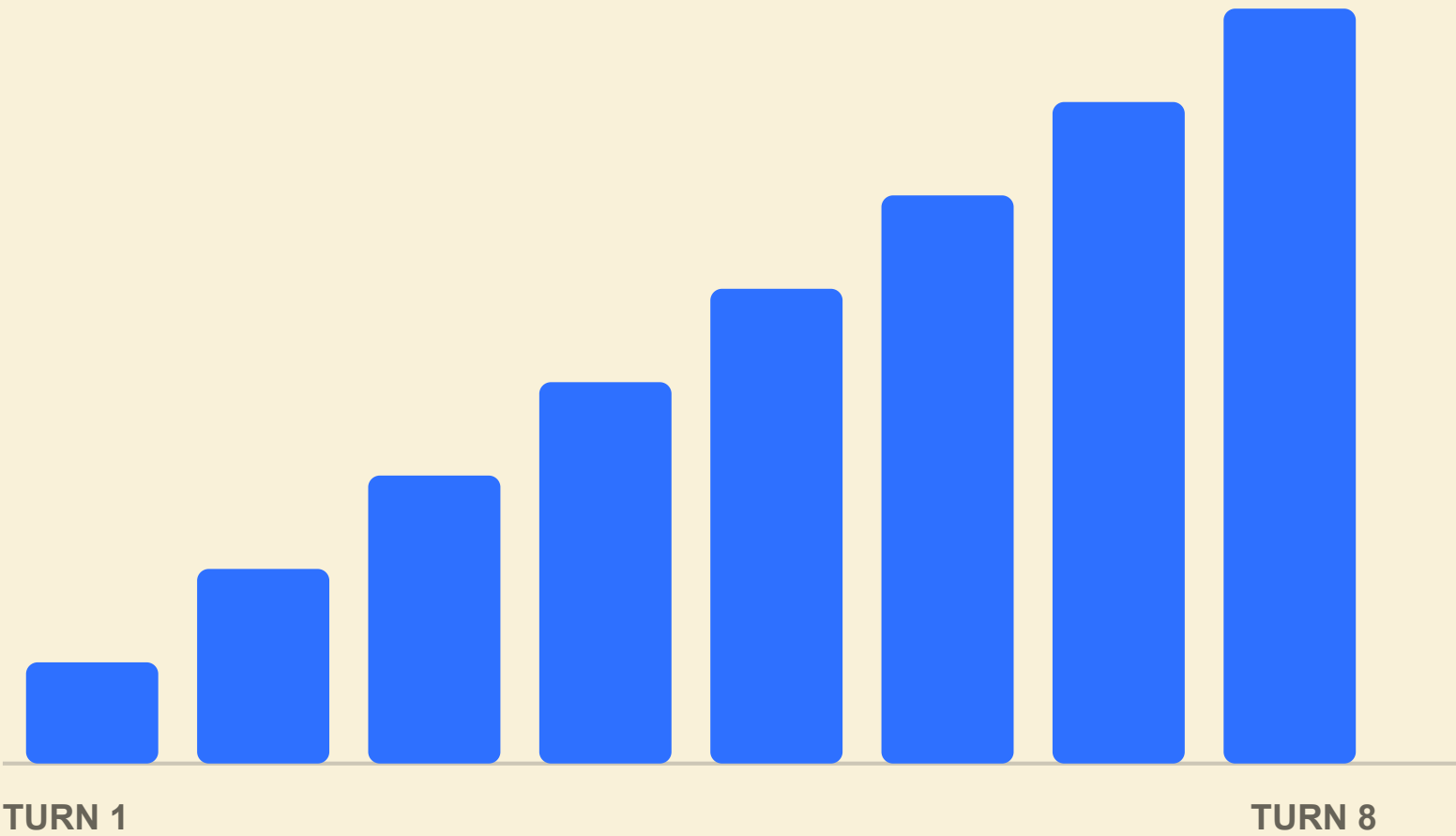


The transcript grows even when the latest tool call is small.

Prefix caching reuses the shared transcript

Without prefix caching

FRESH PREFILL WORK

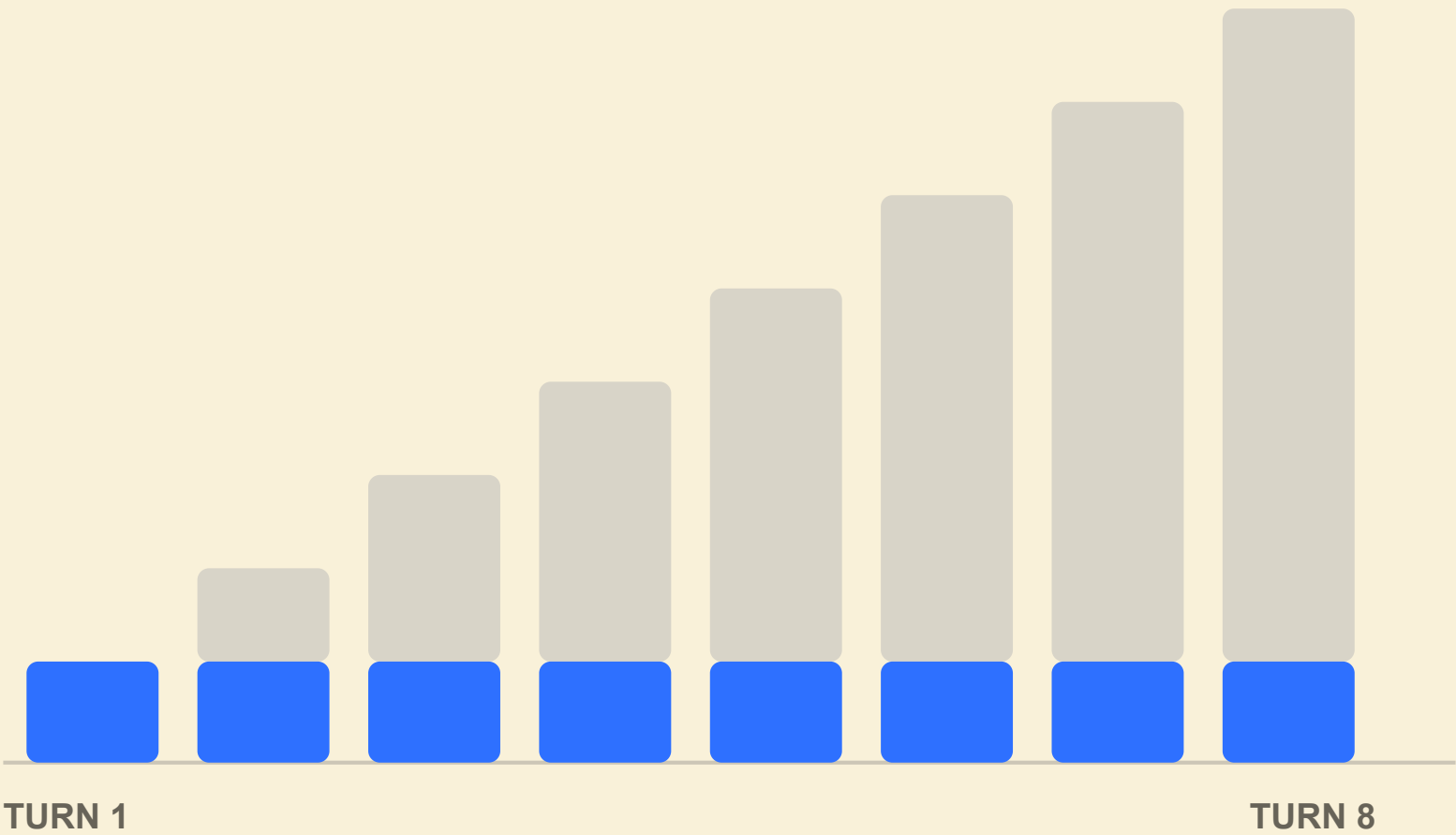


100 turns: $1 + 2 + \dots + 100 = 5,050$ fresh units

Fresh prefill: 5,050 → 100 units, a 50.5× ratio

With prefix caching

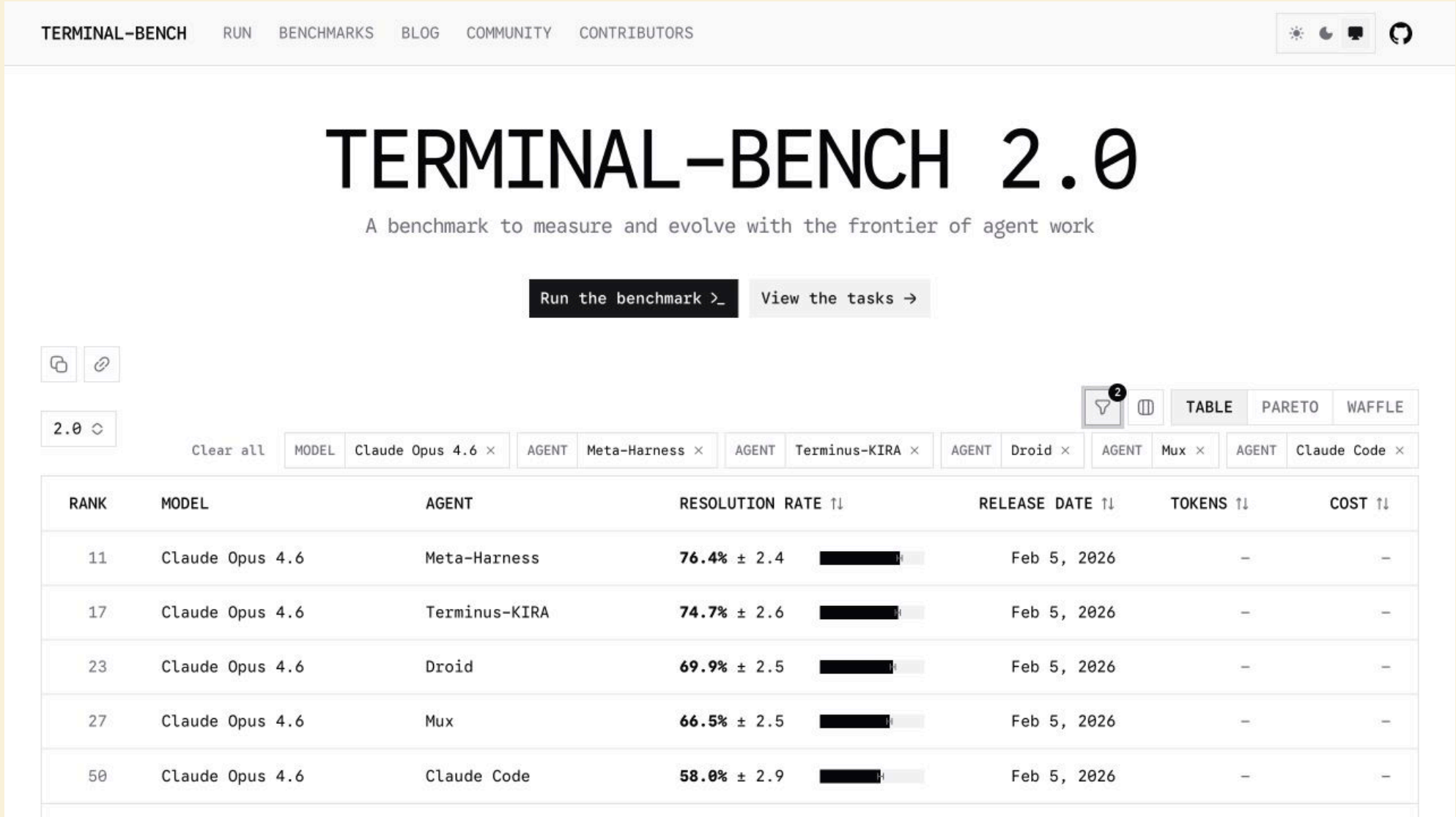
CACHED PREFIX + NEW TAIL



cache read fresh tail

100 turns: $100 \times 1 = 100$ fresh units

An 18-point spread with the same model



Harnesses varied on accuracy, time, and token use

Completion score alone misses differences in process quality, time, and token use.

Harness	Score(%)	Comp.(%)	Secur.(%)	Process			Efficiency	
				Tool(%)	Cons.(%)	Rob.(%)	Tok.(K)	Turns
OpenClaw	52.4	60.0	100.0	79.5	74.0	70.9	82.1	5.0
NanoBot	76.2	81.6	100.0	93.8	93.7	91.7	68.7	7.3
Hermes	71.2	80.4	100.0	88.5	88.4	85.5	139.7	22.6
ZeroClaw	61.4	69.9	100.0	84.1	83.2	79.0	133.2	8.6
NullClaw	64.4	75.9	100.0	85.3	81.4	78.3	175.1	12.1
Moltis	68.8	78.4	100.0	86.3	87.3	84.1	134.9	8.0

No harness led on every measure

OpenBench: gpt-5.6 across 7 harnesses

KIND RELEASE DATE 2026-07-21 DENOMINATORS MATCHED (TASK, TRIAL) MATCHED ROWS 294 COMMON TASK/TRIALS 42 RESULTS SHA 546F167576AA
TASK-SET SHA D949D46E8AAE

gpt-5.6-sol caveats disclosed

+ 1 caveat(s) from the release page

HARNESS ↓	SOLVE RATE · WILSON 95% ↓			SOLVED ↓	MEDIAN WALL ↓	FRESH TOKENS/SOLVE ↓	UNCACHED INPUT/SOLVE ↓	OUTPUT/SOLVE ↓	CACHE-READ/SOLVE ↓
	0	50	100%						
cursor	85.7%			36/42	42.9s	5,519	72	5,446	407,067
devin	81.0%			34/42	68.4s	—	—	—	—
grokbuild	81.0%			34/42	86.3s	46,216	38,727	7,489	368,079
opencode	81.0%			34/42	60.5s	42,634	36,708	5,927	313,916
claude	76.2%			32/42	58.8s	40,255	34,455	5,800	153,488
pi	76.2%			32/42	39.7s	37,602	32,916	4,686	111,408
codex	73.8%			31/42	94.6s	117,107	102,174	14,933	1,164,511

DeepSeek favored a simpler harness in this comparison

Harness configuration	DeepSWE v1.1 · resolved %	Terminal-Bench 2.1 · pass@1 %
Claude Code	69.8	88.0
Codex	65.6	84.1
OpenCode	65.5	85.0
Pi	66.2	86.1
mini-SWE	74.2	90.3
DSH Minimal	72.6	90.6
DSH Standard	70.5	85.8
DSH PTC	67.6	85.8

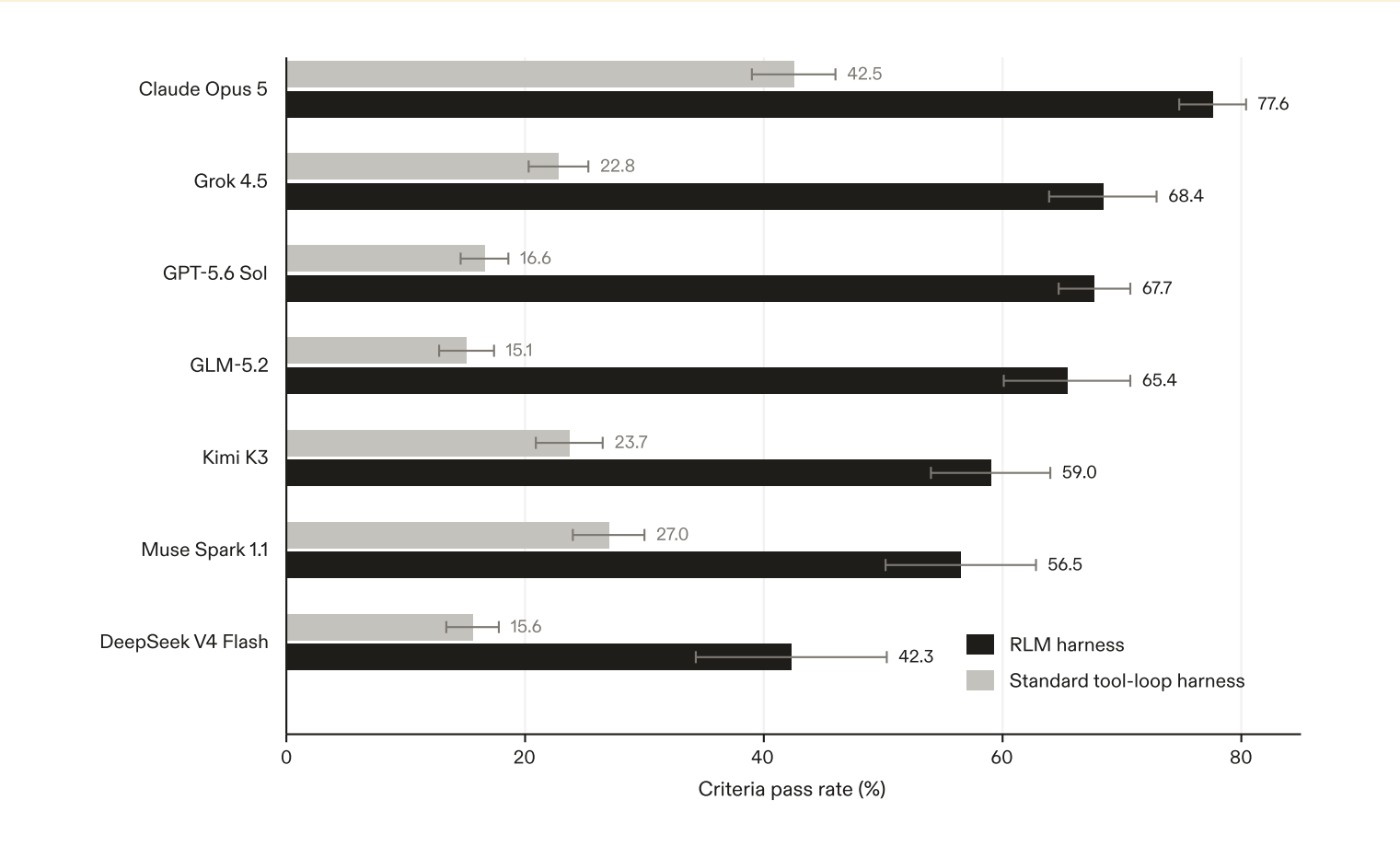
Reported V4.1 Flash results, transcribed from the supplied screenshot. Primary report and settings pending verification.

Supplied @zainhas screenshot, September 10, 2026. Primary release report pending verification.

Snowflake's CoCo scored higher at lower cost

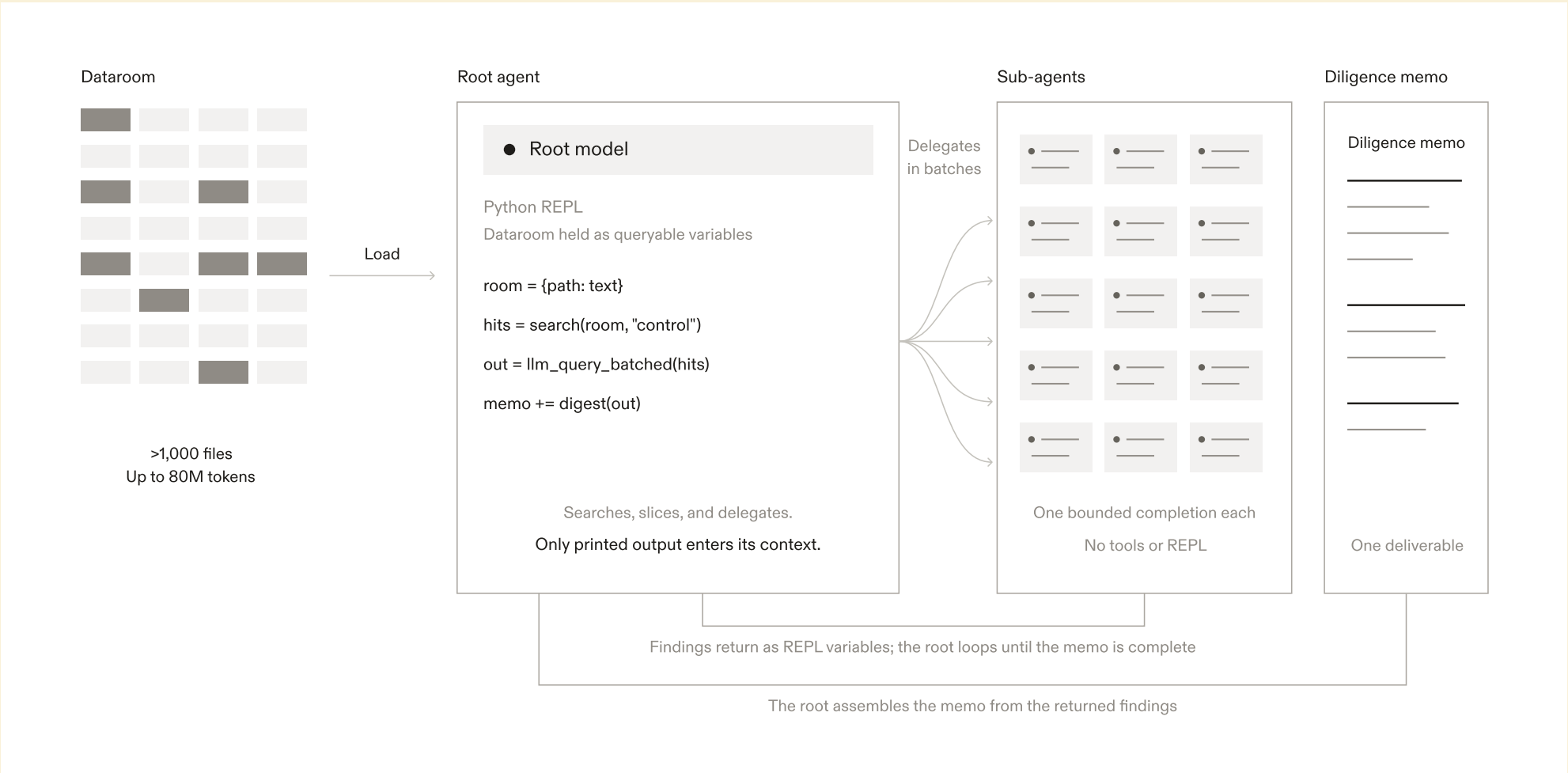


Harvey reported 23.3% to 62.4% across two review setups



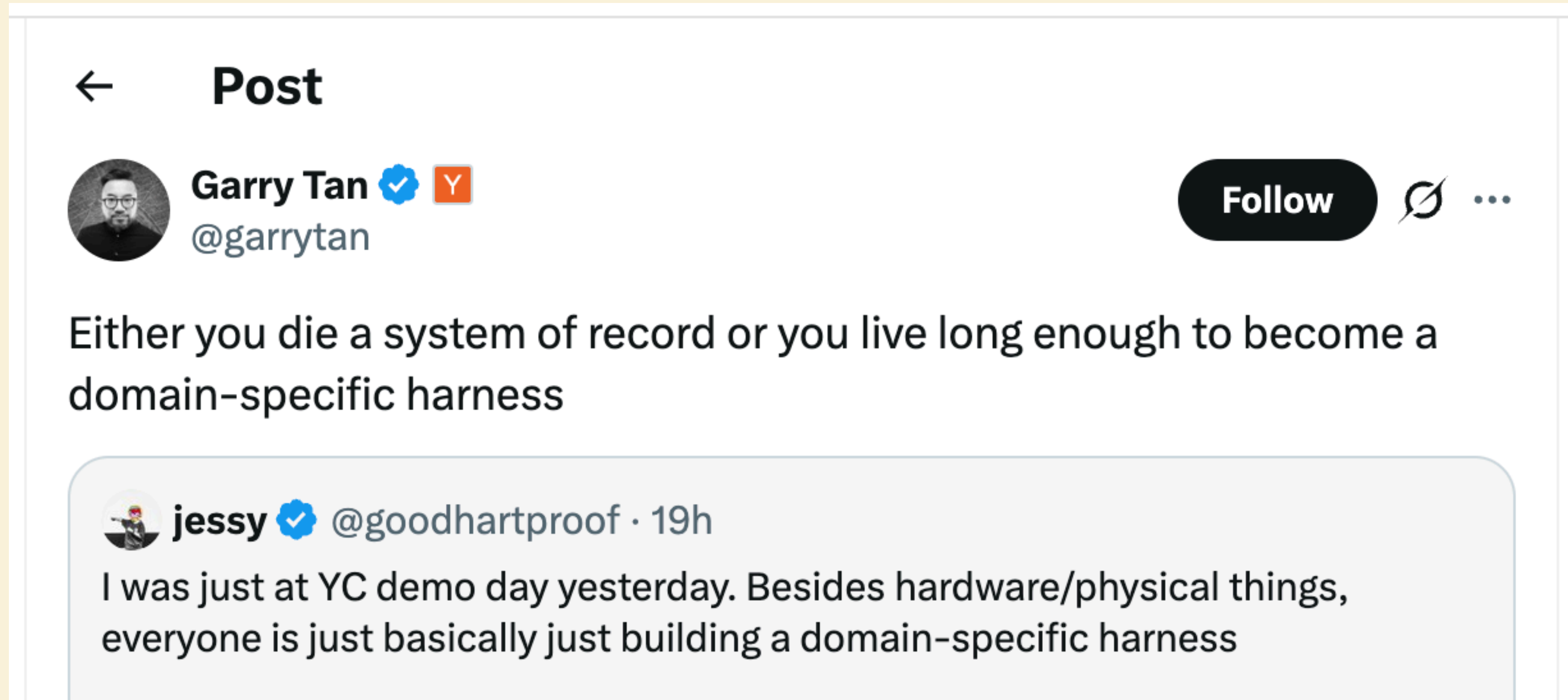
Harvey used a Python REPL and bounded subagents

Bounded subagents review documents through a Python REPL and return selected evidence to the root agent.

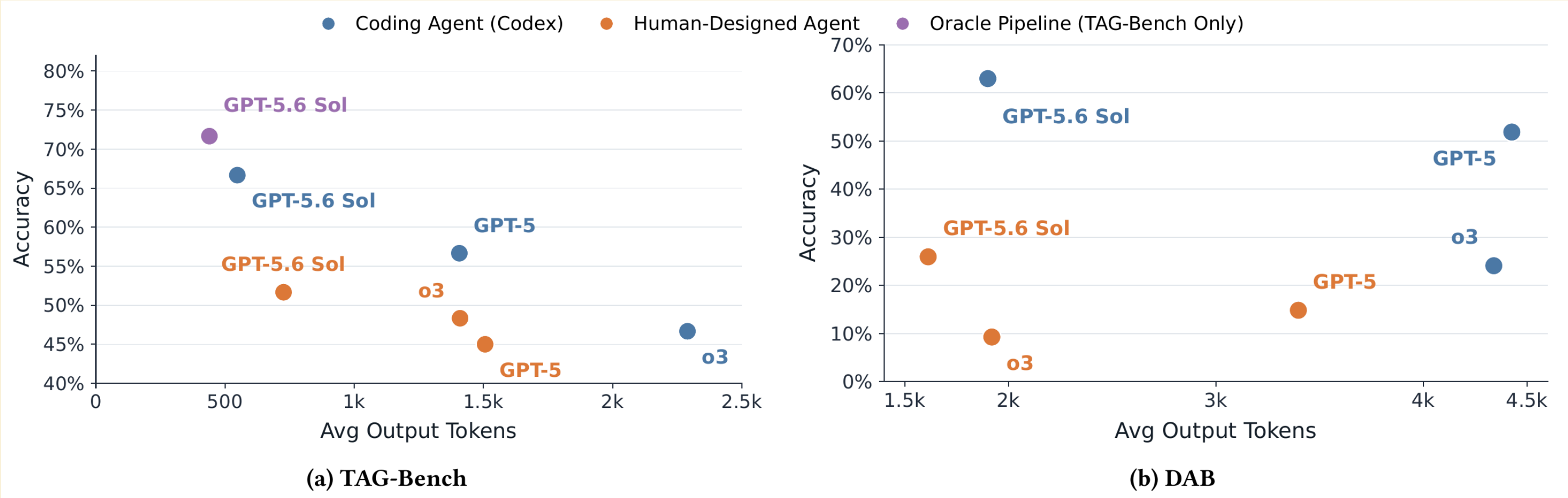


Harvey and Baseten, Figure 5.

“Everyone is building a domain-specific harness”



A stronger model erased the hand-built agent's advantage



Patel et al., What Happens When the Model Eats the Stack?, Figure 1. TAG-Bench uses a partial reconstruction of the custom agent.

Harness changes can reduce quality

Lower reasoning effort, lost thinking blocks, and shorter-response instructions each featured in Anthropic's quality postmortem.



Open-source harnesses offer different starting points

Existing projects offer different tool interfaces, model choices, and extension points.

Harness	Focus
Pi	Minimal, extensible agent loop
OpenCode	Terminal-first coding with provider choice
OpenHands	Autonomous repository work
Aider	Git-native pair programming
Cline	Editor-based tools and control

Examples, not a ranking. Refresh adoption figures before presenting.

Pi's four tools put more work on the model

ONE MODEL-FACING TOOL SURFACE

read

write

edit

bash

SONNET · GUESTS' EXAMPLE

Combines basic actions, runs checks, and continues after a failure

LESS TOOL-CAPABLE MODEL

May write code but stop before testing or reaching the success condition

A smaller tool set saves overhead, but asks more of the model's tool use and persistence.

Different harnesses make different engineering choices

Claude Code vs Codex CLI: Harness & Experience Comparison			
Dimension		 Claude Code Anthropic	 Codex CLI OpenAI
	Primary Emphasis	Polish, safe defaults, integrated experience	Openness, transparency, flexibility
	Core Philosophy	Curated experience with guardrails and conventions	Open and inspectable; user in control of the stack
	Project Instructions	CLAUDE.md (hierarchical, auto-detected)	AGENTS.md (hierarchical, auto-detected)
	Permissions Model	Fine-grained allow/deny for tools, files, and domains (Bash, Edit, WebFetch, MCP, etc.)	Sandbox + approval modes (Suggest, Auto Edit, Full Auto); network off by default
	Sandboxing	Workspace-aware with permission rules and safety checks	Stricter sandbox by default; configurable modes and network control
	Extensibility	MCP support, hooks, subagents, skills	MCP support, config profiles, custom commands; open-source CLI surface
	Observability & Debugging	More productized; less of the internals exposed	Highly inspectable (open source); easy to see prompts, tool calls, and logs
	User Experience	Tight UX, opinionated flows, batteries included	CLI-first, more DIY setup, extremely configurable
	Best For	Teams wanting a secure, polished experience out of the box	Power users/teams wanting transparency, customization, and control
	Common Friction	Permission friction; hidden harness behavior	Sandbox/config complexity; approval semantics can be surprising
	Sweet Spot	Long-lived projects, safety-conscious teams, minimal setup	Automation-heavy workflows, custom integrations, experimentation
	Both are powerful. Claude Code optimizes for a polished, safe experience. Codex CLI optimizes for openness and control.		

Tools and feedback in SWE-agent

The model needs usable tools and informative feedback to choose its next action. That is the next harness decision we will inspect.

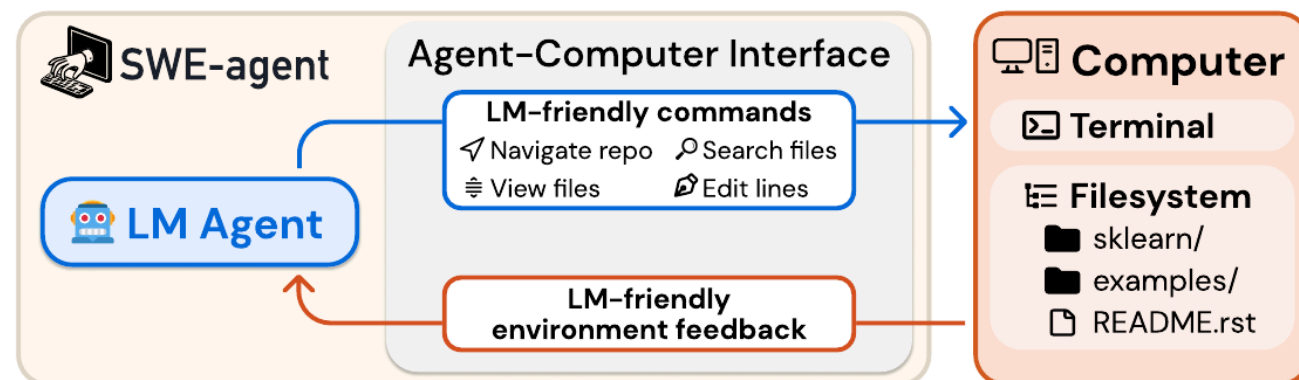


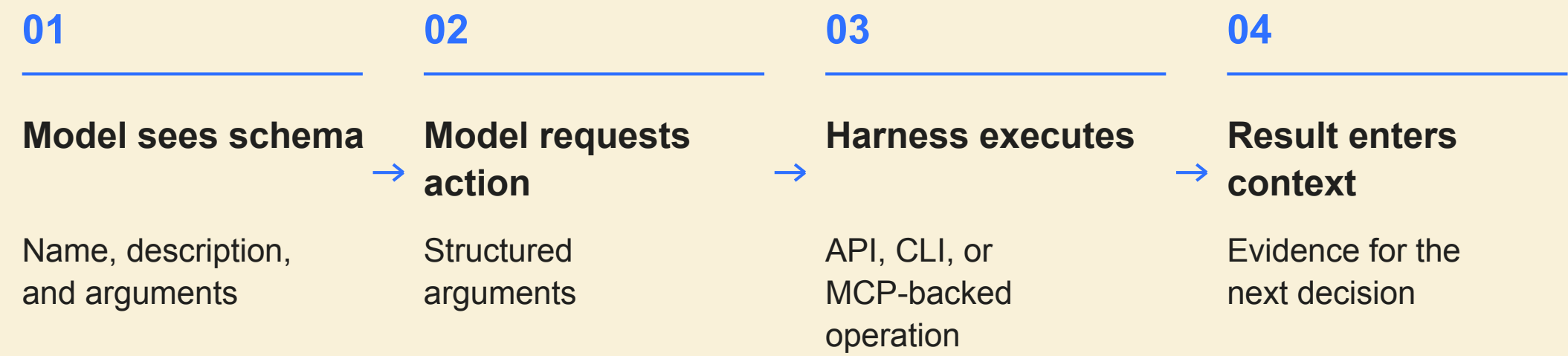
Figure 1: SWE-agent is an LM interacting with a computer through an agent-computer interface (ACI), which includes the commands the agent uses and the format of the feedback from the computer.

Workshop 2: Tool design

02

We will compare what the model can request, what one call can do, and what evidence comes back.

A tool controls the action and the evidence



A tool interface controls both the request the model can make and the evidence it receives.

A tool controls both the action the model can request and the evidence returned for its next decision.

Workshop 2: Which tool interface would you give the agent?

ONE PR REVIEW · THREE WAYS TO ASK FOR THE WORK

Terminal + API

Flexible primitives
More steps to assemble

Can handle unfamiliar work

12 GitHub tools

Broad catalog
More definitions to choose

Can call focused operations

One PR-review tool

Task-shaped workflow
Bounded evidence returned

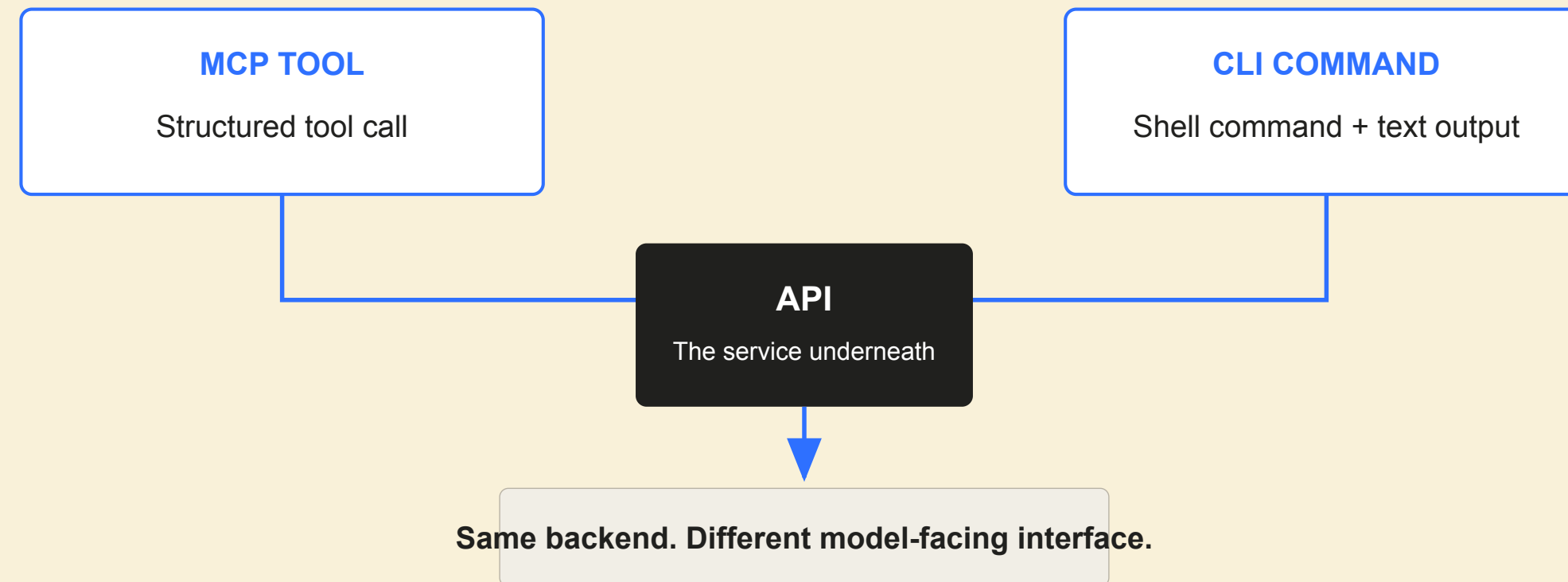
Can be checked in one pass

Which interface will produce the strongest evidence with the least work?

Choose terminal and API access, a broad GitHub tool catalog, or one task-shaped tool.

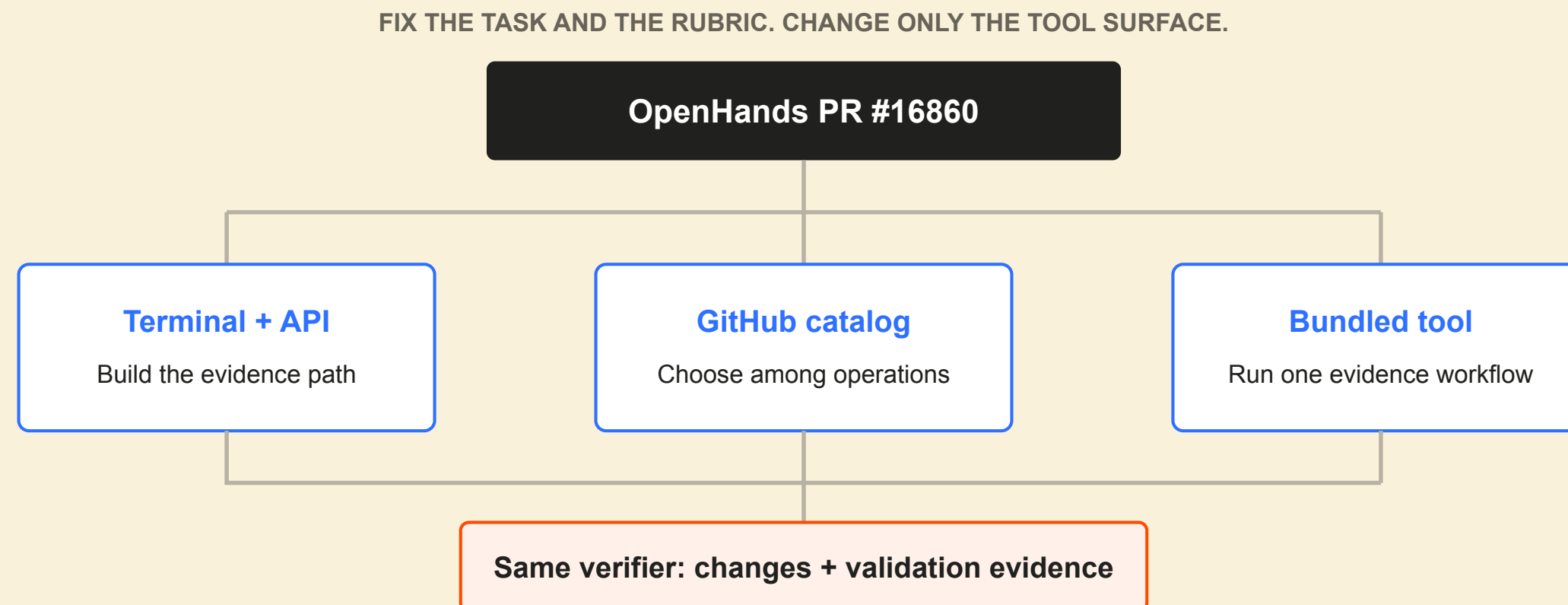
Defend the tradeoff before we reveal the saved runs.

API, MCP, and CLI access



An API defines requests; MCP exposes tools; a CLI exposes commands. Both MCP and CLI can call the same API.

PR review across three tool interfaces



Inspect OpenHands PR #16860 with the same model and rubric. Before the reveal, choose which interface will produce the strongest evidence with the least work.

One task-shaped tool beat the terminal and a broad catalog

Three-trial medians. The bundled tool encoded a recurring evidence workflow. Lower-level operations still support unfamiliar work.

Tool surface	Evidence / 6	Time	Tool calls
Broad GitHub catalog	6 / 6	123 s	12
Terminal + API	4 / 6	101 s	6
Task-shaped tool	6 / 6	72 s	1

Medians of 3 trials per condition. Tool calls can contain different amounts of work.

Common tools in a coding harness

Workshop 2 uses shell and API access, GitHub operations, and one bundled PR-review tool. Real harnesses often expose a broader set.

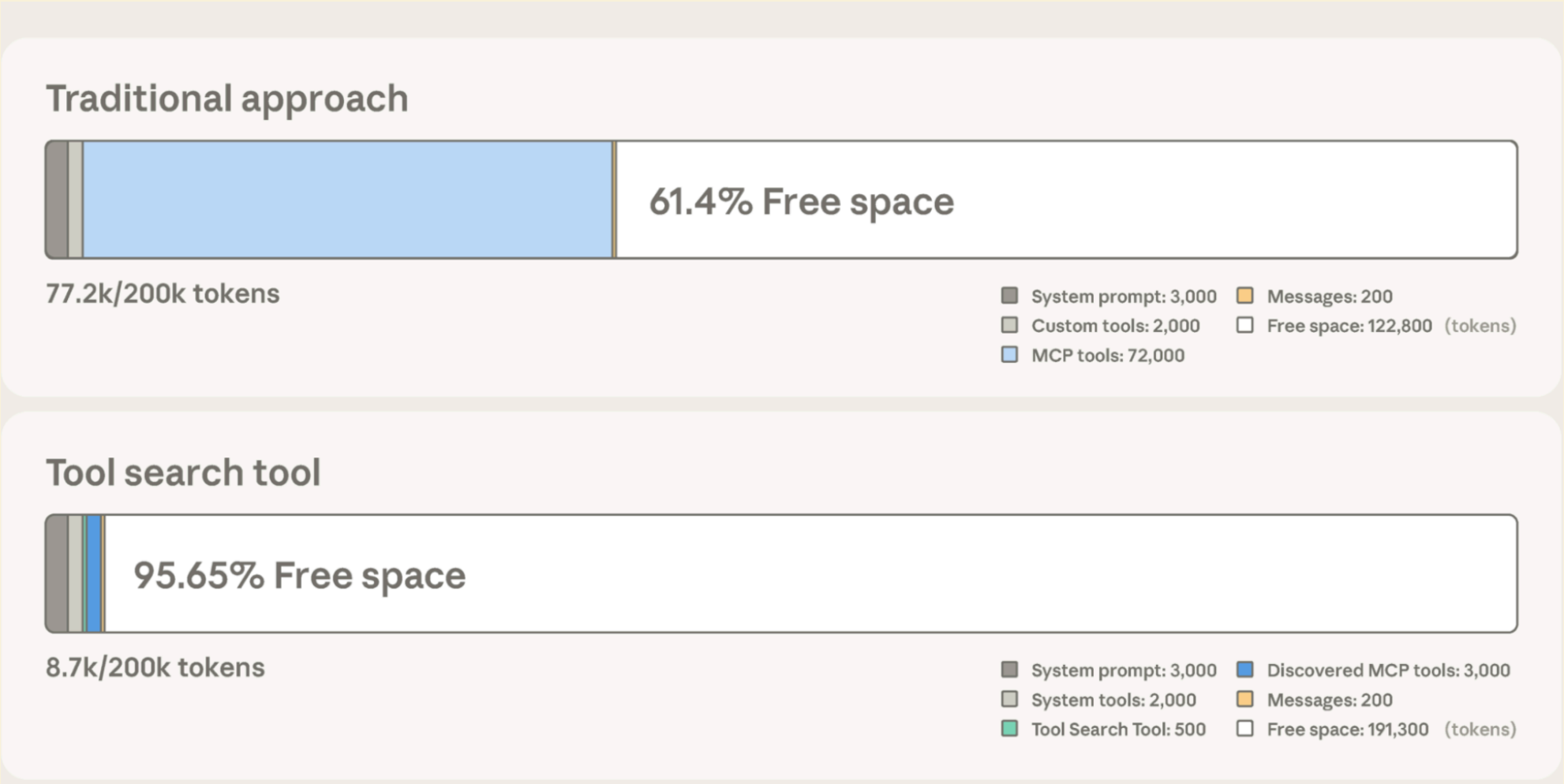
Tool area	Common operations
Files	Find paths, search text, read files
Editing	Write files, replace text, apply patches
Execution	Run shell commands, tests, builds, and background jobs
Git and reviews	Inspect commits, issues, pull requests, and review evidence
Web and interfaces	Search, fetch, browse, click, and inspect images
Services and data	Call APIs, databases, cloud services, and observability systems
Agent support	Search tools, load skills, delegate, and ask the user

A harness exposes the subset needed for its environment and permission policy.

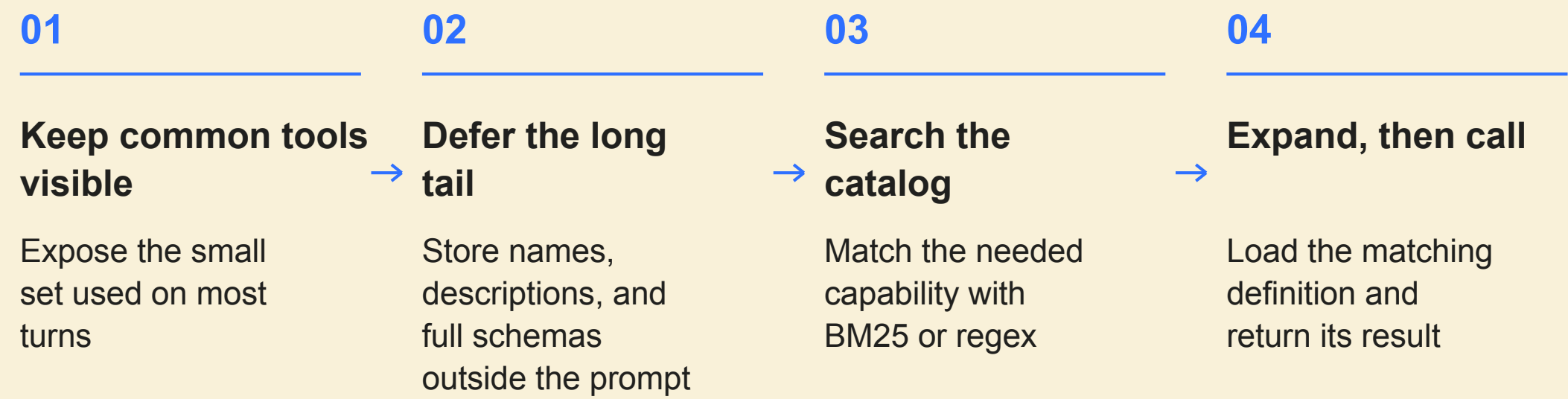
fx built-in tools · broader categories are common examples, not one product's exact inventory.

Tool definitions can fill your context

In Anthropic's five-server example, 58 tools used about 55K tokens before the request. Tool search kept common tools visible and deferred the rest. Initial context fell to about 8.7K tokens in the example.



How to use tool search

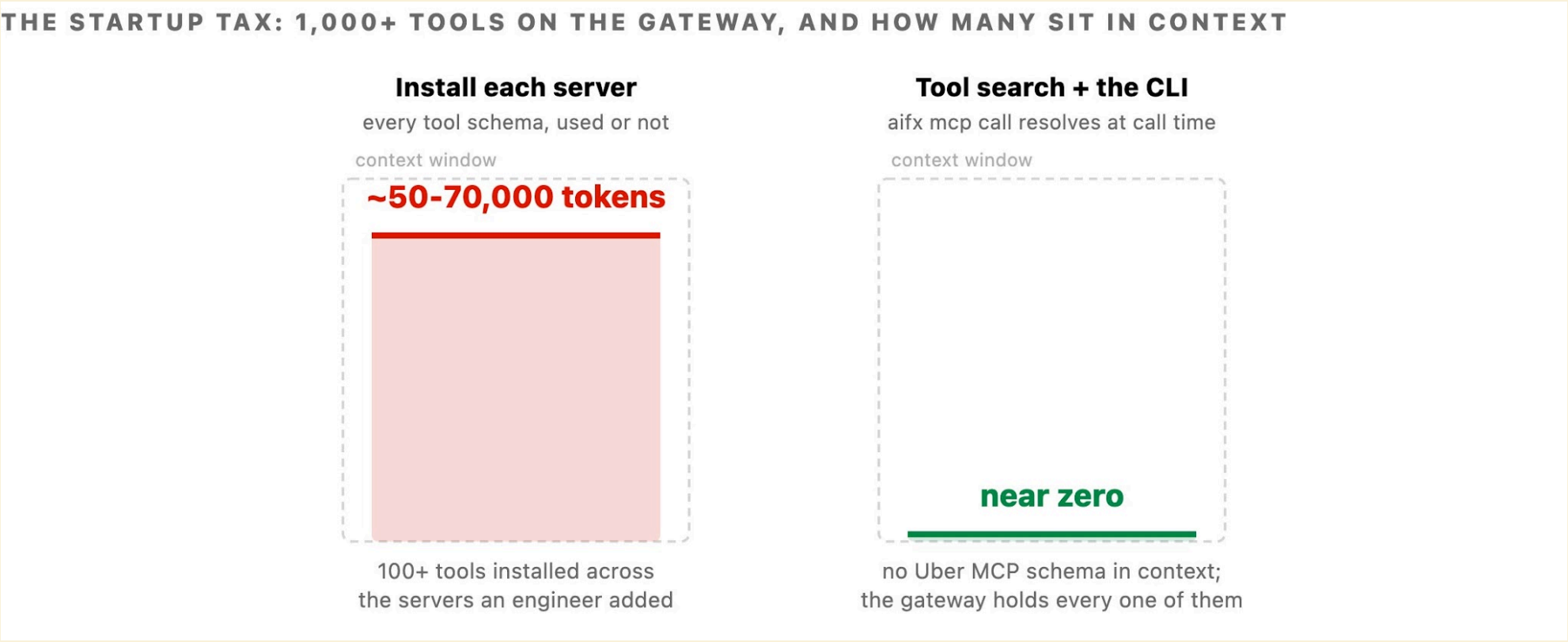


Search changes when a definition enters context. It does not remove authorization, validation, or result handling.

Keep common tools visible. Defer the long tail, search its names and descriptions, then expand the matching definition before the call.

Uber kept 1,000+ MCP tools behind search and the CLI

Loading more than 100 MCP schemas cost 50K to 70K tokens. Uber projected the catalog as CLI commands, so the model could search first and resolve a tool only when it needed to call one.



Uber Engineering, August 27, 2026, Figure 7. Near zero refers to Uber MCP schemas, not all context or discovery overhead.

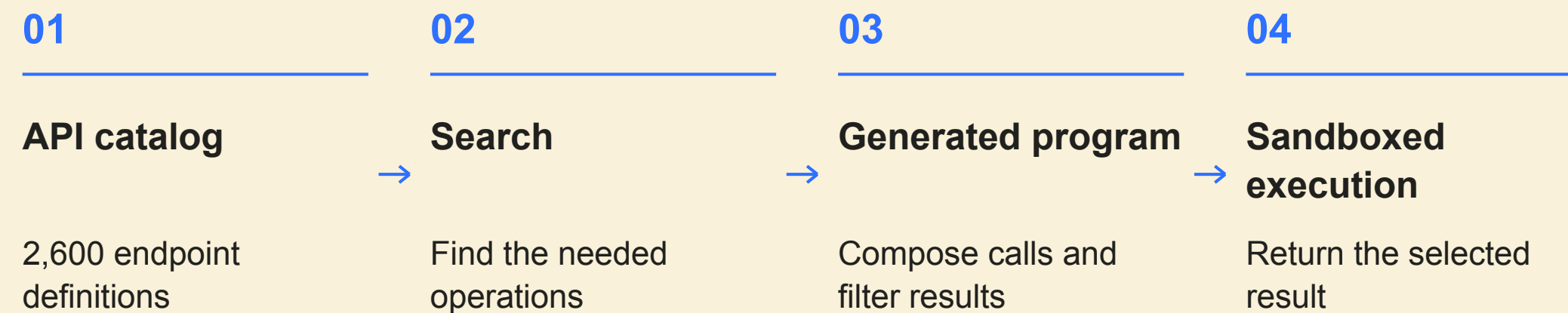
Harness modes in DeepSeek for tools

Minimal mode exposes shell and editor tools. Standard adds a broader built-in set. Code exposes tools through a TypeScript SDK. Creator adds runtime and plugin development.

DeepSeek Harness mode	What the model can use
Minimal	Shell and file editor
Standard	Broader built-in workflow tools
Code	Tools through a TypeScript SDK
Creator	Runtime and plugin development

An extensible platform can expose a small tool surface. Official developer-preview descriptions, September 10, 2026.

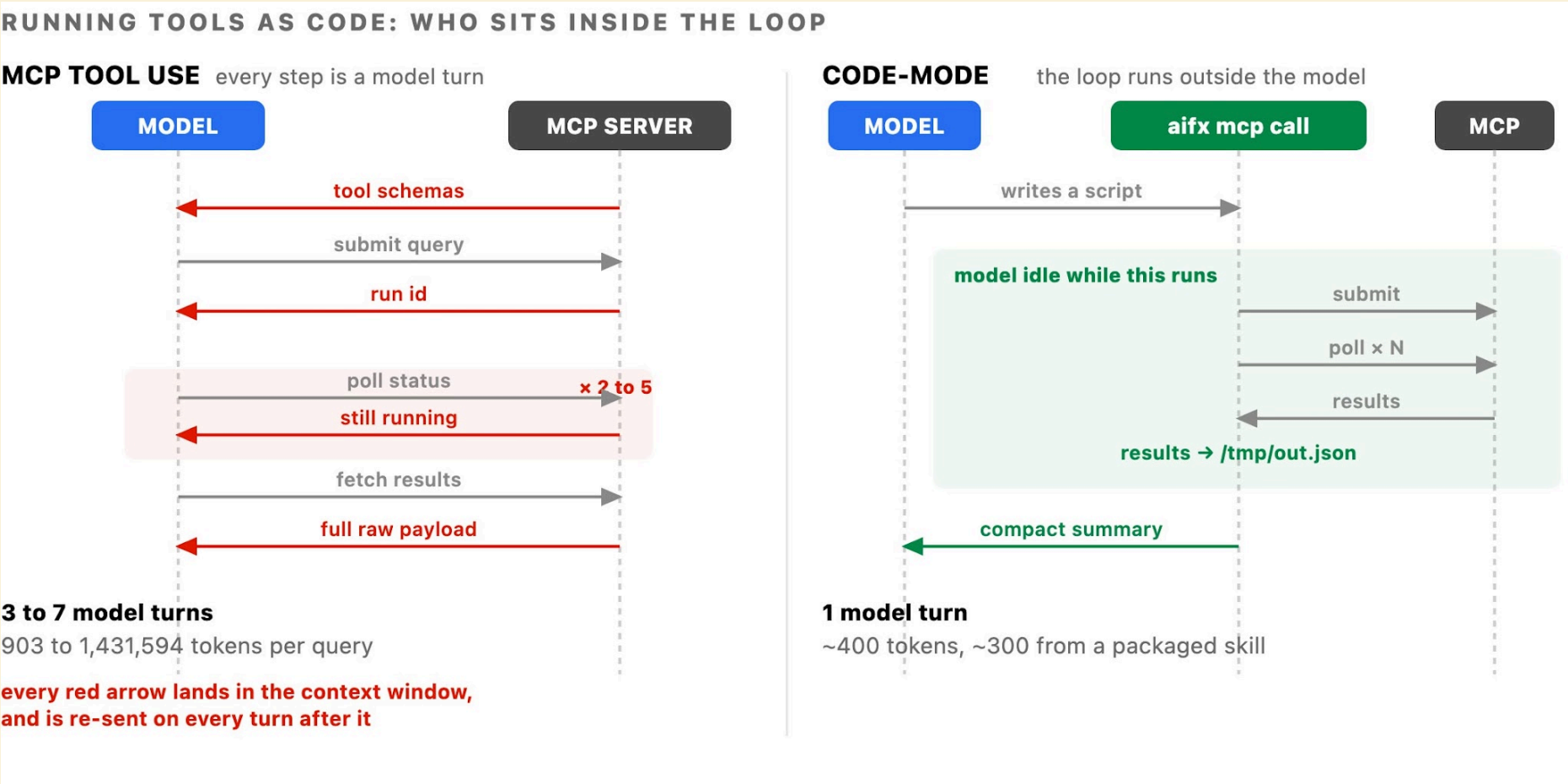
Cloudflare reduced 2,600 API endpoints to two model-facing tools



Cloudflare exposes `search()` and `execute()` instead of one model-facing schema per endpoint.

The model sees `search()` and `execute()`. It searches the OpenAPI catalog, writes a short JavaScript program that can make several calls, and runs that program in a sandboxed V8 isolate. Only the selected result returns.

Uber put SQL polling inside a Python loop



MCP still serves the tools. A Python loop handles query submission, two to five status polls, and result retrieval. Only a compact summary returns to the model.

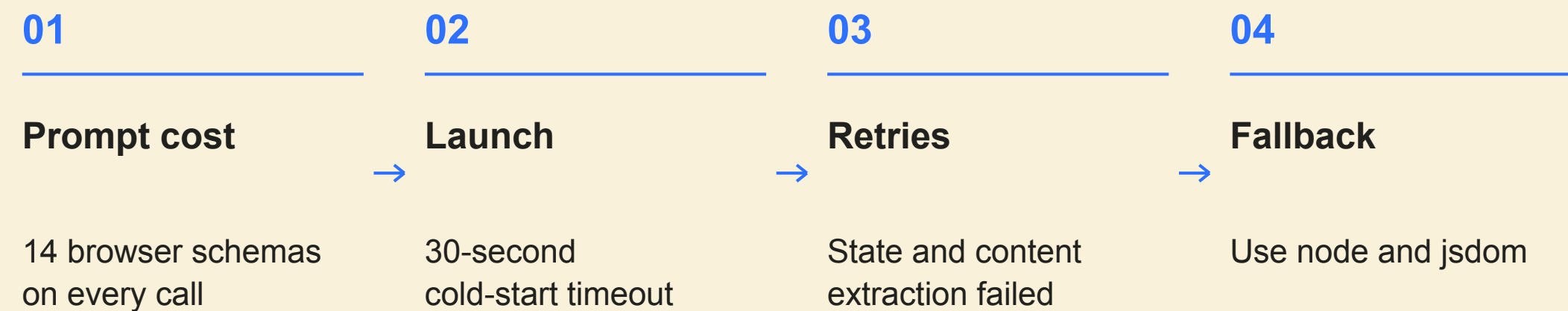
Three ways an agent can describe the same edit

The representation changes how much text the model must reproduce and how the harness recovers from stale context.

Representation	What the model returns	Typical recovery
Whole file	The complete file	Rewrite and inspect the diff
Search and replace	Exact old text and new text	Re-read the match, then retry
Patch	Changed lines with surrounding context	Refresh context and reapply

Same edit, different failure surface. Illustrative comparison.

An unreliable browser created two costs in one run



17 browser actions consumed about 335 seconds, roughly 21% of wall time, in one saved OpenHands run.

The Incident Operations Center run carried 14 browser schemas on every call. Near the end, 17 browser actions hit a 30-second cold-start timeout and repeated extraction failures before the agent fell back to node and jsdom.

A valid tool call can still leave the task unfinished

Correct arguments and a successful response do not establish that the returned evidence answers the task.

Level	Question	Example metric	Execution?	Typical failure
Selection	Right tool?	Precision / recall	No	Missing or extra call
Arguments	Right values?	AST / schema match	No	Wrong field or value
Trajectory	Right order?	Sequence success	Usually	Bad dependency
Task	Goal achieved?	End-to-end success	Yes	Plausible wrong answer

The tool surface should match the repeated work

Bundle a task-shaped tool

The trace repeats

The output can be bounded

A verifier can check it

Keep primitive tools

The next question is unknown

Exploration matters

Exceptions are common

Start with primitives. Bundle the path only after the repeated work is visible.

Start with primitives. Bundle the path after the trace repeats, the output can be bounded, and a verifier can check the result.

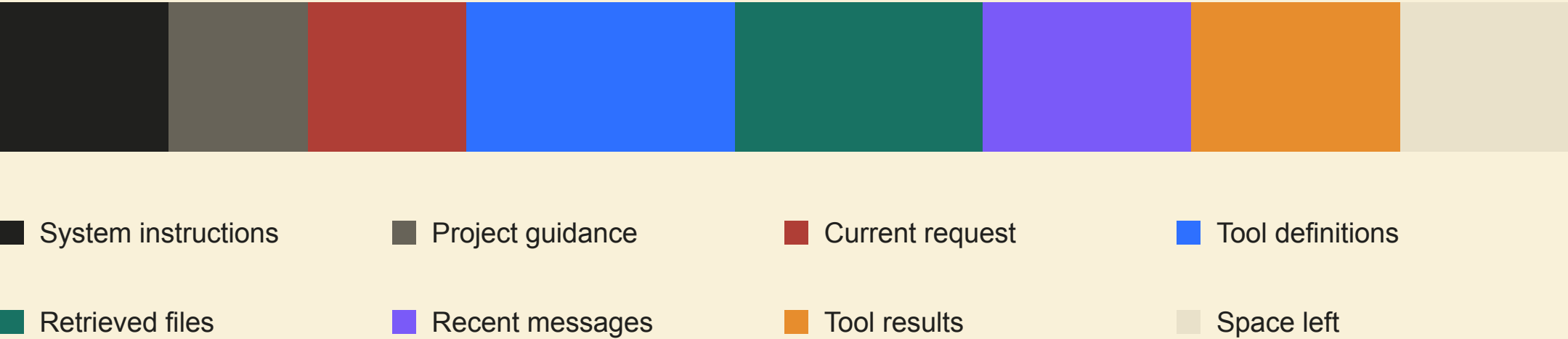
Workshop 3: Context, working state, and memory

03

We will separate what the model sees now from what the system saves for later.

What fills the next model call

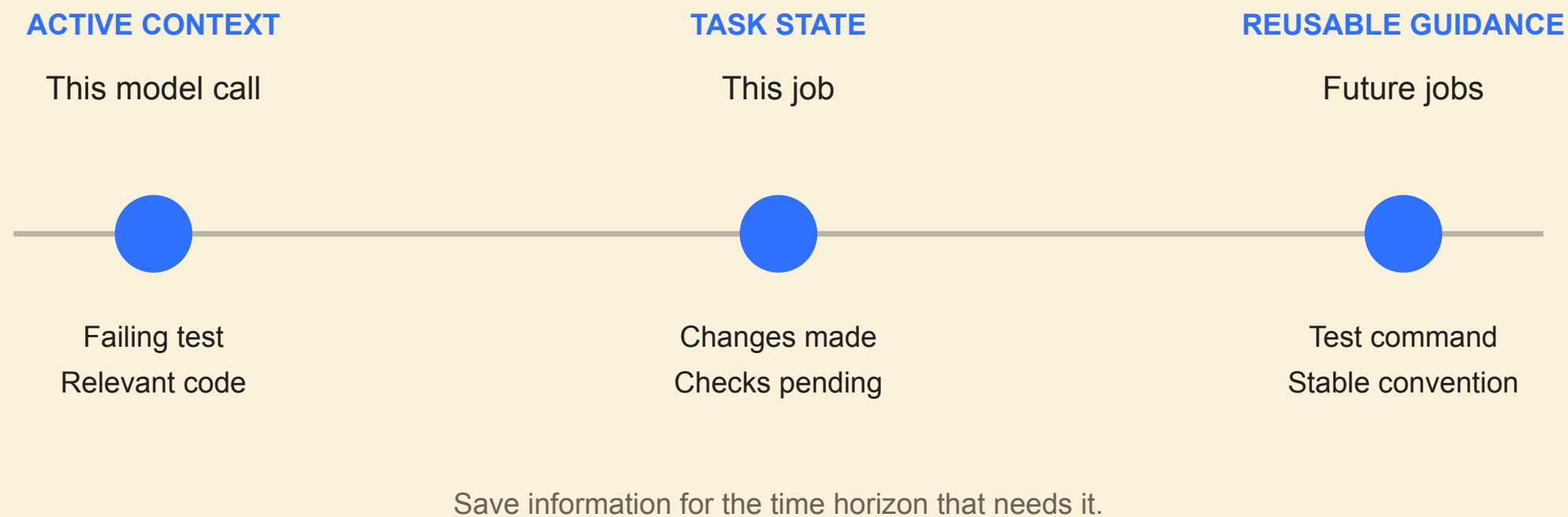
One context window



Illustrative proportions. Only loaded information occupies this model call.

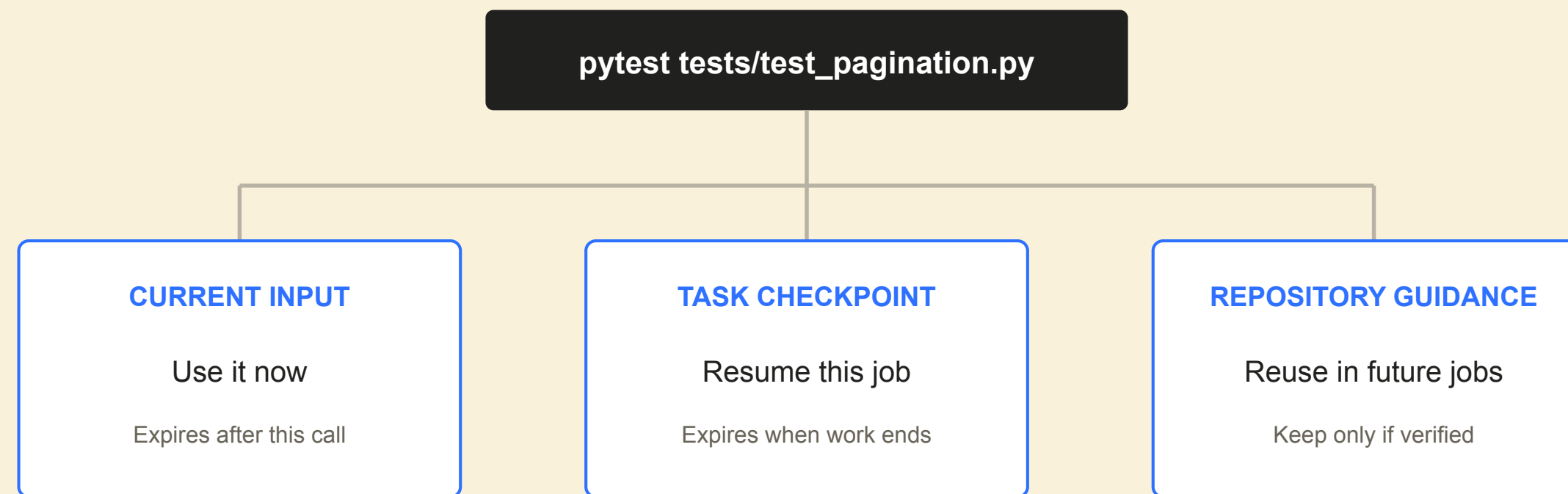
The context window is shared. Instructions, tool definitions, retrieved files, recent messages, and tool results all reduce the space left for the next turn.

Context, task state, and reusable guidance serve different timescales



The failing test belongs in this call, unfinished work belongs with the task, and a verified test command can guide future jobs.

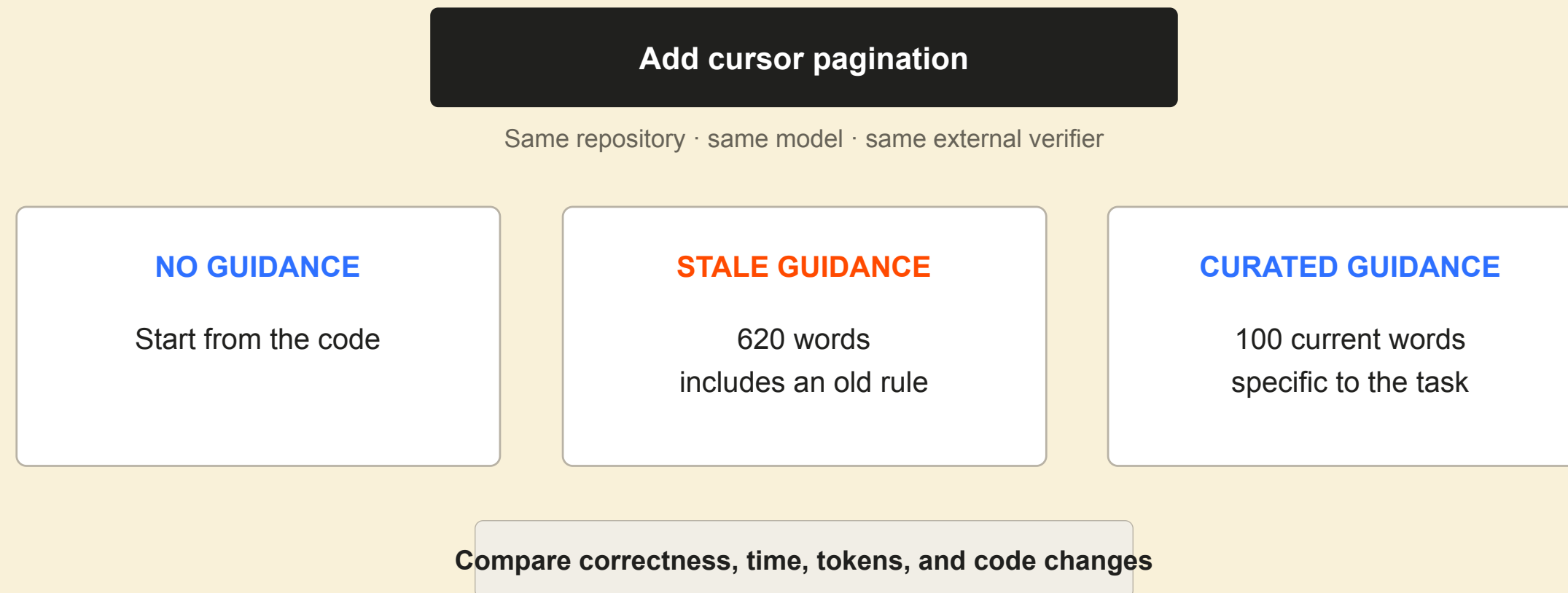
Apply the distinction: where should the test command live?



The same fact belongs in a different place depending on when it must survive.

Use the command in current input now, save it in a checkpoint to resume this job, or promote it to repository guidance after it is verified and reusable.

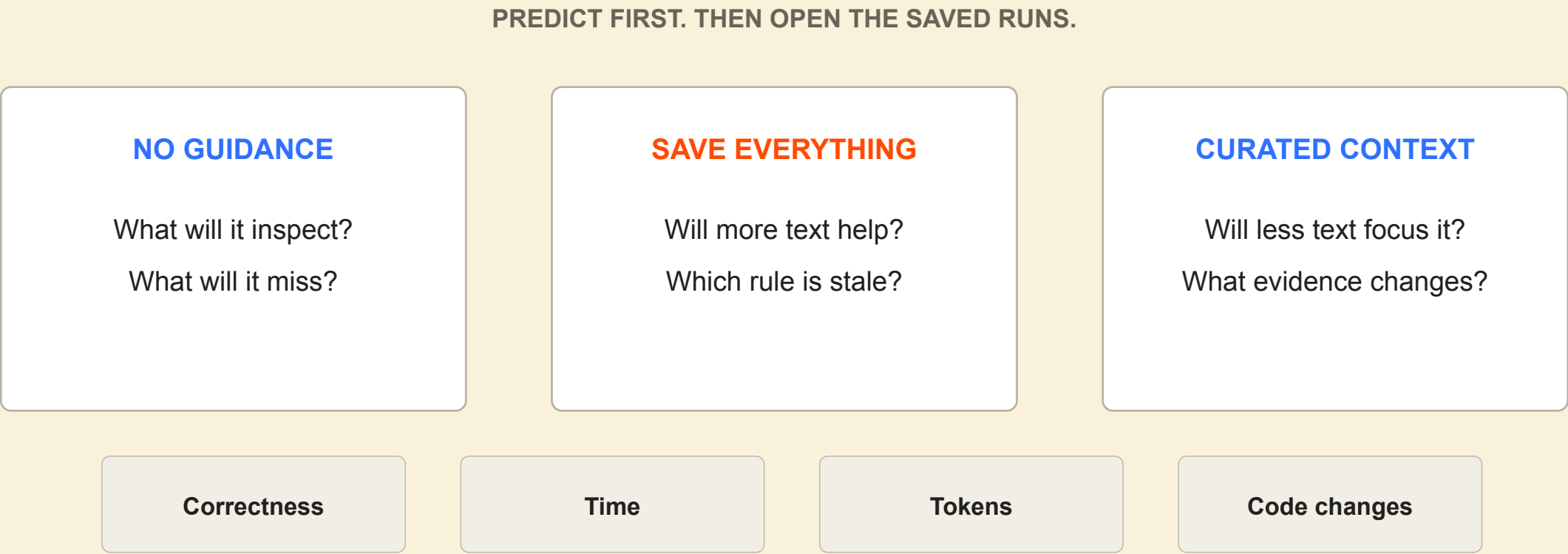
Workshop 3: Add cursor pagination



Add cursor pagination without breaking the existing API.

Compare no guidance, 620 words with a stale rule, and 100 words of current guidance. Same repository, model, and external checks.

Comparing the three context conditions



Compare what each condition tells the agent, then check correctness, time, tokens, and code changes.

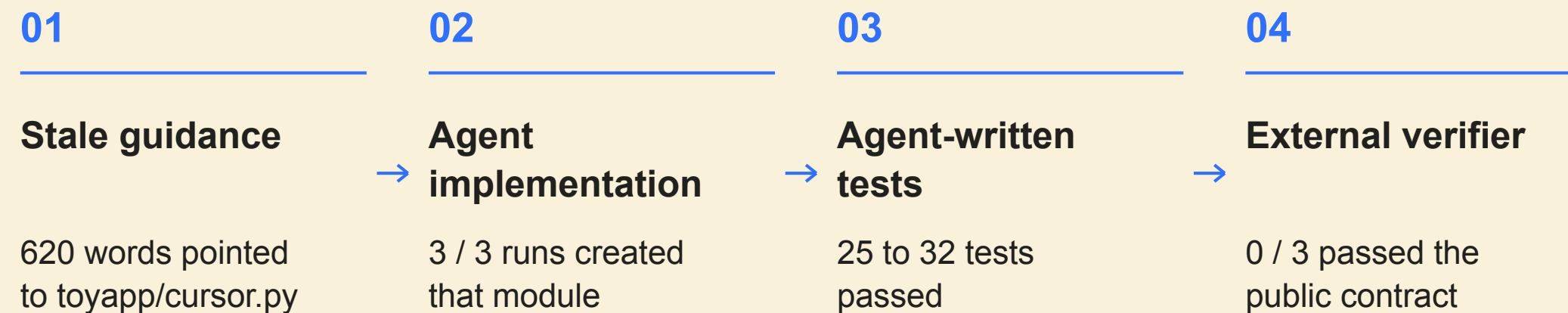
The stale-guidance condition failed all three trials

Three trials per condition. The save-everything guidance deliberately included a stale architecture rule.

Saved guidance	Verifier passes	Time	Processed tokens
None	3 / 3	141 s	283K
Save everything	0 / 3	196 s	414K
Curated	3 / 3	126 s	291K

3 trials per condition. Stale guidance deliberately planted. Time and token figures are medians.

All three stale-guidance runs built the wrong module



Median cost versus no guidance: 39% more time and 46% more processed tokens.

Each run reported 25 to 32 green tests. The external verifier failed all three because `CursorPage` was absent from `toyapp.pagination`.

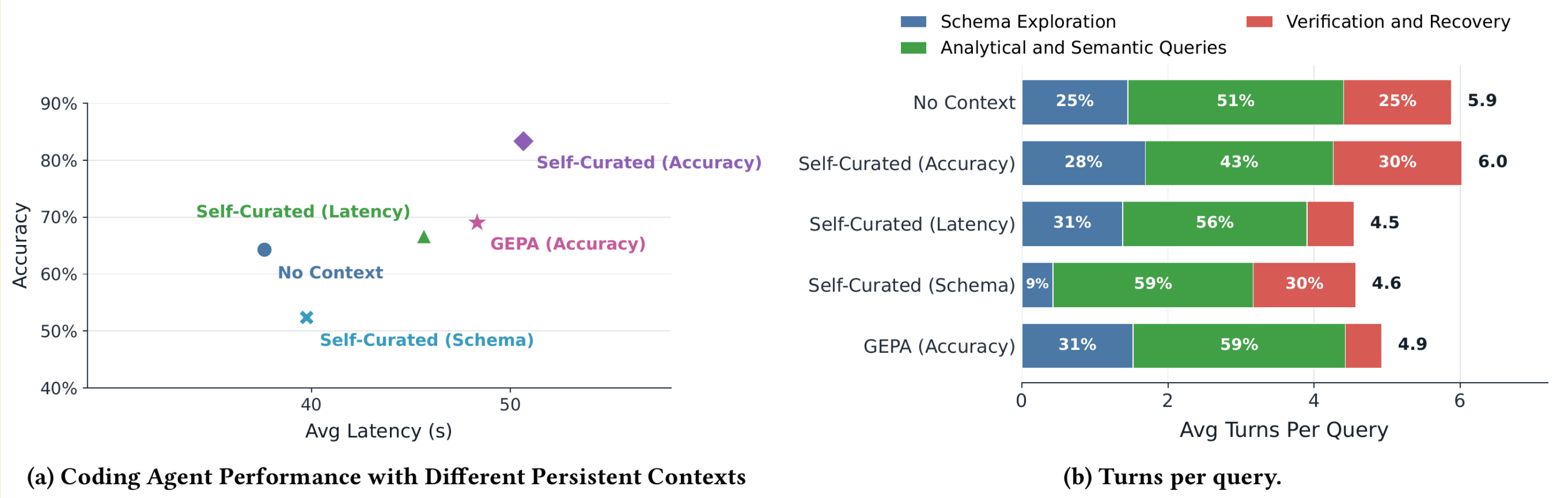
Curated context changed the process, not the pass rate

Both conditions passed the external verifier in all three trials. Curated guidance changed verification behavior without reducing processed tokens.

Condition	External passes	Median time	Processed tokens	Full suite
No guidance	3 / 3	141 s	283K	0 / 3
Curated context	3 / 3	126 s	291K	3 / 3

Elapsed-time ranges overlapped. Curated context used 3% more processed tokens and followed the requested full-suite check.

Providing better context gained 19 points

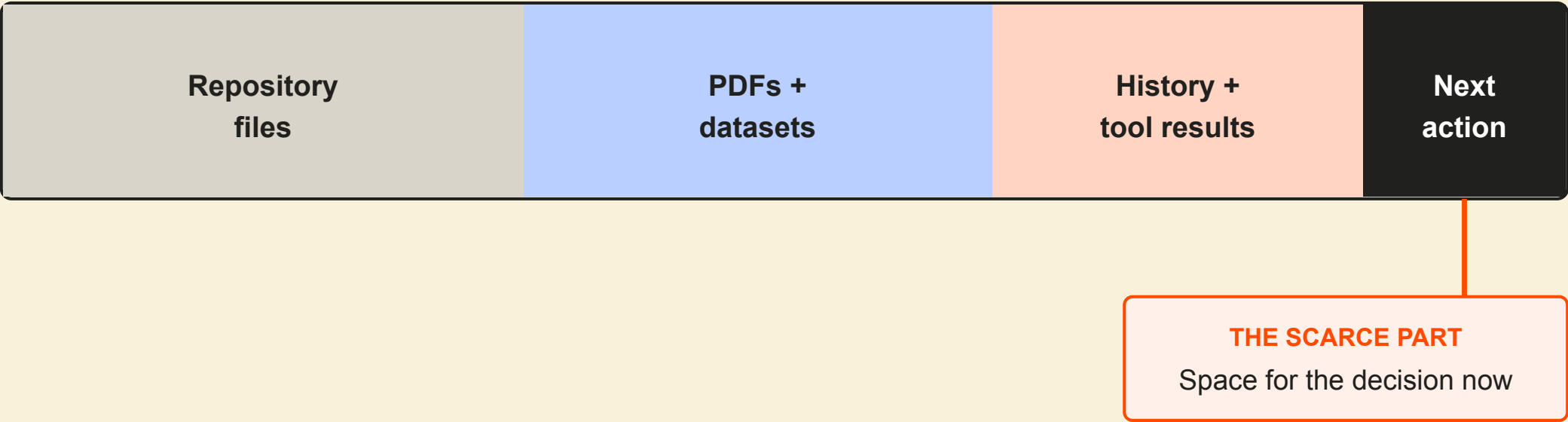


They identified past failures to generate better prompts and instructions to improve the context.

Patel et al., What Happens When the Model Eats the Stack?, Figure 4 and Appendix A.2.

What can fill a long context window

ONE LONG CONTEXT WINDOW

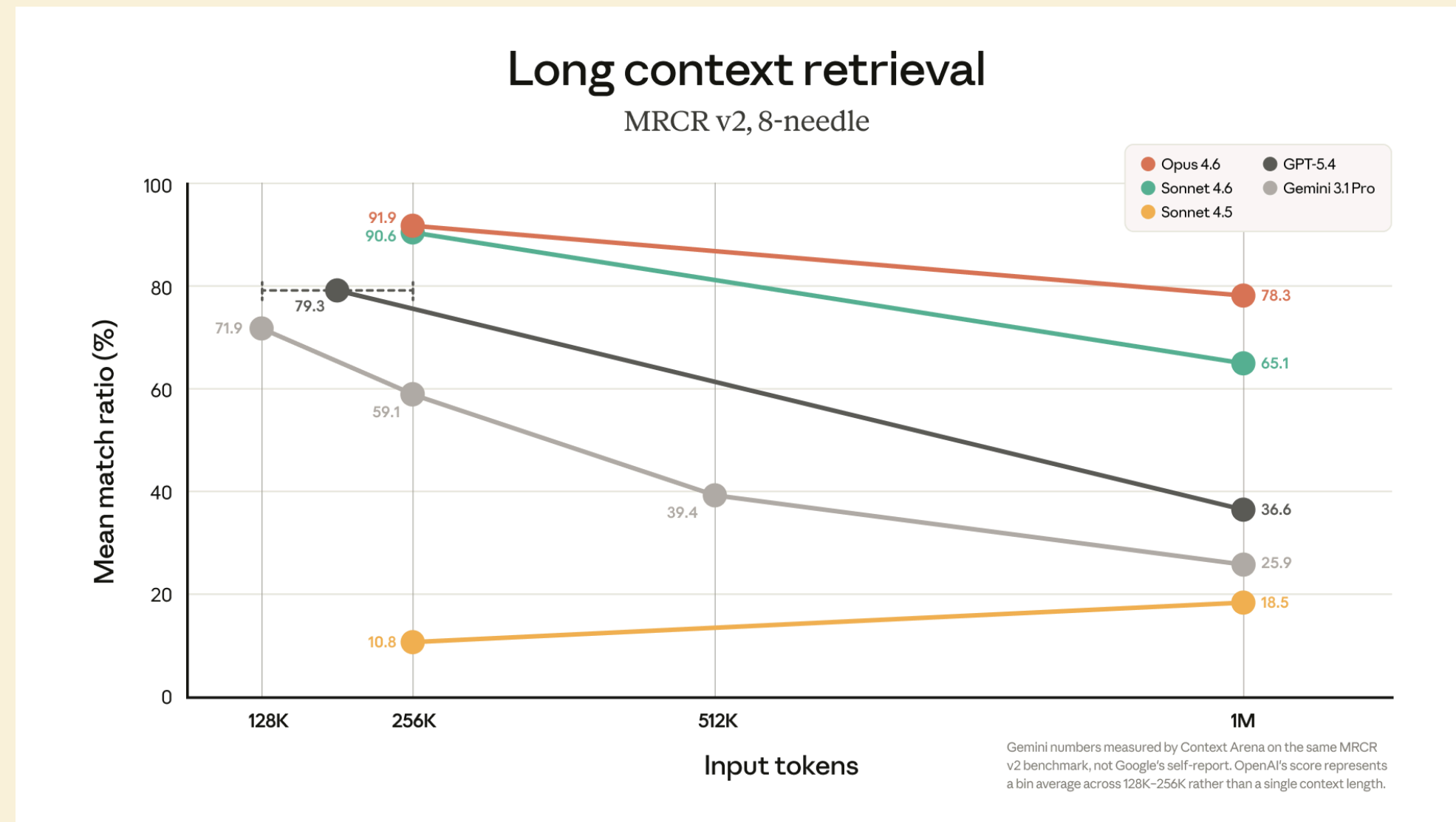


Load the evidence the task needs. Capacity alone does not choose it.

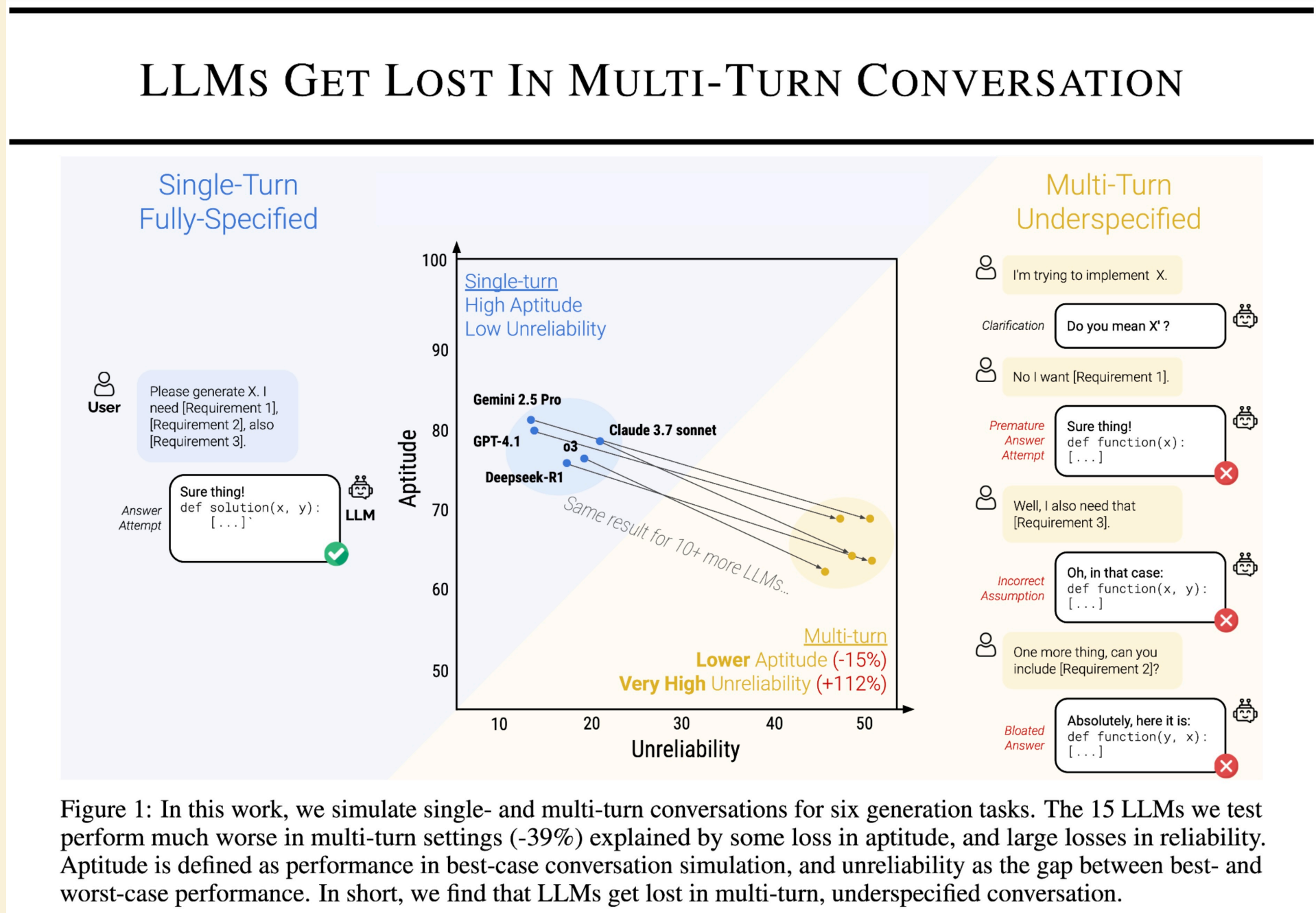
Large PDFs, datasets, and repositories can fill even a long context window. Load what the task needs.

A long context window does not guarantee useful recall

Fitting the input does not guarantee accurate use of every relevant detail.



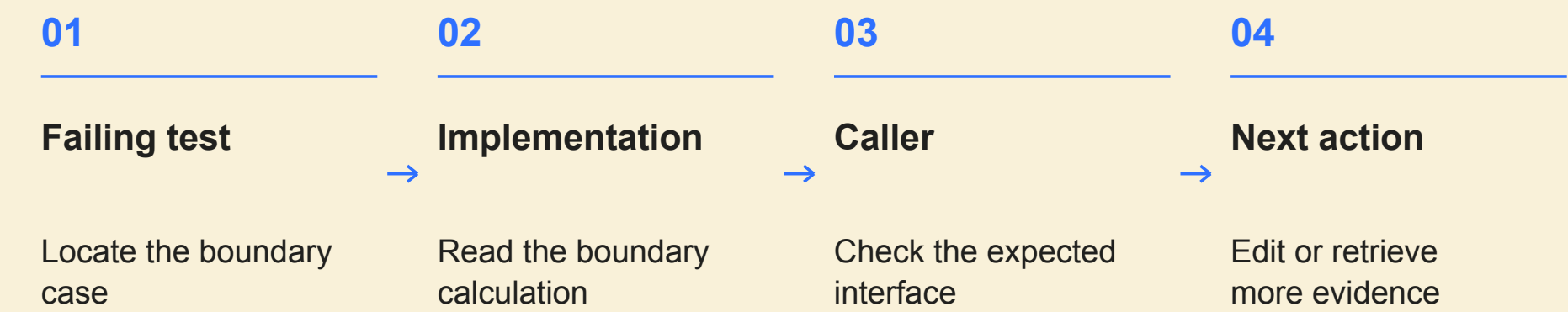
Long conversations can make relevant facts harder to use



Across 15 tested models, performance fell when one fully specified request became a multi-turn, underspecified conversation. Early assumptions made later corrections harder.

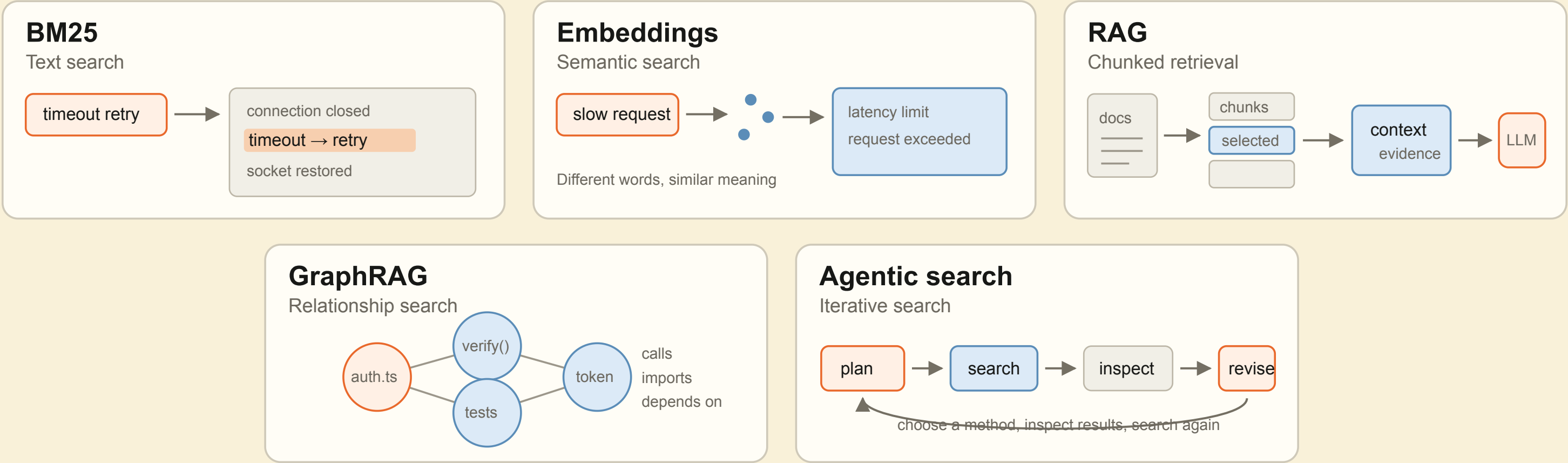
Retrieval for the next action

Retrieve the failing test and relevant implementation; include those excerpts in the next model call.



Illustrative pagination walkthrough. Each result guides the next retrieval.

Five ways to retrieve context

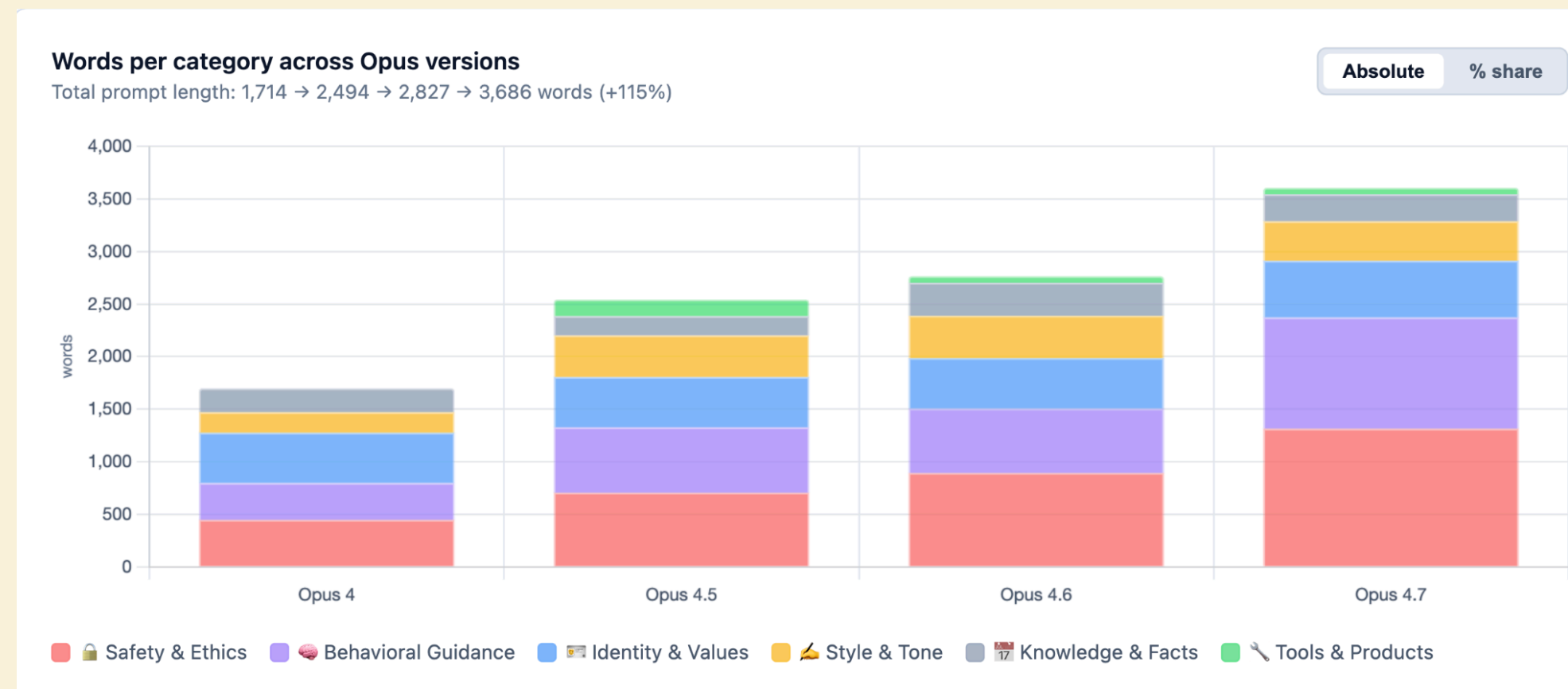


Each method chooses evidence differently. Agentic search can use the other four methods as tools, inspect the results, and search again.

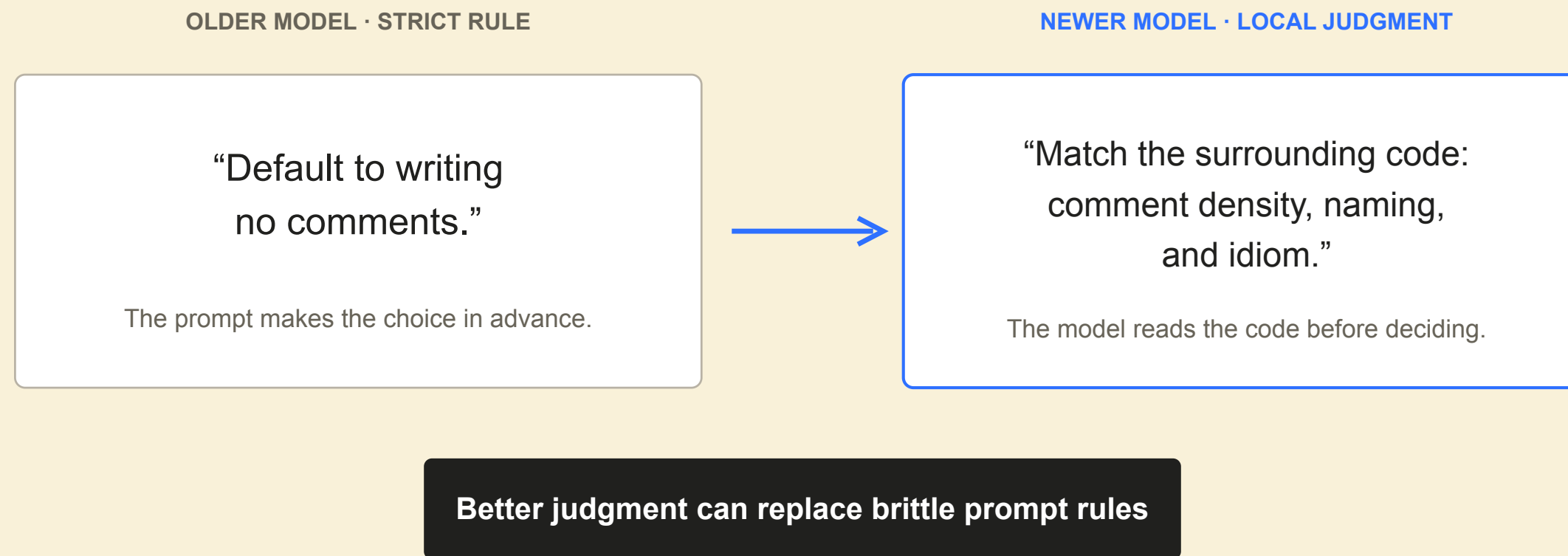
A system prompt gives the agent standing instructions

The system prompt shapes behavior across the conversation. Claude Code uses it for tool guidance, safety rules, and information about the working environment.

The user prompt supplies the current task. CLAUDE.md adds guidance about the project.



New models can replace old rules with judgment



Anthropic says older models needed the strict rule. Newer models could inspect the surrounding code and make the decision.

Anthropic removed over 80% of Claude Code's system prompt

Give Claude Rules	→	Give Claude Judgement
Give Claude Examples	→	Design Interfaces
Put it all upfront	→	Use Progressive Disclosure
Repeat Yourself	→	Simple Tool Descriptions
Memory in Claude.MDs	→	Auto-memory
Simple Specs	→	Rich References

Section headings from the post, in order.

“We removed over 80% of Claude Code’s system prompt ... with no measurable loss on our coding evaluations.”

Thariq Shihpar, Anthropic. The result covered Opus 5 and Fable 5.

LangChain cut the base input from 6K to 2K tokens



System prompt: removed general guidelines and tool-use prose.

Tool definitions: shortened built-in descriptions by 43%.

Planning middleware: made the todo prompt and tool optional after evals.

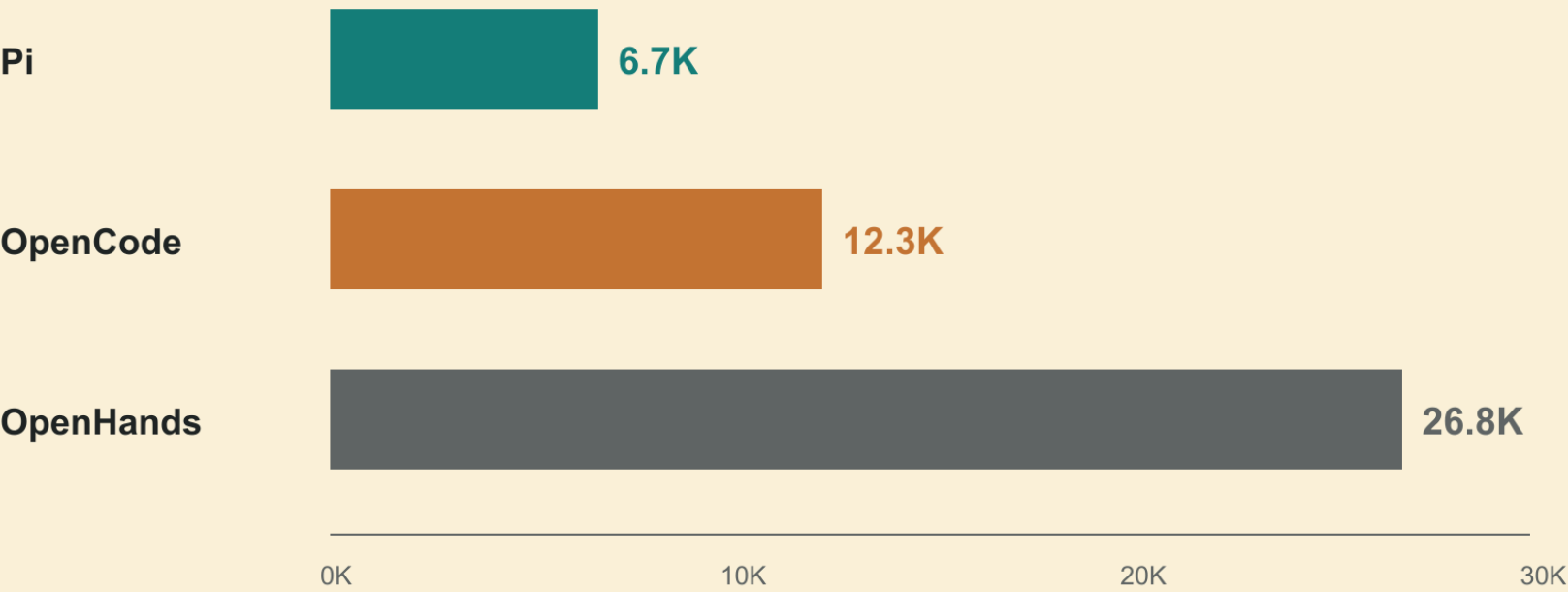
Together, the built-in prompt, tools, and middleware fell about 65%. Conversation history is not included in this number. Overall reward held steady across four tested models.

Workshop result: the same model saw a 4× range in context per call

With GLM-5.2, Pi averaged 6.7K input tokens per call and OpenHands averaged 26.8K. All three harnesses passed 8/8.

Average context per call varied across harnesses

GLM-5.2 · eight tasks per harness · Aug 24, 2026 · all three harnesses passed 8/8



Input footprint is not dollar cost. Cache reads and output tokens need separate accounting.

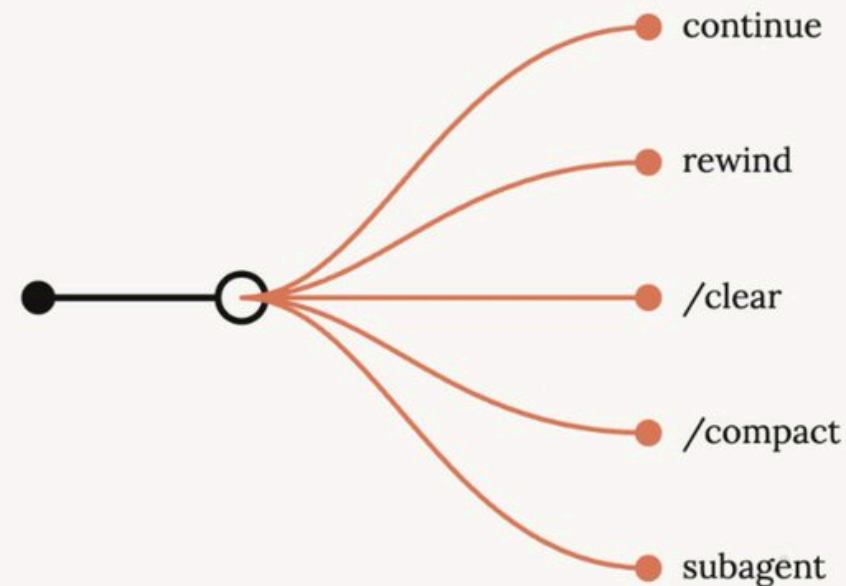
Aug. 24 short-suite results. Rounded mean input per call, including repeated context.

Five ways to manage a full context window

Continue keeps the history. Rewind returns to an earlier point. Clear starts fresh. Compact replaces older turns with a summary. A subagent moves bounded work into another context.

USING CLAUDE CODE

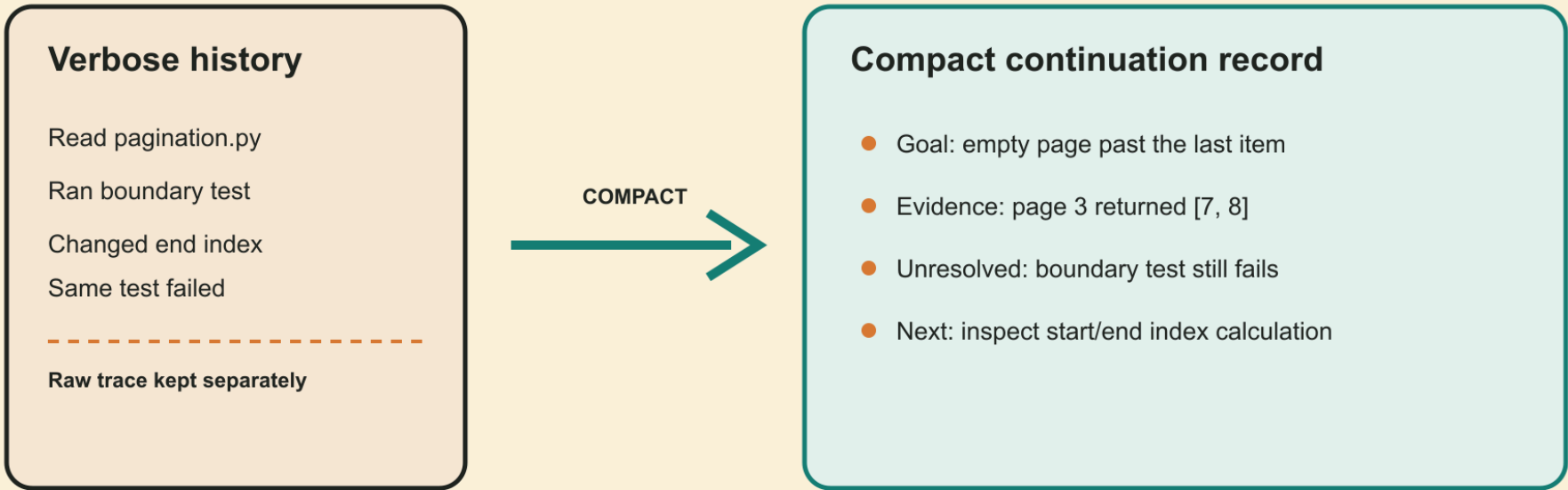
Session Management & 1M Context



Compaction replaces old turns with a continuation summary

ILLUSTRATIVE WORKED EXAMPLE • NOT A RECORDED RUN OR MEASURED RESULT

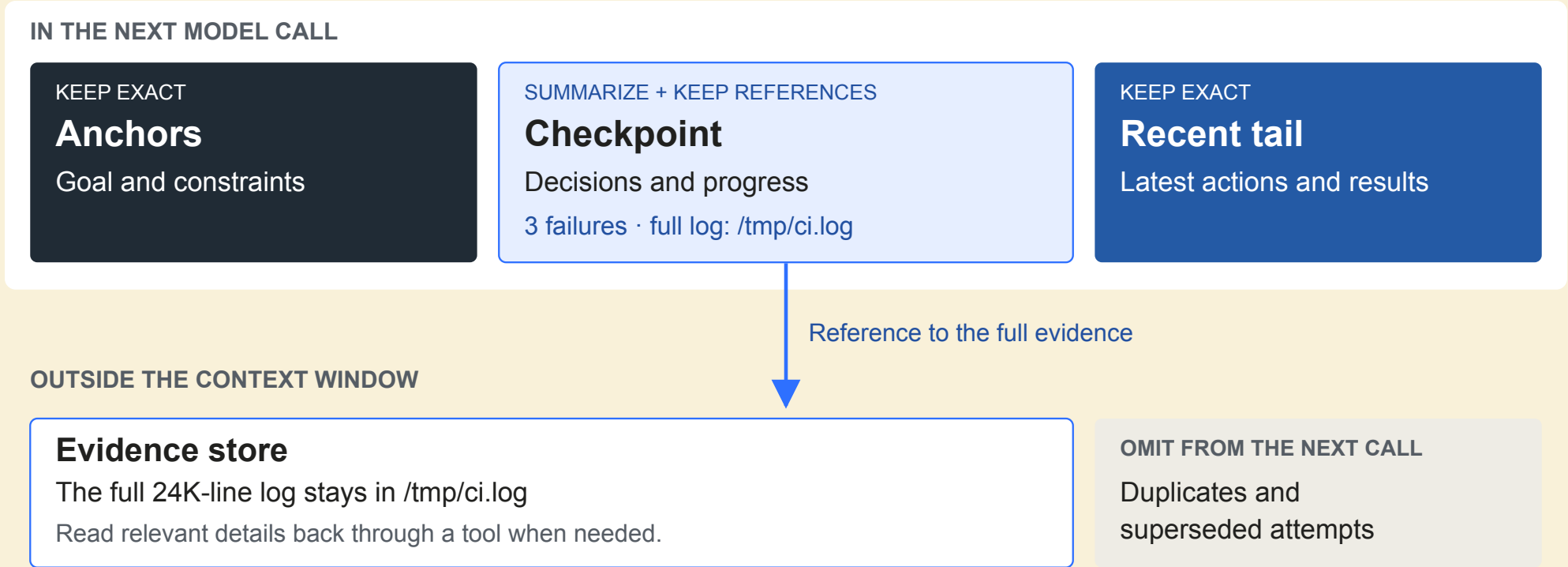
Compaction makes a continuation record



The summary keeps the task, completed work, current evidence, and next action. Older tool output leaves the active input but can remain available in the trace. Illustrative pagination example.

What to preserve during compaction

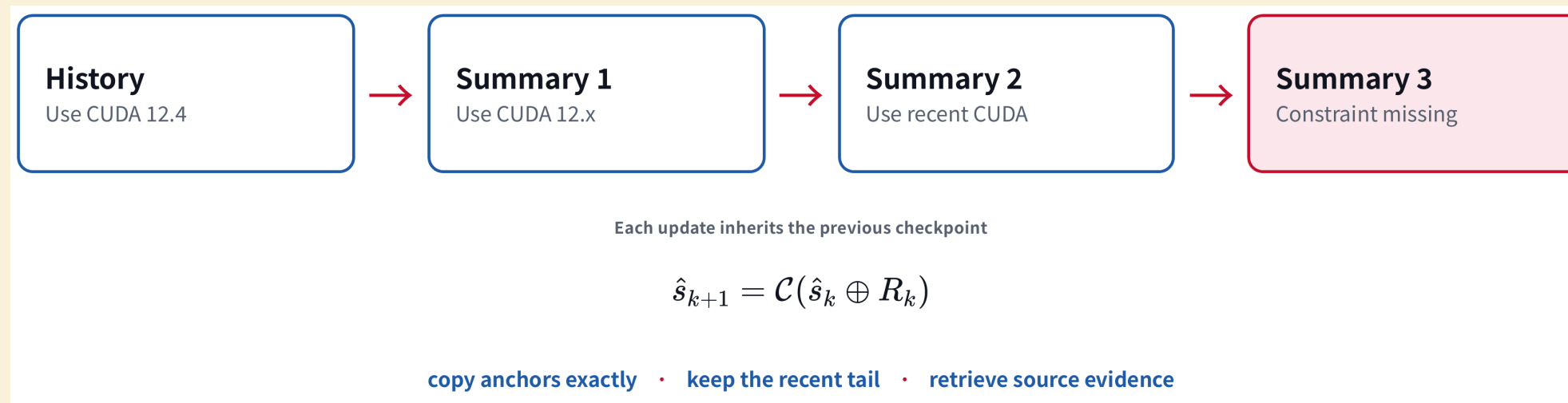
Preserve exact requirements, changed files, verification results, and the next action. Keep bulky evidence retrievable.



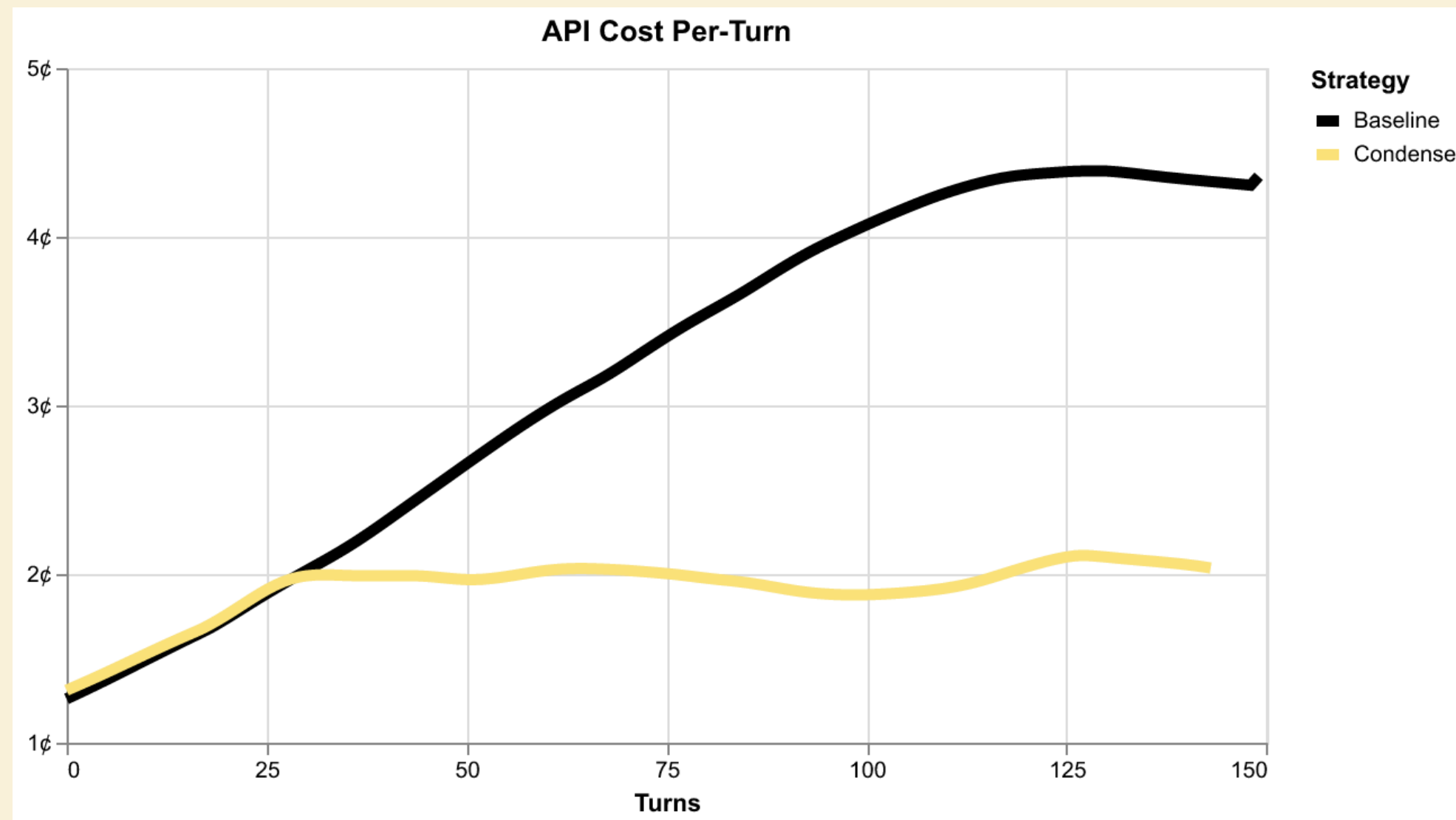
Adapted from [CMU 11-768, Lecture 3, page 48](#). A design framework, not a measured workshop result. The file path is illustrative. Omitted history is excluded from the next model call, not necessarily deleted from audit storage.

Repeated compaction can weaken an exact requirement

“CUDA 12.4” loses specificity across summaries. Keep exact constraints in a stable record. Illustrative example.

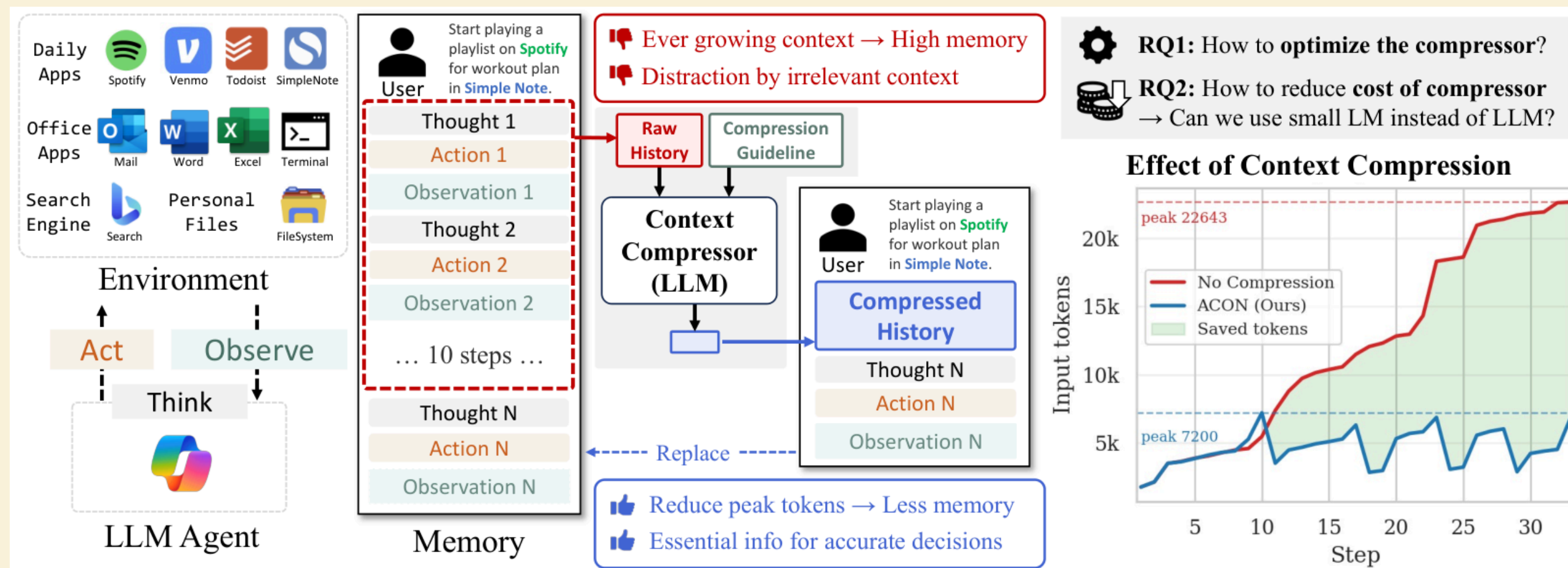


OpenHands reported up to 2× lower per-turn API cost



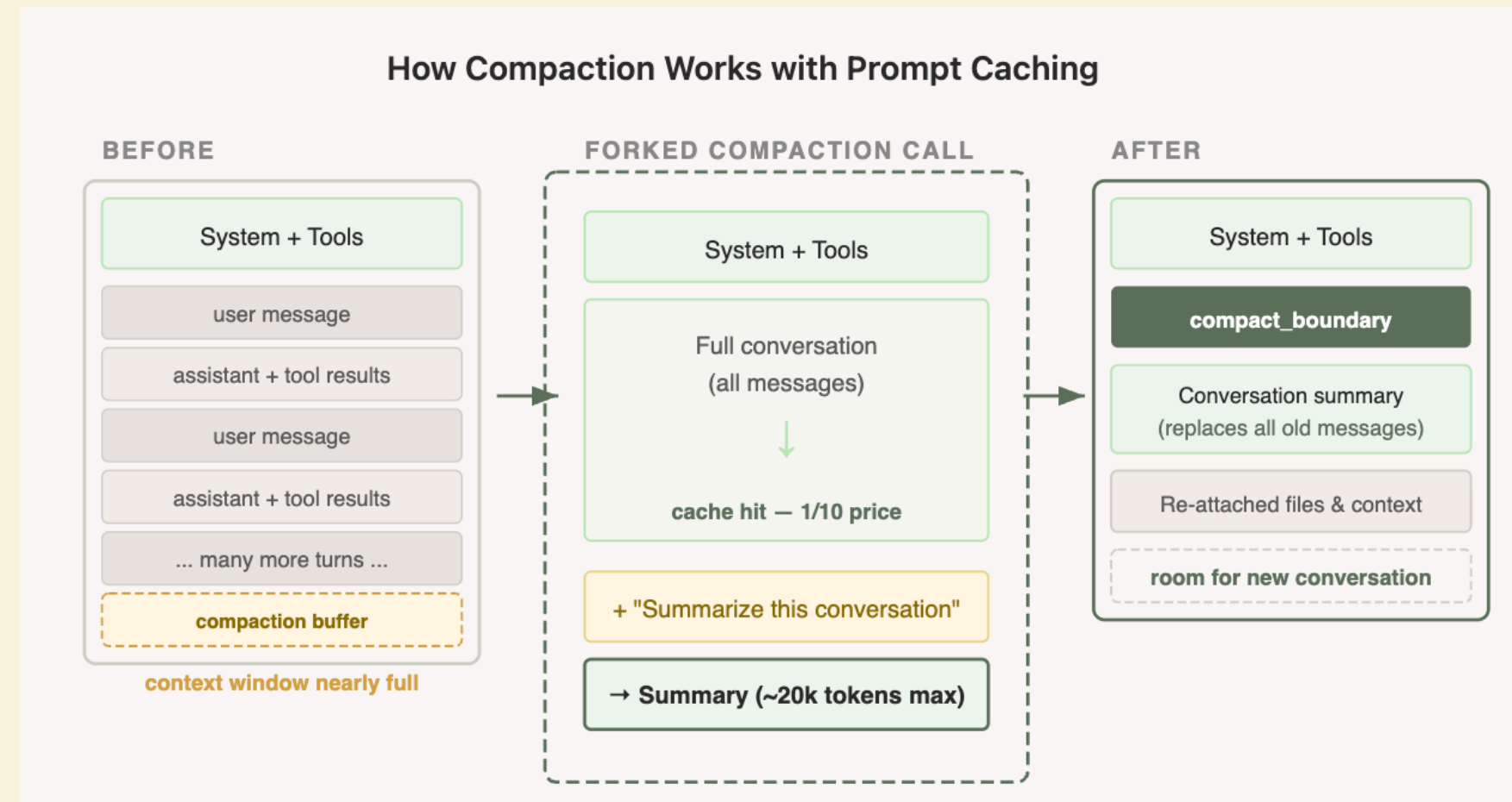
On a tested SWE-bench Verified subset, the baseline rose above 4¢ per turn while the condenser stayed near 2¢. Average solve rate was 54% with condensation and 53% without it.

ACON used failed runs to improve context compression



Researchers compared failed compressed runs with successful uncompressed runs, then refined the instructions about what to preserve. Across three benchmarks, peak context fell 26–54% and task success improved over other compression baselines.

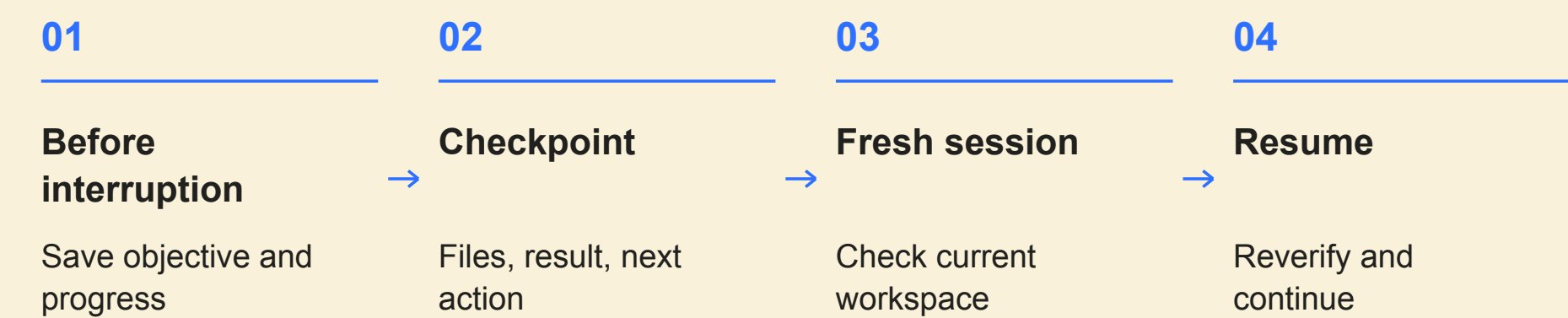
Claude Code compacts without abandoning the cached prefix



A separate summarization request would miss the cache. Claude Code instead reuses the same system prompt, context, tools, and history, then appends the compaction instruction. Only the final instruction is new.

Resuming a task from saved working state

Save progress and the next action. On resume, verify that the saved state still matches the files.



A checkpoint records evidence; it does not guarantee that the evidence is still current.

Saving working state in files

Task Breakdown

Phase 1 — Project Setup

- ☐ Initialise Node.js project (`npm init`)
- ☐ Install dependencies: `express`, `prisma`, `jsonwebtoken`, `bcrypt`, `dotenv`
- ☐ Configure `.env` with `DATABASE_URL`, `JWT_SECRET`, `JWT_REFRESH_SECRET`
- ☐ Set up Prisma schema with `User` model

Phase 2 — Database Layer

- ☐ Define `User` model in `schema.prisma`
 - Fields: `id`, `email`, `passwordHash`, `createdAt`, `refreshToken`
- ☐ Run initial migration: `npx prisma migrate dev`
- ☐ Create `db.ts` Prisma client singleton

Phase 3 — Auth Logic

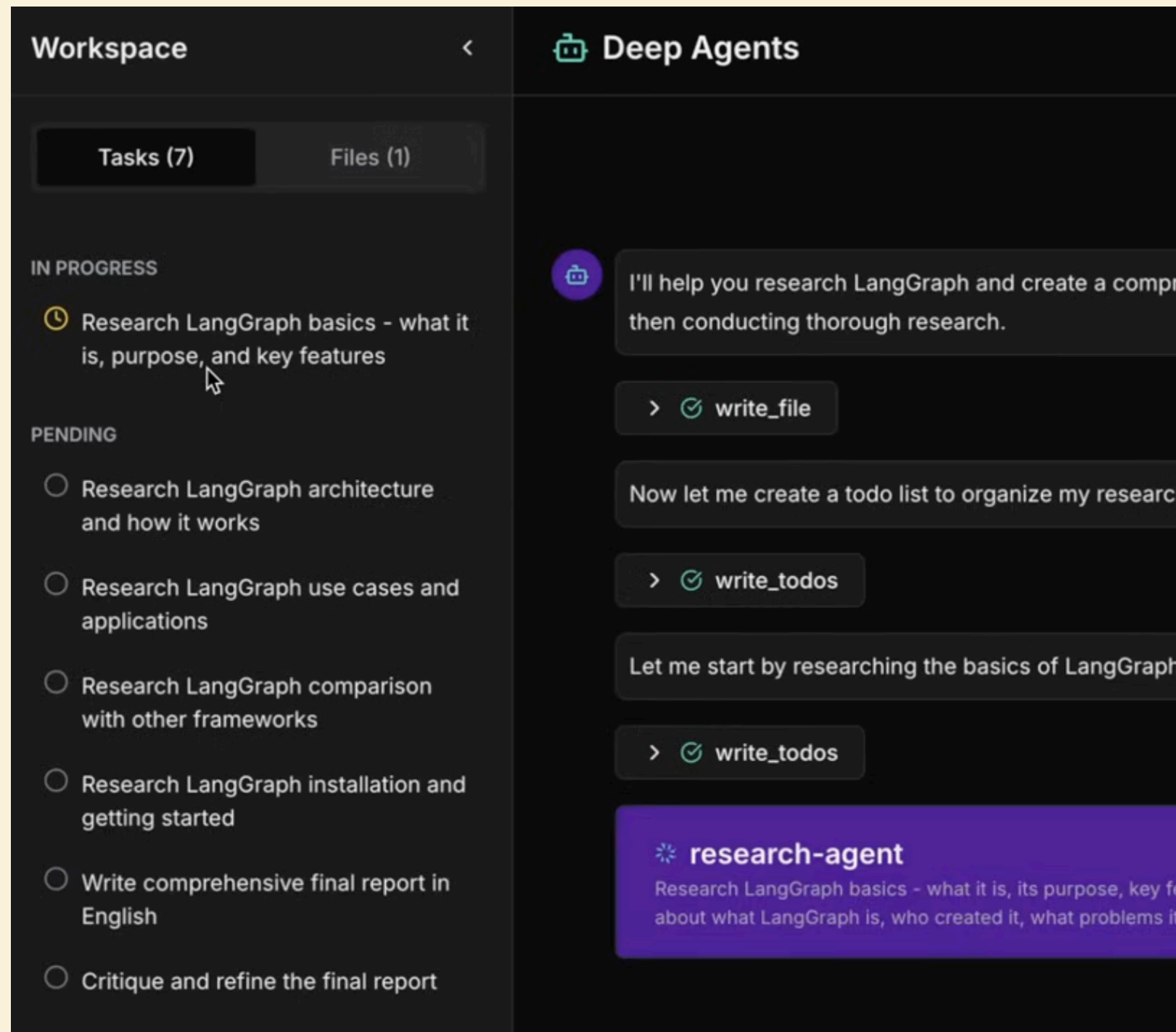
- ☐ `auth/hash.ts` — `hashPassword()` and `verifyPassword()` helpers
- ☐ `auth/tokens.ts` — `generateAccessToken()`, `generateRefreshToken()`, `verifyToken()`
- ☐ Token expiry: access = 15 min, refresh = 7 days

Phase 4 — API Routes

- ☐ `POST /auth/register` — validate input, hash password, create user
- ☐ `POST /auth/login` — verify credentials, return token pair
- ☐ `POST /auth/refresh` — validate refresh token, issue new access token
- ☐ `POST /auth/logout` — invalidate refresh token in DB
- ☐ `GET /me` — protected route returning current user profile

A workspace file lets a fresh session resume the plan without replaying the conversation.

Using planning to guide ongoing work



A saved task plan records unfinished work across turns. Update it as the work changes.

A durable scratchpad predicted success in YC-Bench

Only three of twelve models grew their starting capital. Scratchpad use was the strongest predictor, without proving causation.

Weak run state

- Relies on conversation history
- Loses decisions across turns
- Repeats old mistakes

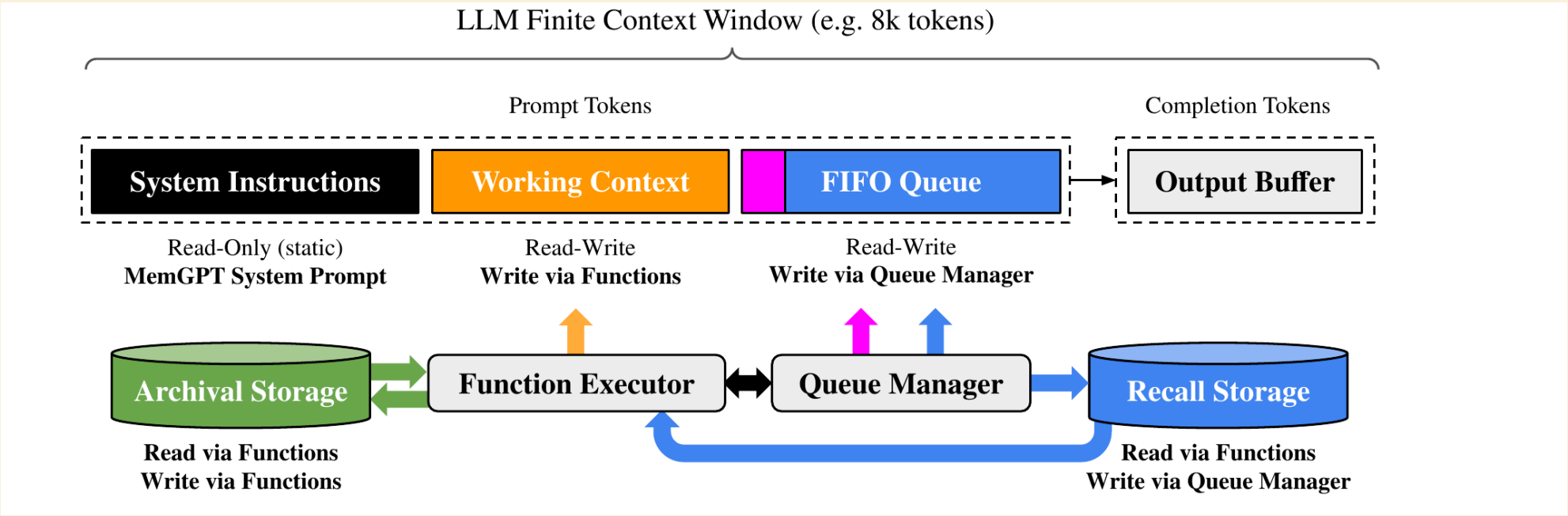
Durable scratchpad

- Records facts and decisions
- Carries plans across turns
- Correlated with higher success

YC-Bench found scratchpad use was the strongest predictor of success. This is correlation, not proof of causation.

Saved information must be loaded before the model can use it

Saved information extends context only when the agent retrieves or loads it.



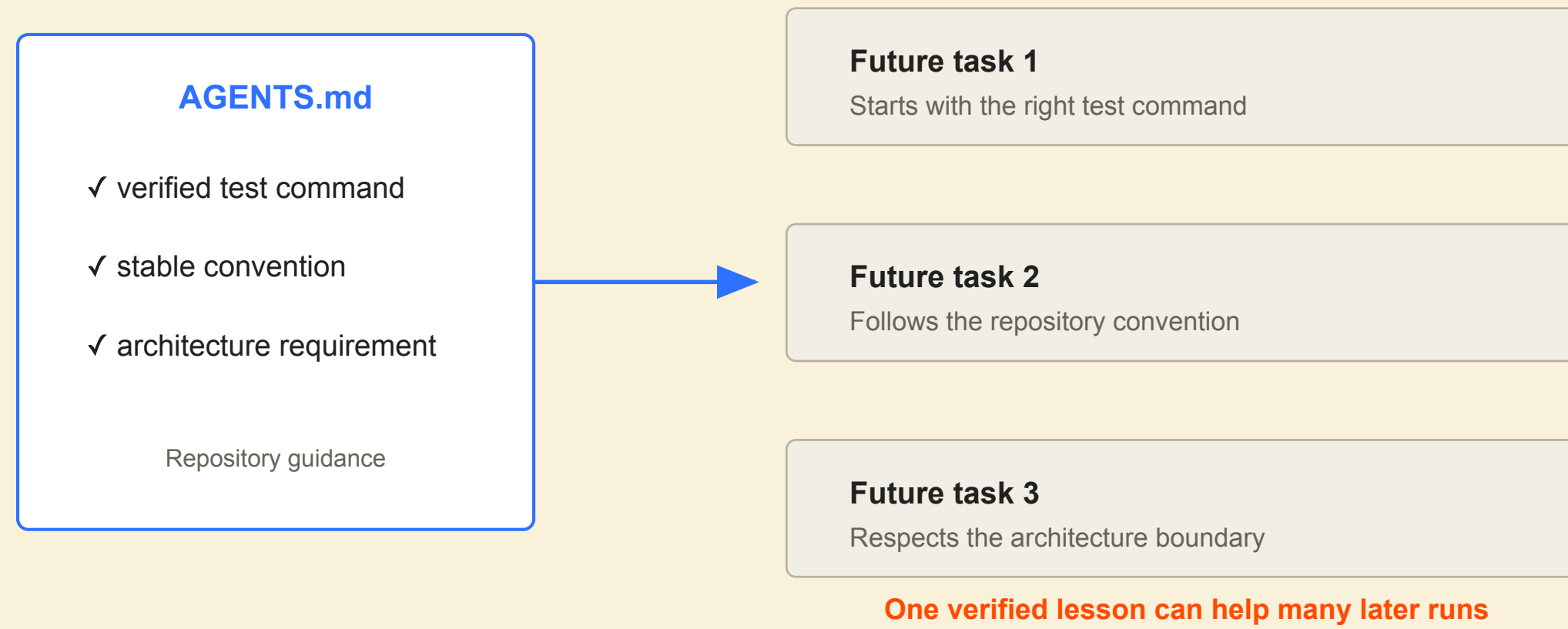
A verified task lesson can become future guidance

Move a verified command or stable convention into reusable guidance. Keep the evidence and unfinished work with the current task.

Current task state	Future guidance
One-off stack trace	Reviewed test command
Latest verifier result	Stable repository convention
Unfinished next action	Task-relevant procedure

Load durable guidance when relevant. Recheck it when the repository changes.

AGENTS.md: repository guidance for future tasks



AGENTS.md supplies repository guidance: verified test commands, conventions, and architecture requirements. Keep temporary progress in task notes.

One test-cost warning reduced wasted full-suite runs

Strategy	Full-suite runs/cell	Duration (s)
NONE	3.67	2689
ALWAYS_ON	2.44	2066
SELECTIVE	1.67	2032

The repository warned that its full test suite takes more than 20 minutes. With no guidance, Claude launched the full suite 3.67 times per experimental cell. With selective guidance, that fell to 1.67 and wall-clock time fell from 2,689 to 2,032 seconds. Correctness did not change, and the result was exploratory.

The token cost of repository instructions

Sample AGENTS.md file

Dev environment tips

- Use ``pnpm dlx turbo run where <project_name>`` to jump to a package instance
- Run ``pnpm install --filter <project_name>`` to add the package to your workspace
- Use ``pnpm create vite@latest <project_name> -- --template react-ts`` to create a new project
- Check the name field inside each package's package.json to confirm the package name

Testing instructions

- Find the CI plan in the .github/workflows folder.
- Run ``pnpm turbo run test --filter <project_name>`` to run every check defined in the CI plan
- From the package root you can just call ``pnpm test``. The commit should pass
- To focus on one step, add the Vitest pattern: ``pnpm vitest run -t "<test_name>"``
- Fix any test or type errors until the whole suite is green.
- After moving files or changing imports, run ``pnpm lint --filter <project_name>``
- Add or update tests for the code you change, even if nobody asked.

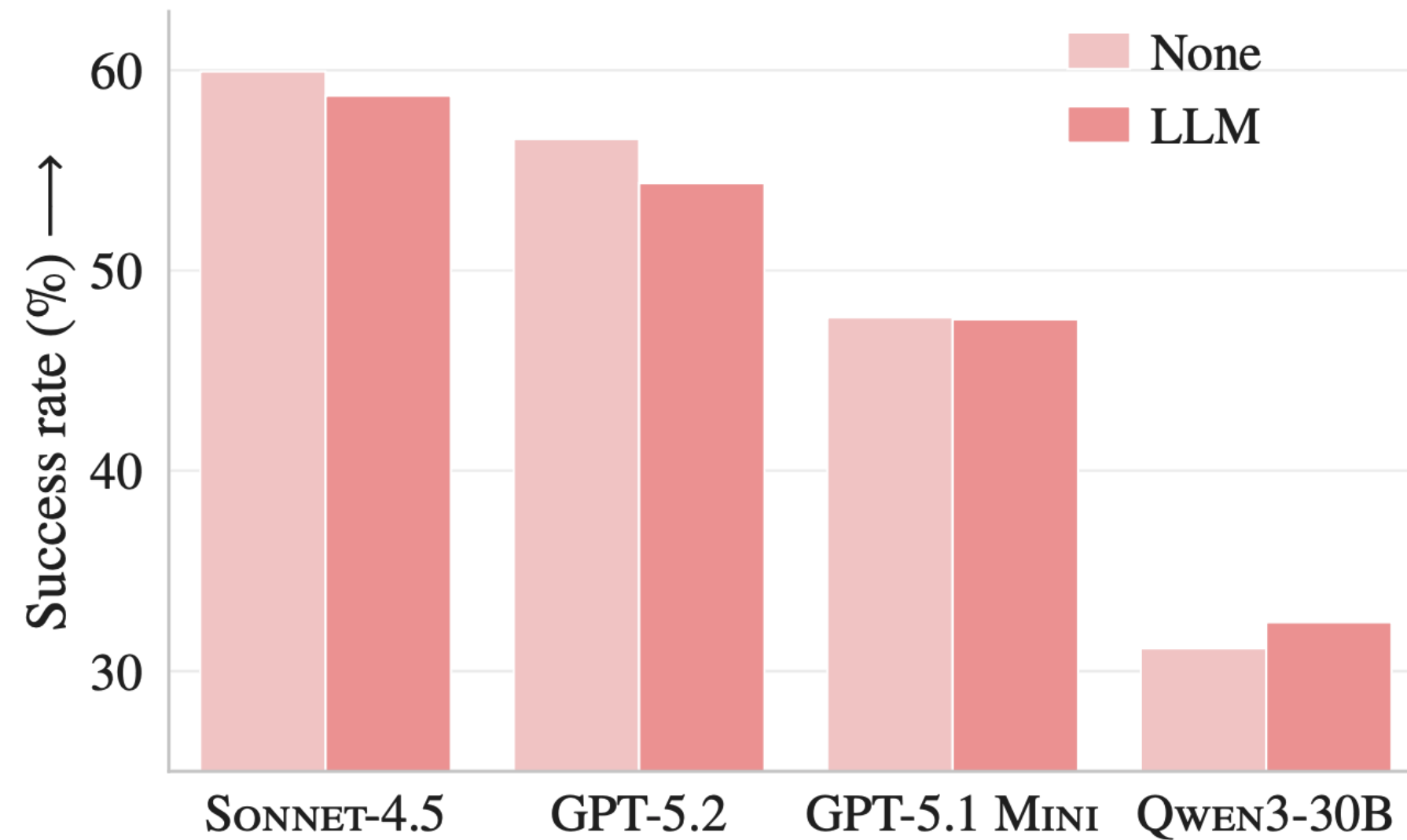
PR instructions

- Title format: [`<project_name>`] `<Title>`
- Always run ``pnpm lint`` and ``pnpm test`` before committing.

Repository instructions consume tokens whenever the harness includes them in a model call.

Generated AGENTS.md files lowered success and raised cost

Automatically generated guidance increased inference cost by over 20% in the cited study.



91 of 100 files had duplicated, oversized, misplaced, stale, or unclear instructions

TABLE II
SMELLS DETECTED IN REAL PROJECTS
(FP: FALSE POSITIVES; PREC: PRECISION)

	Instances	FP	Prec. (%)
Context Bloat	42	-	-
Skill Leakage	35	6	82
Lint Leakage	62	4	93
Blind Reference	16	2	87
Init Fossilization	24	-	-
Conflicting Instructions	28	12	57

The audit found repeated lint rules in 62 files. Forty-two files exceeded 200 lines. Thirty-five loaded specialized procedures for every task. Other files had unexplained references, unchanged generated content, or conflicting instructions.

Pass rates barely changed after adding repository guidance in 288 runs

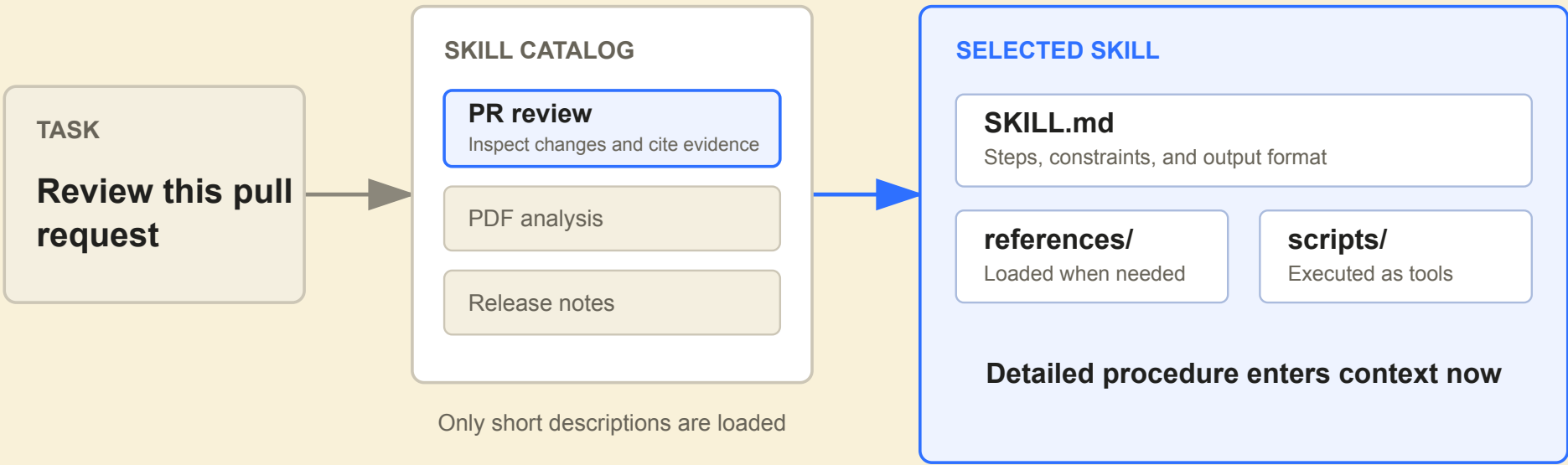
Table 1: Pass-rate by strategy and agent. Omnibus permutation p -values test whether strategy explains correctness beyond task difficulty.

Strategy	Claude (15 tasks)	Codex (17 tasks)
NONE	53.3% (24/45)	58.8% (30/51)
ALWAYS_ON	55.6% (25/45)	56.9% (29/51)
SELECTIVE	55.6% (25/45)	52.9% (27/51)
Omnibus p	1.000	0.66

Researchers compared no guidance, AGENTS.md loaded every turn, and files retrieved only when needed. Claude passed 53.3% to 55.6%. Codex passed 52.9% to 58.8%. The study covered 17 tasks in three Python repositories and could not rule out smaller effects.

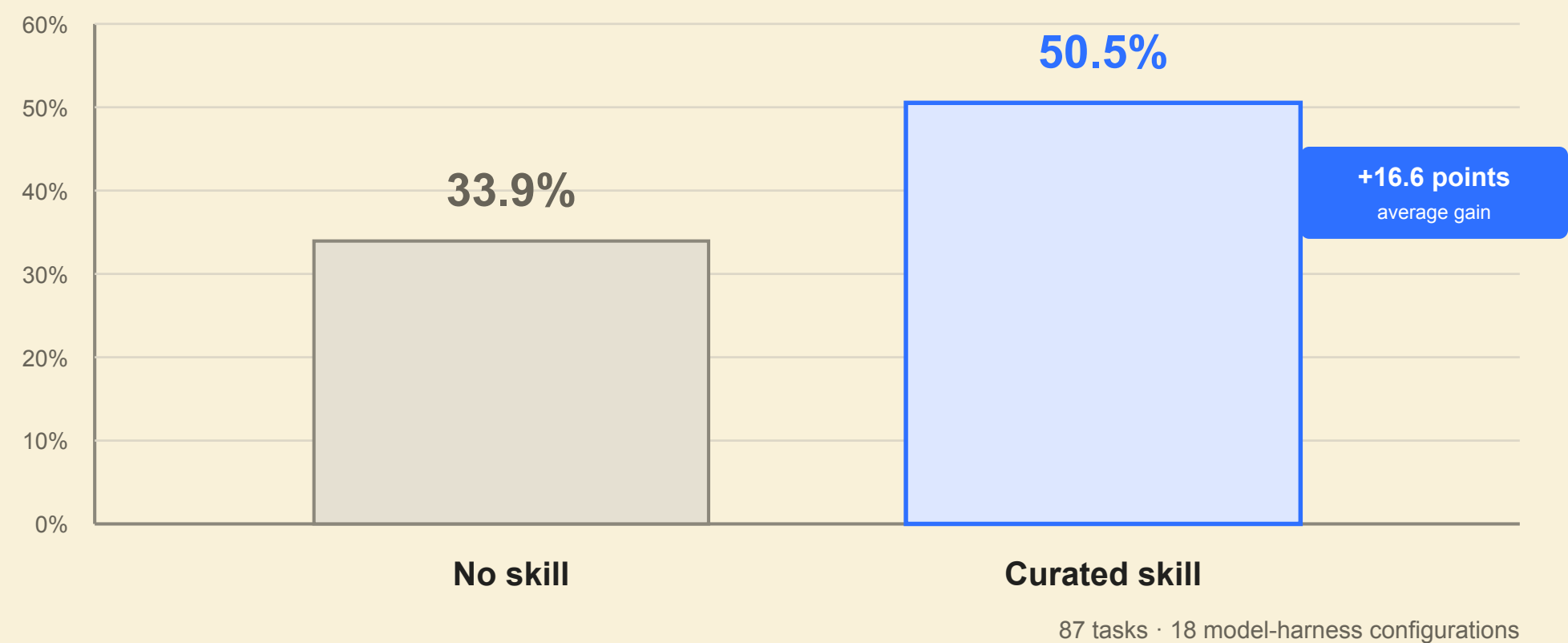
A skill loads detailed instructions only when the task needs them

The prompt carries a small catalog. The full SKILL.md, references, and scripts enter context only after the agent selects that skill.



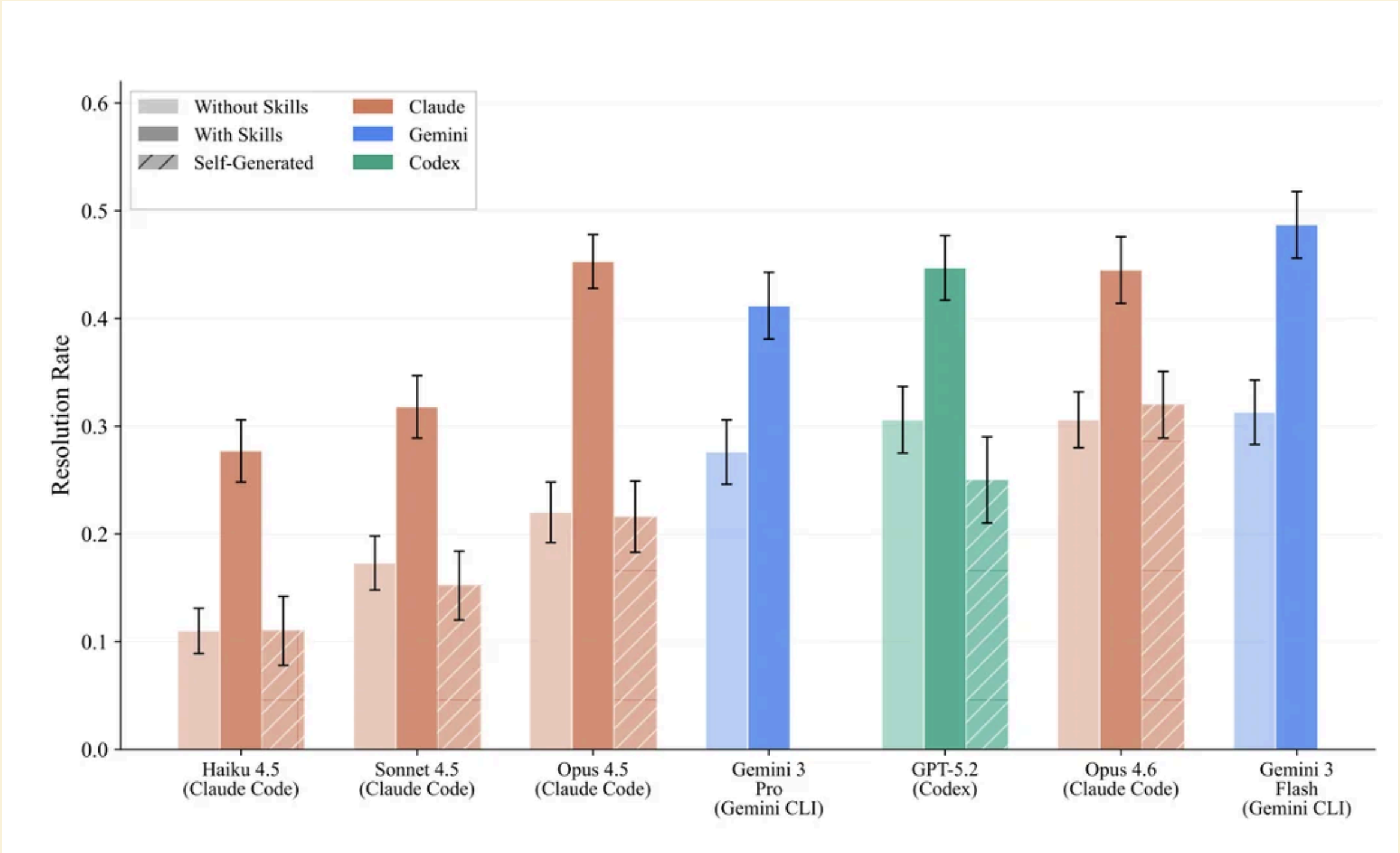
Progressive disclosure keeps unrelated procedures out of the prompt.

Curated skills raised average pass rate from 33.9% to 50.5%



The 16.6-point average gain covered 87 tasks and 18 model-harness configurations. Individual configurations gained 4.1 to 25.7 points.

Some skills still reduced performance

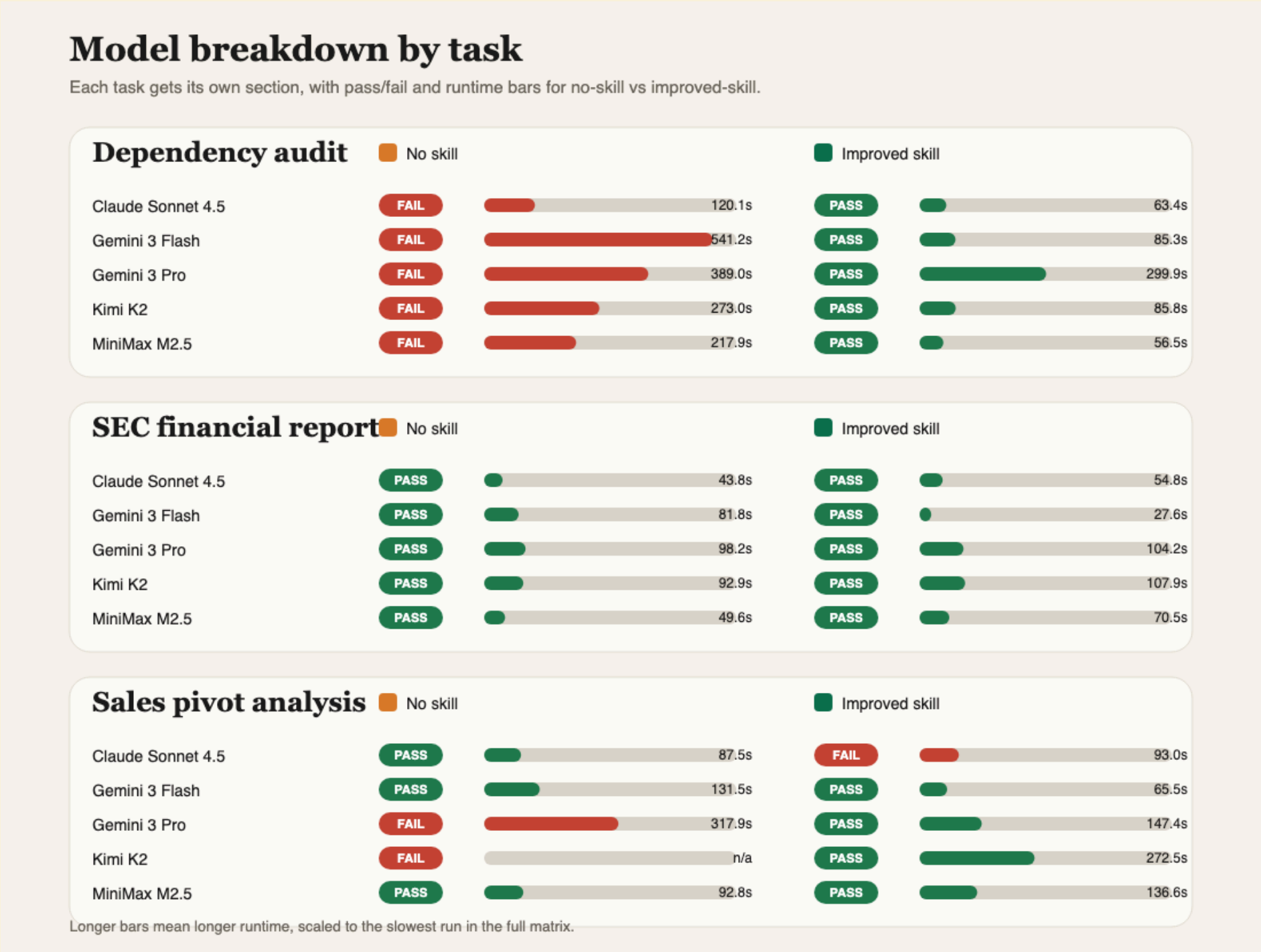


Review source: confirm which tested skills and baseline support this fraction.

https://www.reddit.com/r/rajistics/comments/1r77v1h/skillsbench_showed_models_arent_good_at/

Evaluate a skill on the work it should improve

Compare the same work with and without the skill, including correctness, time, and cost.



Where should each lesson go?

CHOOSE A DESTINATION FOR EACH ITEM

- A

CursorPage belongs in toyapp.pagination

Verified repository contract
- B

How to validate cursor traversal and malformed tokens

Reusable procedure for pagination work
- C

I changed two files; the next step is to rerun the external verifier

Current task progress

AGENTS.md

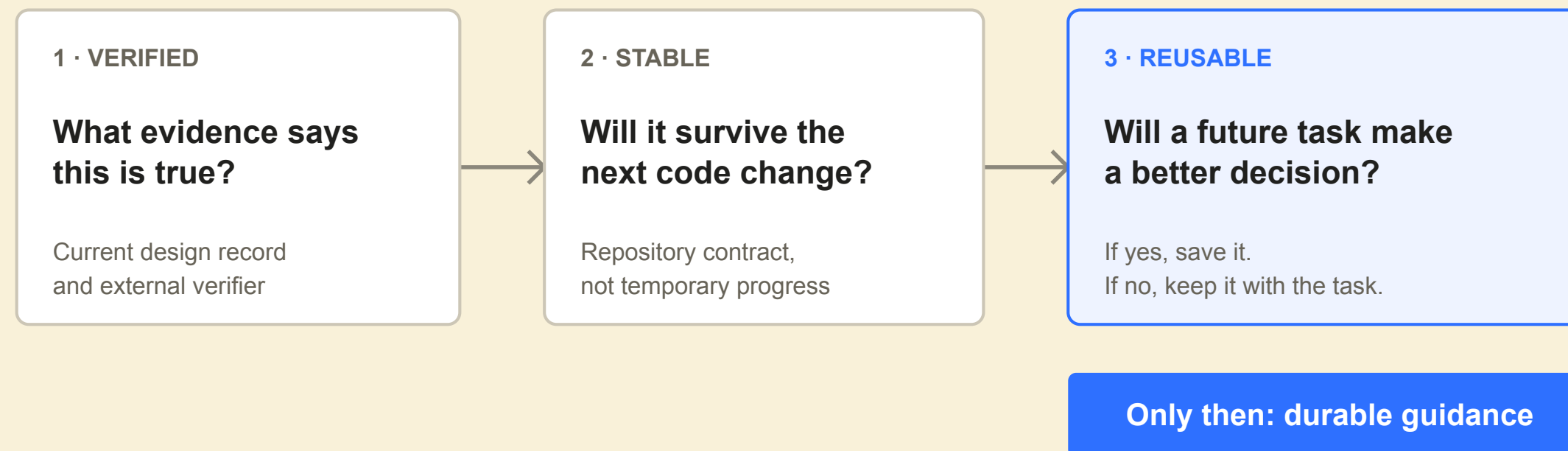
Skill

Task note

Delete

Ask the room to choose AGENTS.md, skill, task note, or delete for each item. Reveal the destinations after volunteers explain their choices.

Only verified, stable, reusable information should survive the task



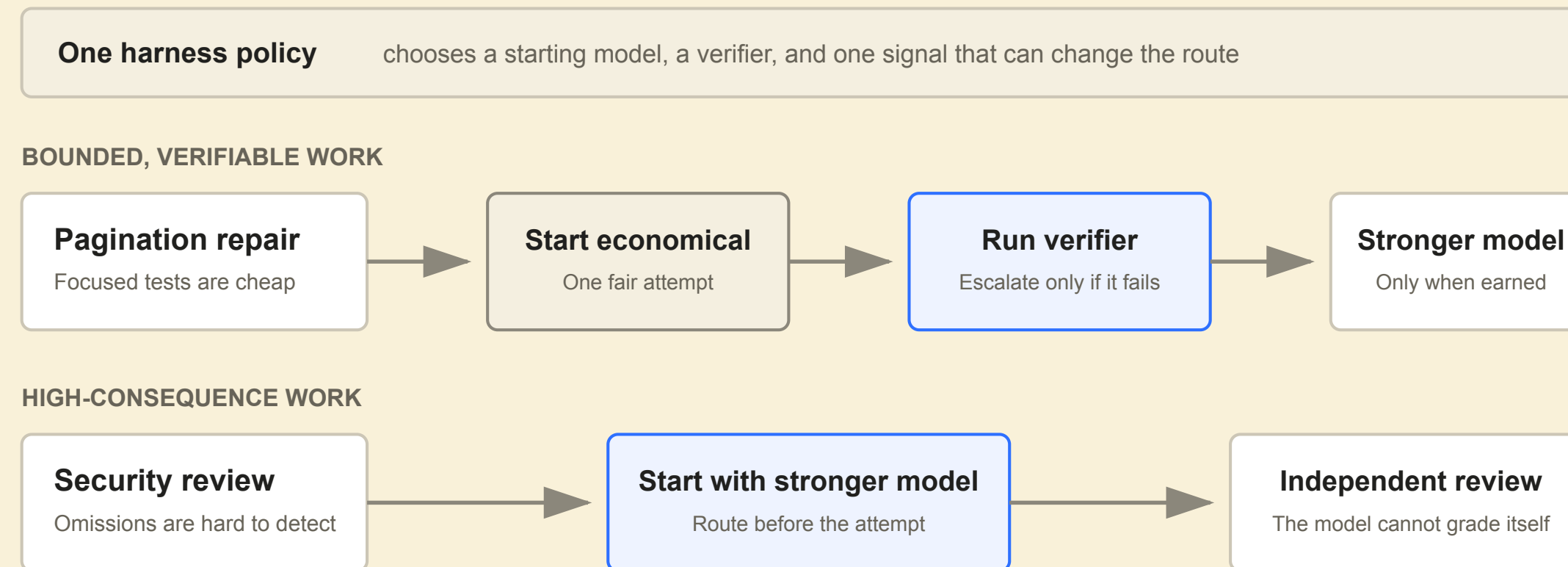
The destination depends on scope, but every durable item must be supported by evidence, likely to remain true, and useful beyond the current run.

Workshop 4: Flexible model choice

04

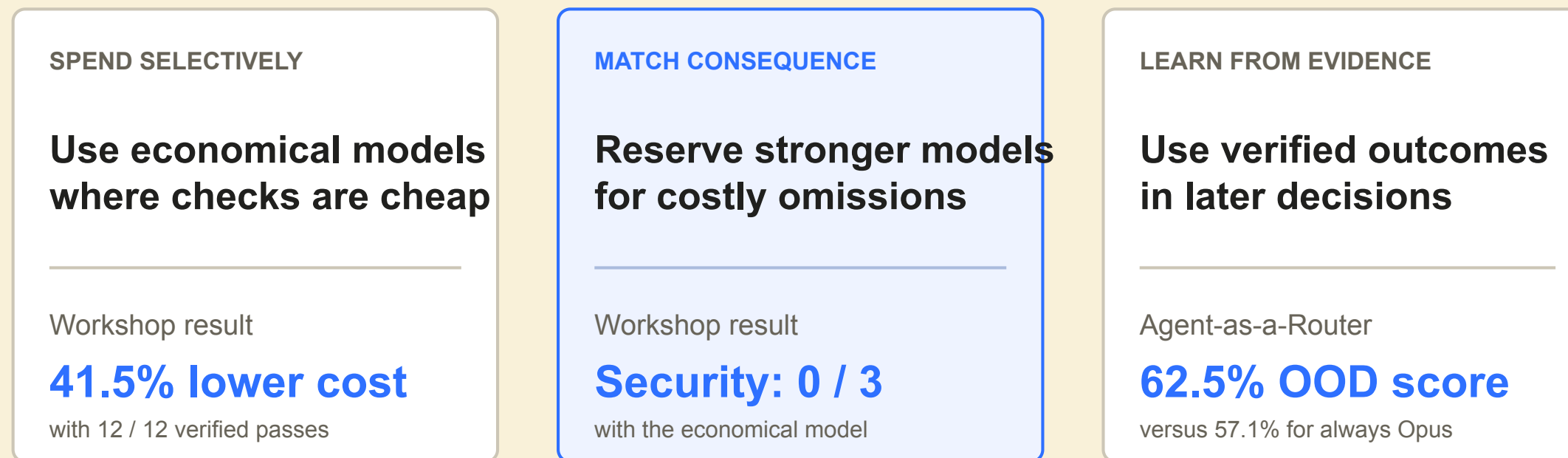
The harness can choose by task, delegate to another model, or switch after new evidence.

A harness can choose a different model for each task or attempt



Choose the starting model, verification method, and one signal that may change the route. Ask the room to design both paths before revealing the workshop result.

Routing lets the harness spend capability where it matters



Routing is useful only when the harness has a trustworthy signal for choosing or switching.

The benefit is not merely a cheaper model. A policy can reserve stronger capability for costly failures and use verified outcomes to improve later routing decisions.

Static routing passed 12/12 at 41.5% lower cost

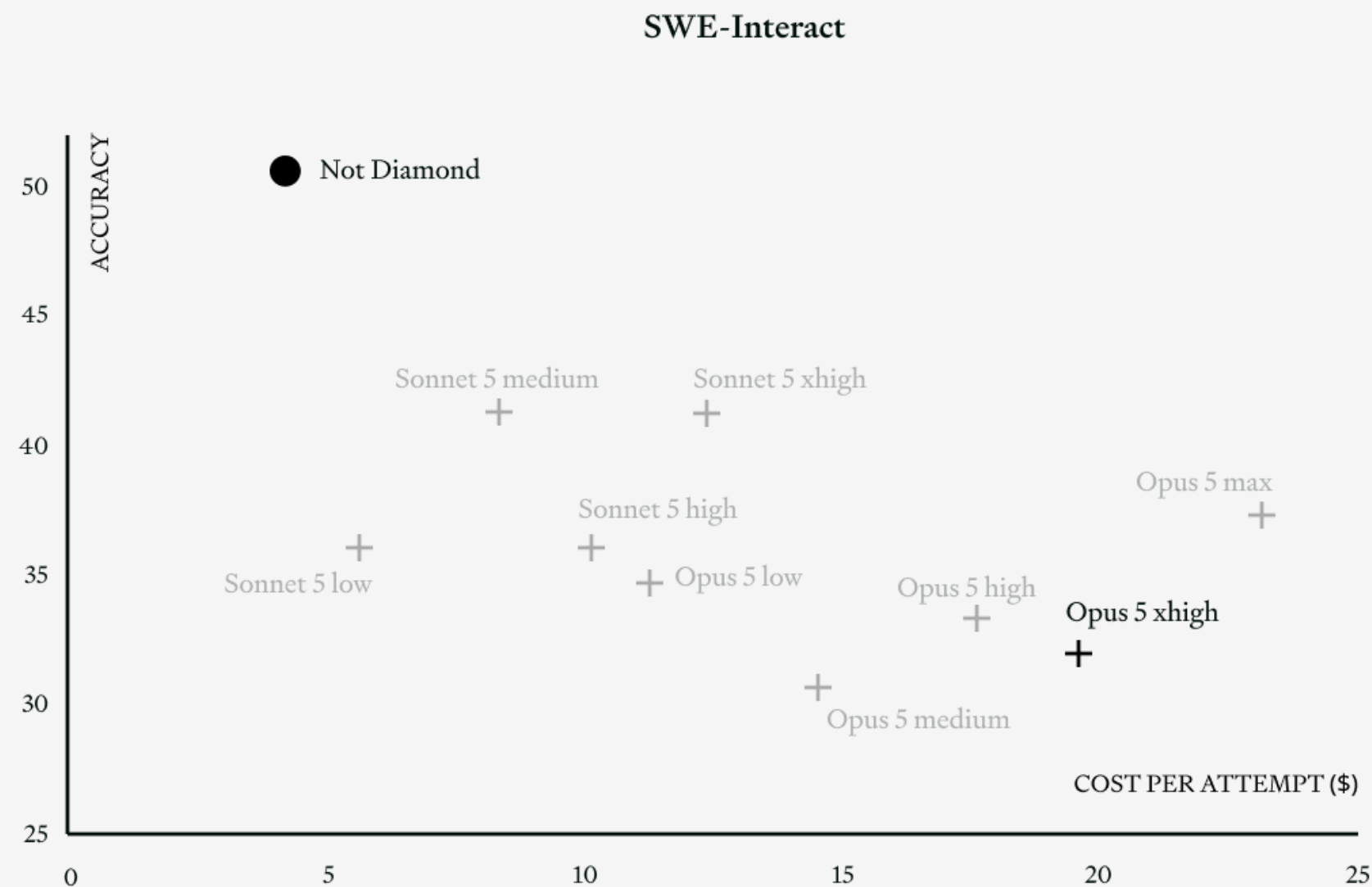
Four tasks × three trials. Costs are median four-task bills. Strongest-only finished faster.

Routing policy	Verifier passes	Cost / trial	Time / trial
Strongest everywhere	12 / 12	\$0.883	116 s
Economical everywhere	9 / 12	\$0.247	252 s
Risk-aware static	12 / 12	\$0.517	206 s

4 tasks × 3 trials. Passes cover all 12 runs; cost and time are medians for one 4-task trial.

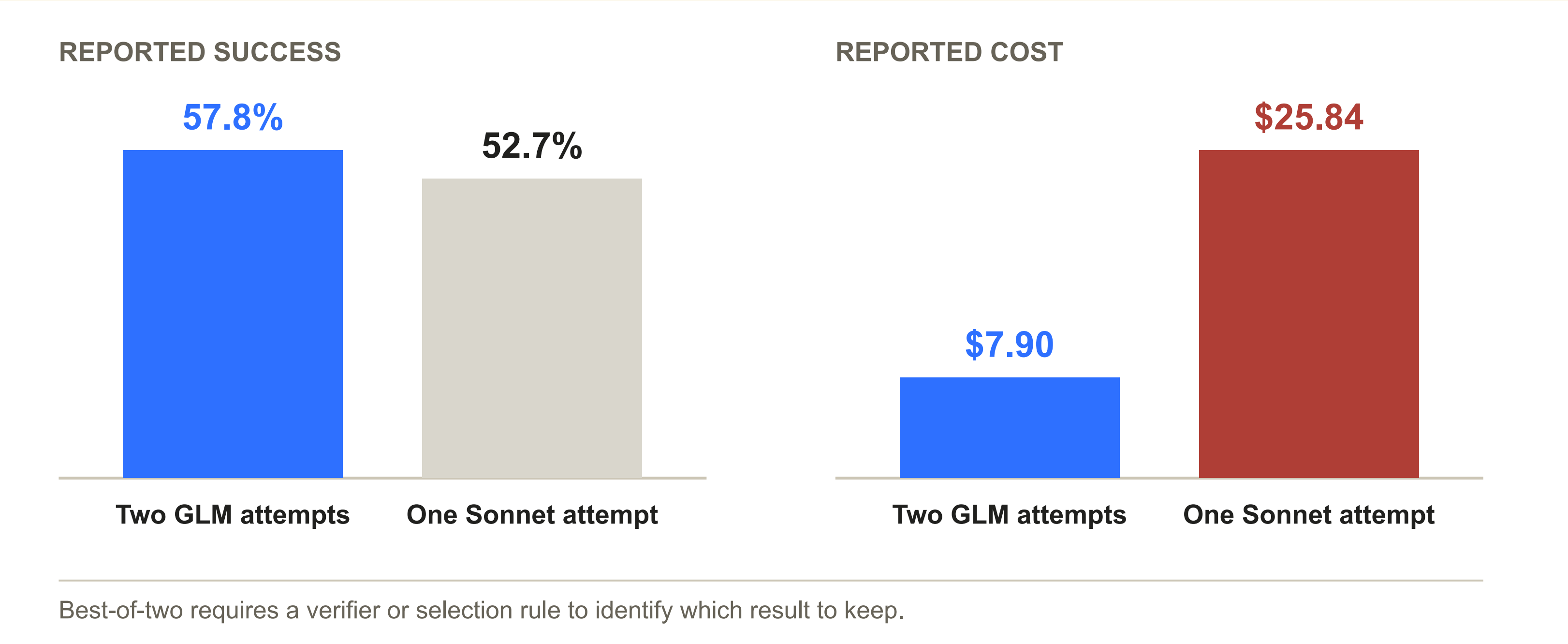
A router found a better quality-cost point than one fixed model

The reported router selected among Anthropic models and reasoning levels: 50.7% at \$4.28 per attempt on a simulated-user benchmark. This is evidence that the model choice can change with the interaction.



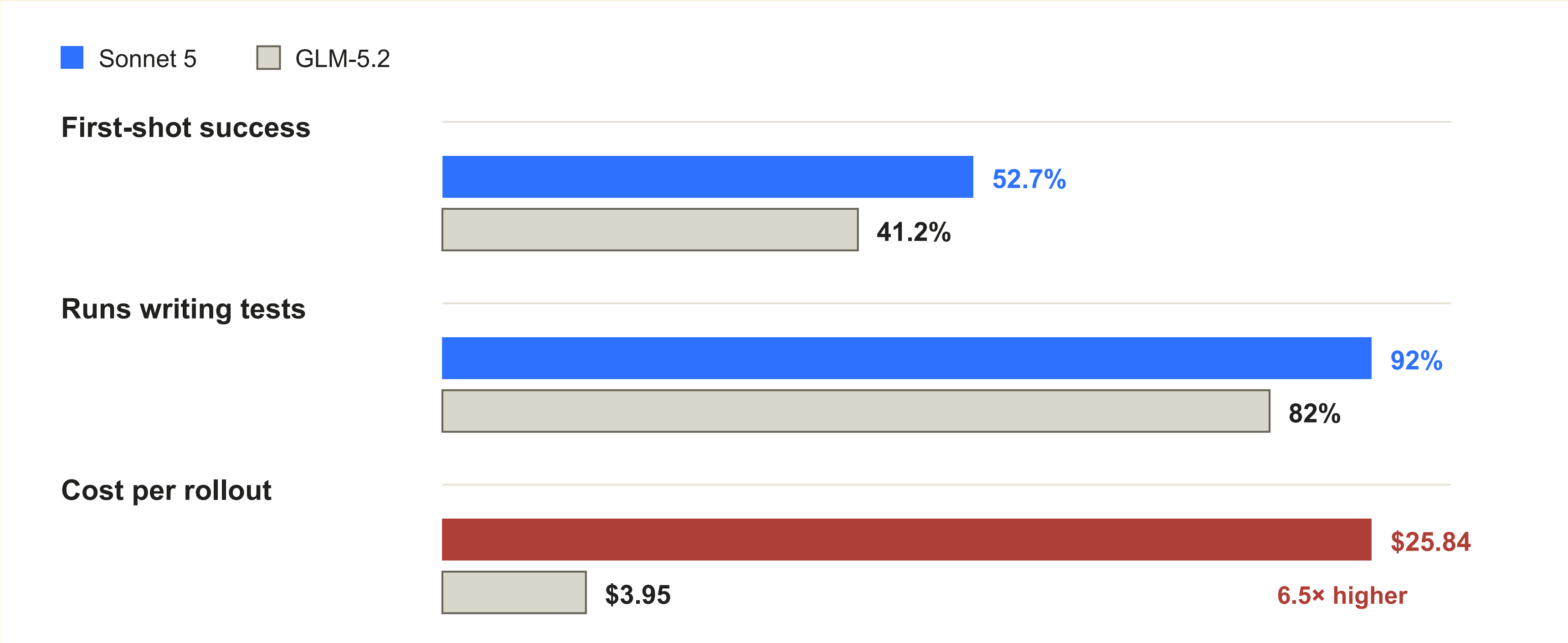
Not Diamond, September 1, 2026. Original chart. Preserve the benchmark and cost-per-attempt labels.

Routing can choose another attempt instead of a stronger model



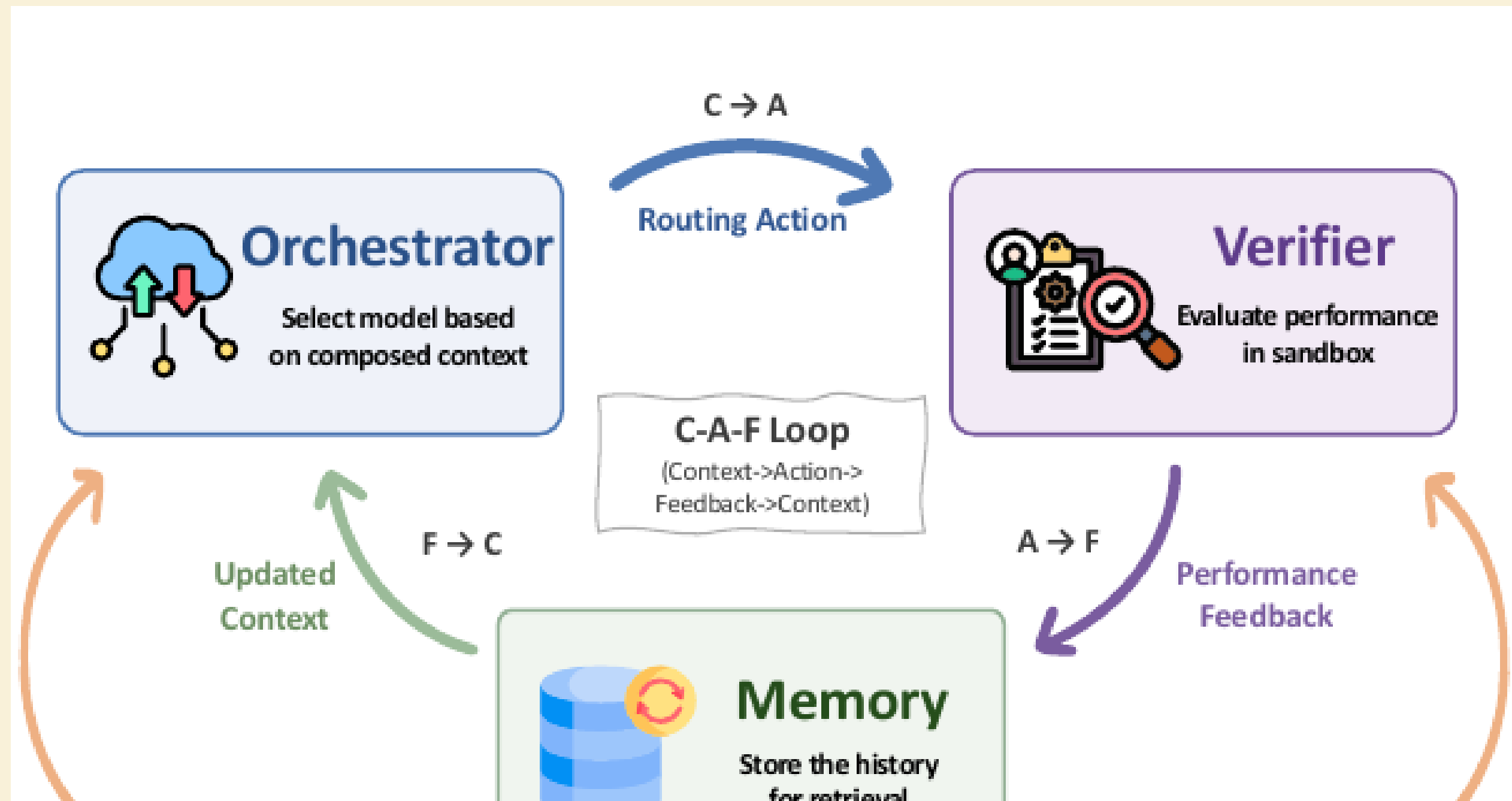
Zain Hasan, DeepSWE rollout analysis. 452 rollouts per model.

Different models spend the harness budget differently



[Zain Hasan, DeepSWE rollout analysis](#). Bars use separate scales for each reported measure.

Execution feedback can improve the next routing decision



Escalation is useful only when the signal is trustworthy

3.1. Preliminary: The Performance Gap Diagnosis

Table 1 | Preliminary ablation study diagnosing the performance bottleneck in LLM-as-a-Router (Claude Sonnet 4.6 tested on 2,919 tasks). AvgPerf: average performance score. Perf/\$: AvgPerf% per USD. Providing prior performance statistics significantly improves routing performance.

Ablation	Interpretation	AvgPerf%	Perf/\$
Oracle	Theoretical upper bound using the best model for each task	57.00	8.20
DimensionBest	Select the best model for each dimension by prior	47.50	3.69
Vanilla	Standard zero-shot LLM-as-a-router	41.41	1.97
+Dimension	+Task dimension description	41.18	1.81
+Perf stats	+Prior performance statistics from a probing set	47.74	1.71

Switching models can trade away cache reuse

Stay with Opus

Reuse the existing model-specific cache

100K cached history

new

cache hit

Switch to Haiku

Build a new cache for the new model

100K history reprocessed

new

cache miss

Anthropic's example: after 100K tokens, switching to Haiku can cost more than asking Opus to answer the easy question.

A bounded subagent handoff can avoid sending the full history to the second model.

Which evidence would make you change the model?

EVIDENCE FROM THE RUN

Verifier result Which requirement failed?	Execution trace Did the attempt make progress?	Budget left Can another attempt finish?	Cache cost What must be reprocessed?
---	--	---	--

YOUR ROUTING POLICY

1

Stay

2

Retry

3

Switch model

4

Stop or ask

Name the signal, the threshold, and the next action. Then count the full cost of that action.

Workshop 5: Goals, validation, and stopping

05

I missed Claude Code's `/loop` until Codex Goals made the pattern click: keep the objective active, return to the work, and check whether it is done.

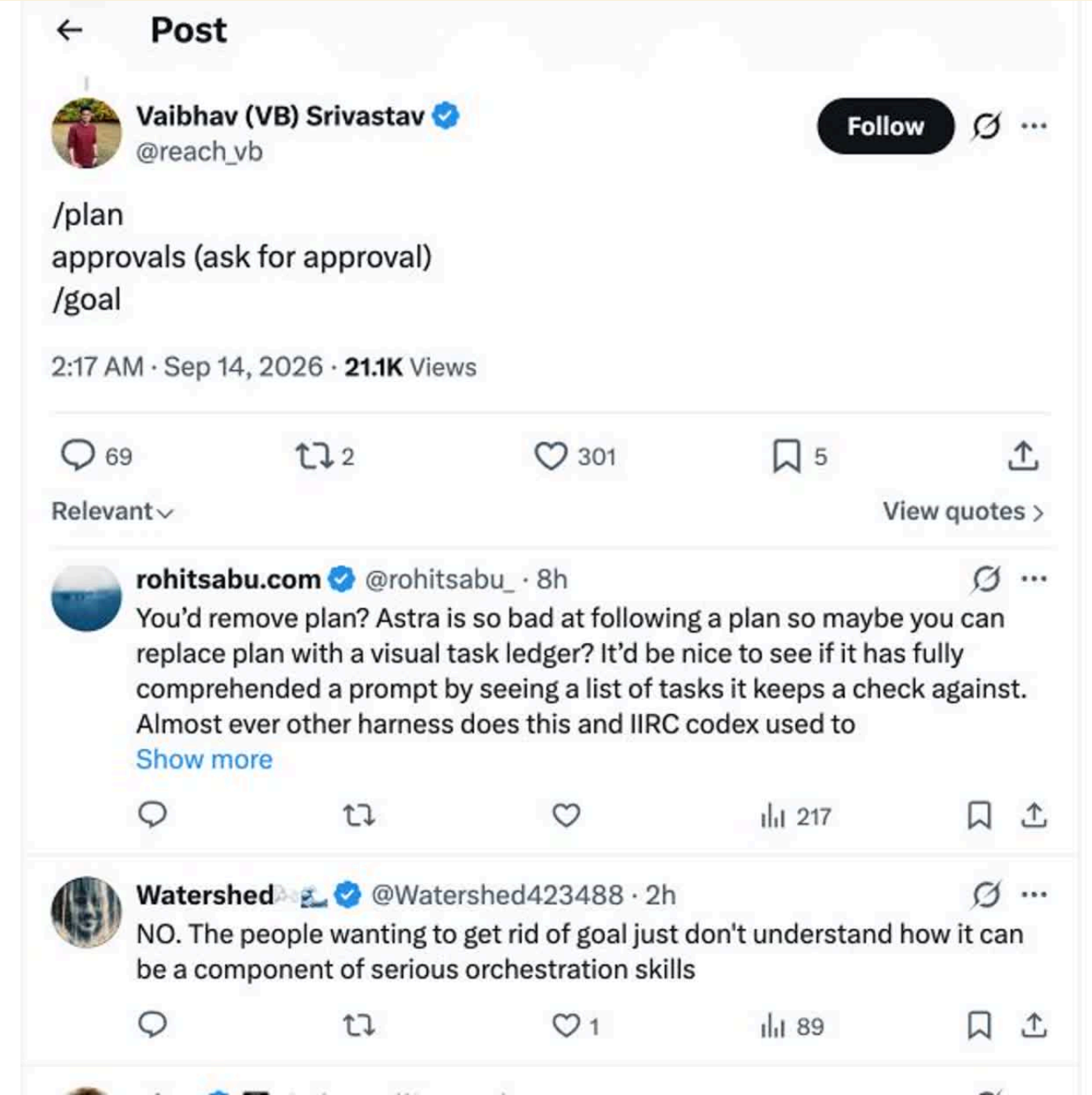
A useful goal loop records six things

The outcome stays stable. State and evidence change after each bounded attempt.

Part	What the harness records
Outcome	The observable result that must be true
Current state	Verified facts and unfinished work
Next action	One bounded attempt
Check	How the harness measures progress and records evidence
Limits	Maximum time, spend, attempts, and permissions
Terminal states	Complete, blocked, needs help, or out of budget

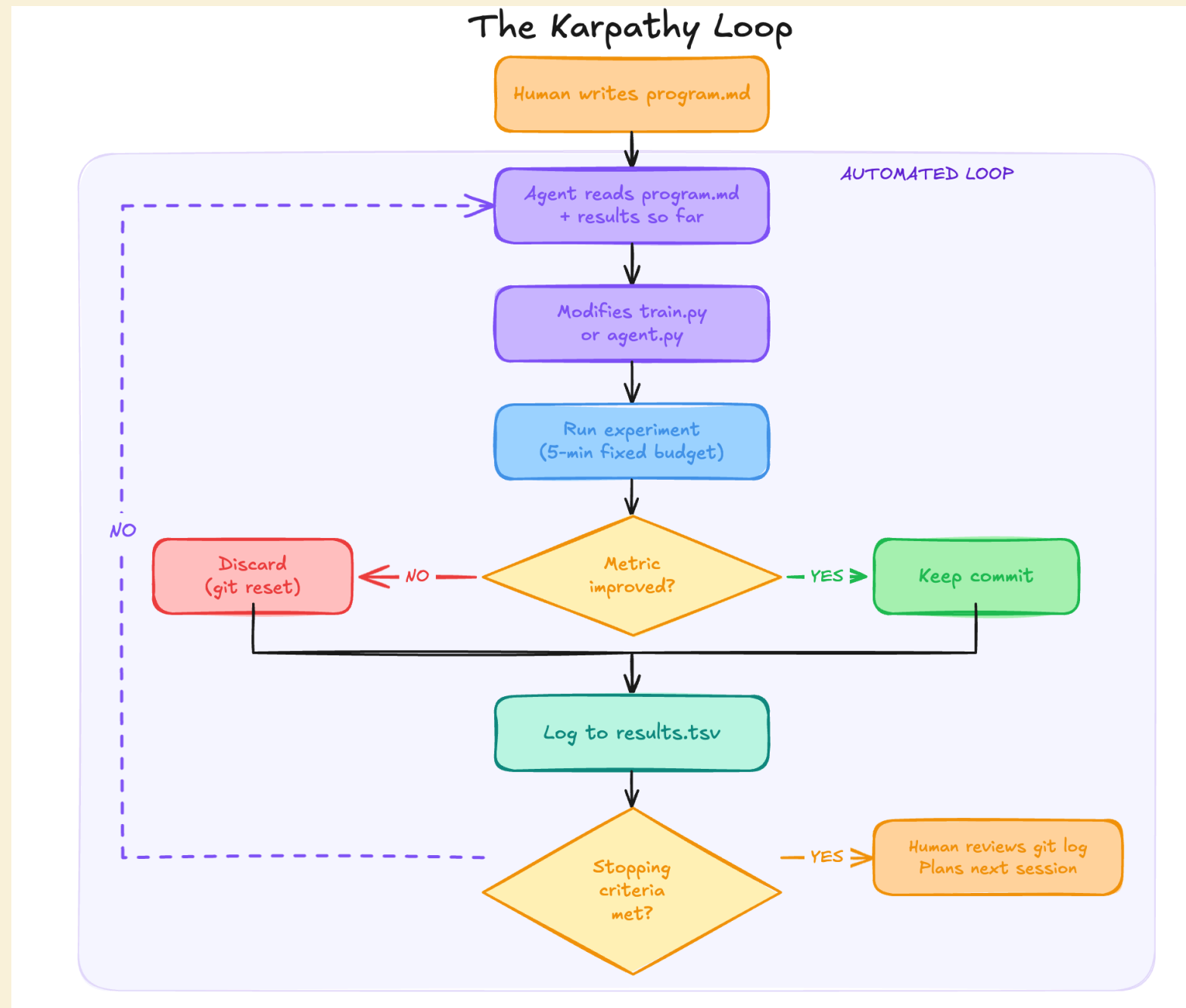
The outcome stays stable. State and evidence change after each bounded attempt.

Should goal, plan, and approval survive stronger models?



Vaibhav Srivastav on X, September 14, 2026

AutoResearch keeps a change only when the metric improves



Each attempt has a fixed budget. Keep the commit when the metric improves, otherwise reset. Record the result and stop when the stated criterion is met.

Does this request need a code change?

Fix `slugify` in two repository variants. One is already correct. Inspect before editing.

Repository A

The requested behavior already works

Checks pass before any edit

Correct action: explain and stop

Repository B

A boundary case fails

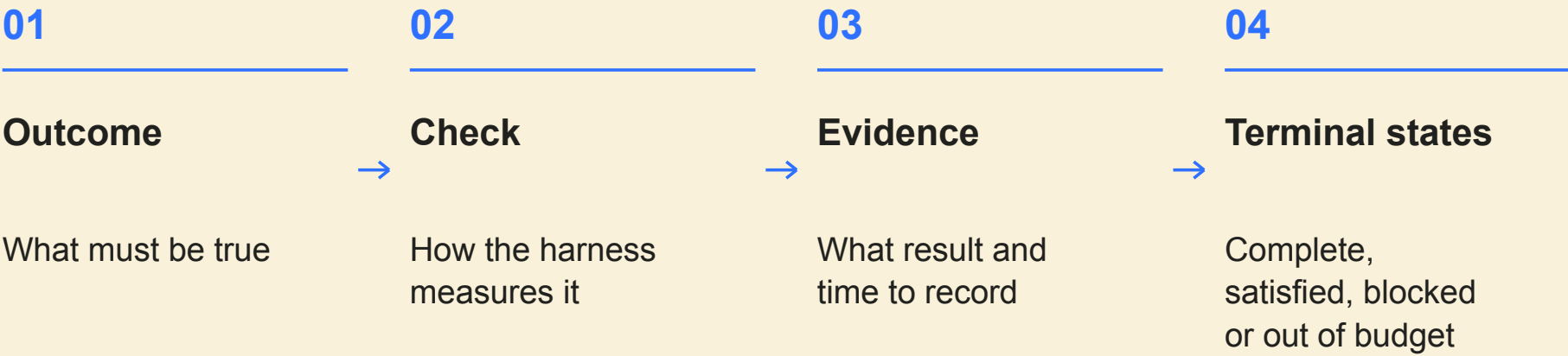
Make the smallest repair

Rerun the same checks

The request is identical. The current evidence determines whether the next action is edit or stop.

Define done before the agent starts editing

Name the outcome, check, evidence, and allowed terminal states. Then compare every attempt against the same contract.



Define the contract before the agent starts editing.

Validation added cost when the first result already passed

8/8

The completion system found nothing to repair. Time rose 21% and provider tokens rose 67%.

Incident task	Checks	Wall time	Provider tokens
OpenHands single	8 / 8	20m 21s	4.10M
Completion system	8 / 8	24m 41s	6.84M

One campaign per condition. Provider tokens include cached input. Not a matched-budget trial.

Completion-loop campaign. One campaign per condition, Sonnet 4.6. Provider tokens include cached input.

Validation found two repairs, but Pi still stopped at 6/9

Two repair rounds recovered two requirements. Three stayed unmet when the budget ended.

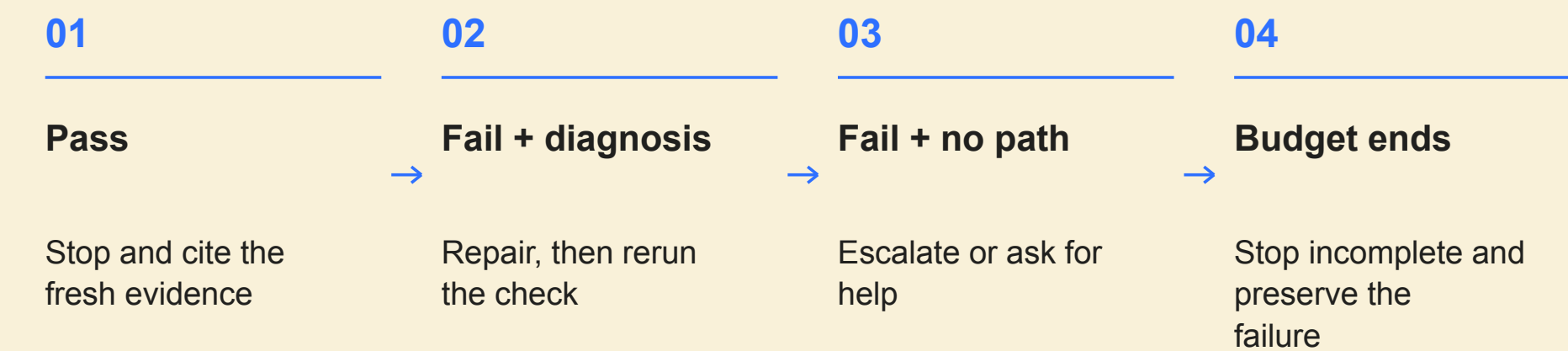
Freight task	Checks	Wall time	Provider tokens
Pi single	4 / 9	25m 36s	1.96M
Pi completion system	6 / 9	99m 10s	22.44M

Separate trajectories. Corrected external scoring. Two repair rounds still left 3 requirements unmet.

Corrected freight results. Separate exploratory trajectories, not matched-budget trials.

Validation decides whether to finish, repair, escalate, or stop

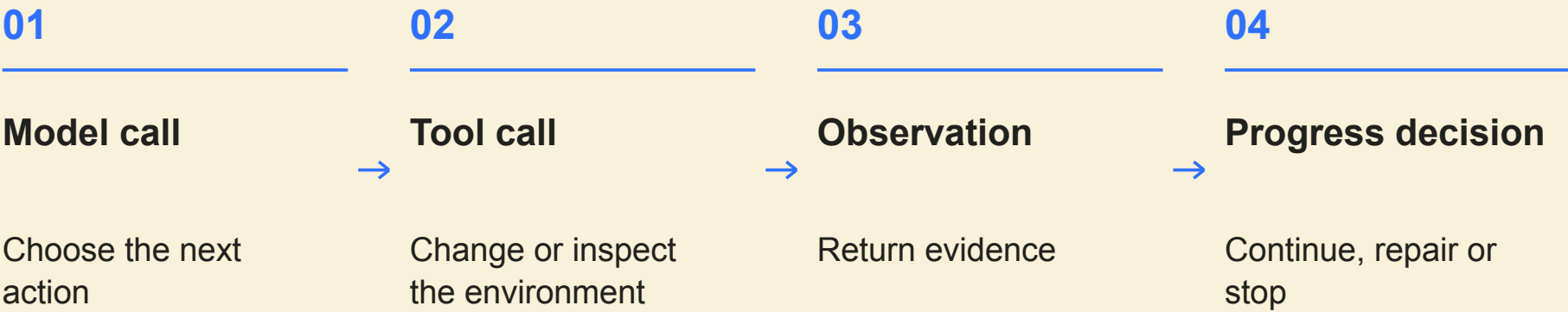
A pass supports completion. A useful failure supports a repair. A budget limit supports an honest incomplete result.



A failed check is useful only when it changes the next action.

One request can contain hundreds of progress decisions

The harness repeats the loop until the goal passes, work is blocked, or a limit ends the run.



A single user request can contain hundreds of these work-check decisions.

GPT-6 Astra showed that a software factory can run without making progress

A broad goal kept the factory running for 35 hours. It lacked checks for readable, maintainable code and a rule for when to stop.

Reported run measure	Result
Wall-clock time	35 hours
Token use	About 1 billion
Raw API cost	About \$1,200
Repository activity	79 commits, 75K net lines
Armin's assessment	“Absolutely nothing of value”

Armin stopped the unattended run himself. The figures and assessment come from his September 7 account.

A goal needs quality checks and a kill switch

Hours, tokens, commits, and lines show activity. Continue only when behavior and code quality still justify another bounded attempt.	
Activity that can hide drift	Controls that measure progress
Hours elapsed	Behavior and quality checks
Tokens consumed	Readable diff review
Commits and lines added	Budget cap and kill switch
The harness should continue only when current evidence supports another bounded attempt.	

Long runs can lose the goal even when each step is simple

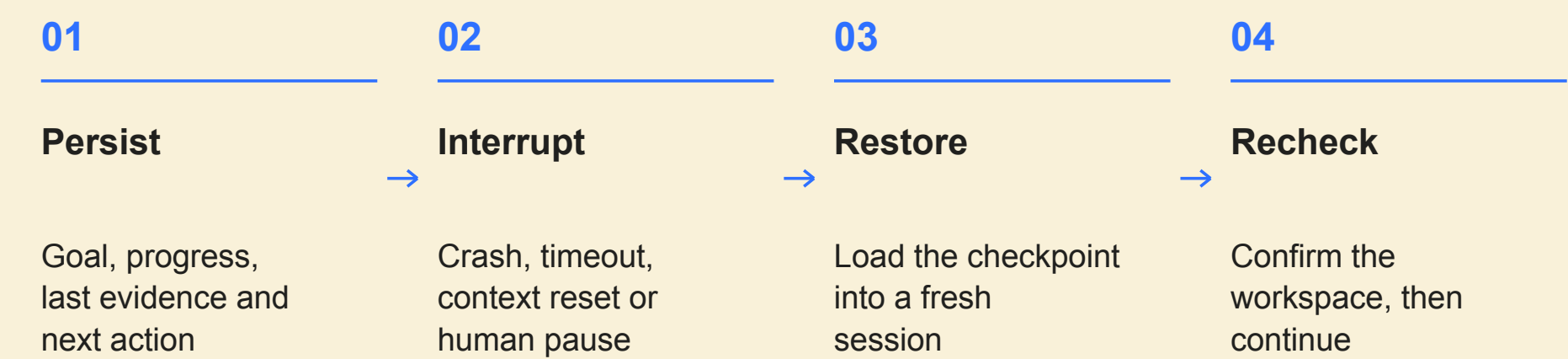
In Vending-Bench, activity declined and every model eventually stopped selling. A longer run needs durable progress state.

Simple work	Long-run outcome
Order stock	Activity declined over time
Set prices	Agents stagnated on average
Pay fees	Every model eventually stopped selling

Vending-Bench exposes failures that do not appear in one short task.

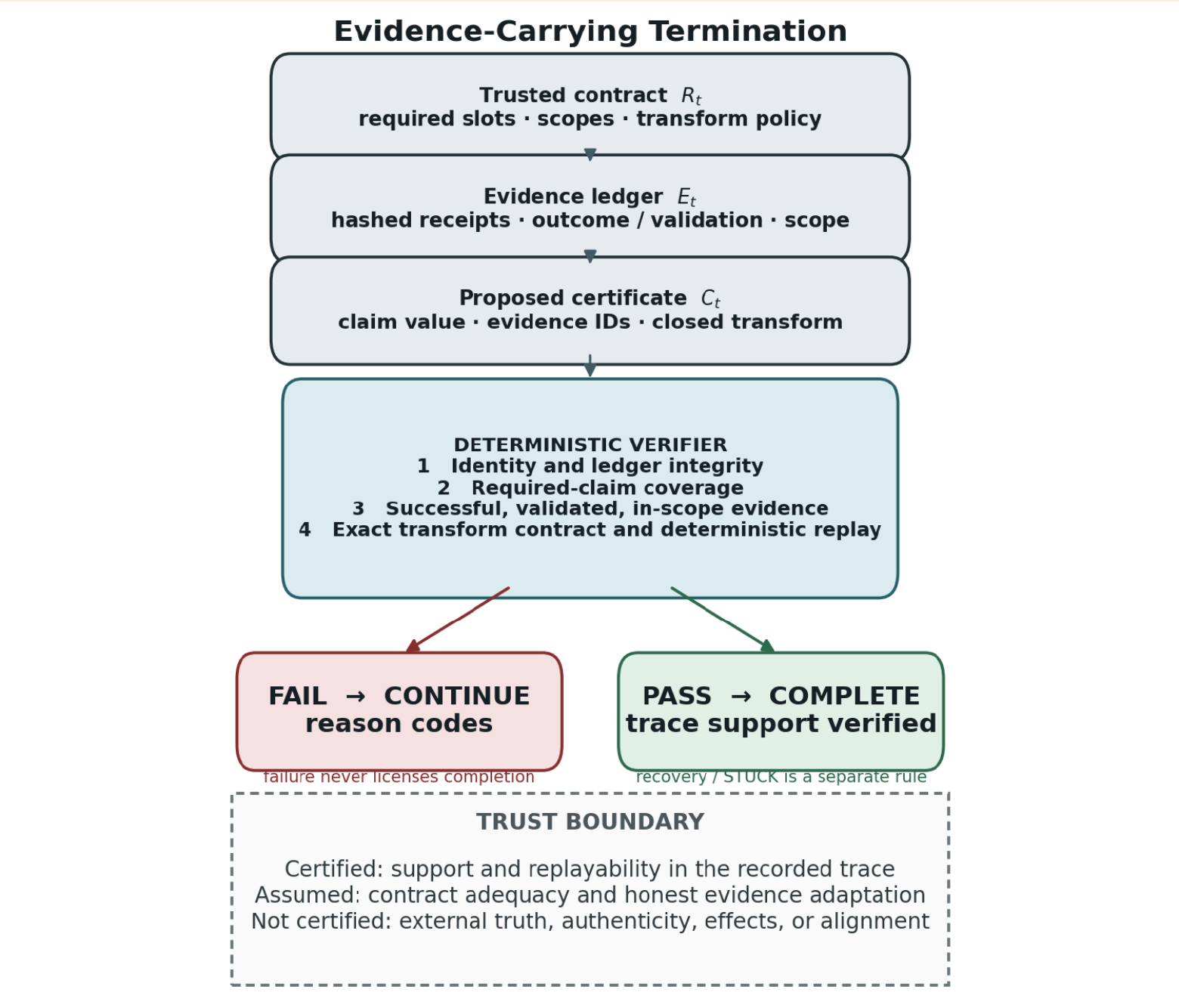
Durable goal workflows survive interruptions and restore state

Persist the goal, progress, latest evidence, and next action. After a restart, verify the workspace before continuing.



A checkpoint restores the decision state. Fresh evidence still must follow the restart.

An evidence gate blocks completion until the required proof passes



An evidence gate is an executable stop rule: no completion claim is accepted without current, in-scope evidence for every required outcome.

In 66 incomplete checkpoints, the paper's gate reduced unsupported stops from 40 to 0. It verifies trace support, not external truth or overall code quality.

Define the validation contract before the agent writes code

● HOW THEY WORK

The validation loop

Tests written after implementation don't catch bugs. They confirm decisions.

PLANNING PHASE

Validation Contract

Written by orchestrator during planning, before any code. Hundreds of assertions define correctness independently of implementation.

After each milestone...

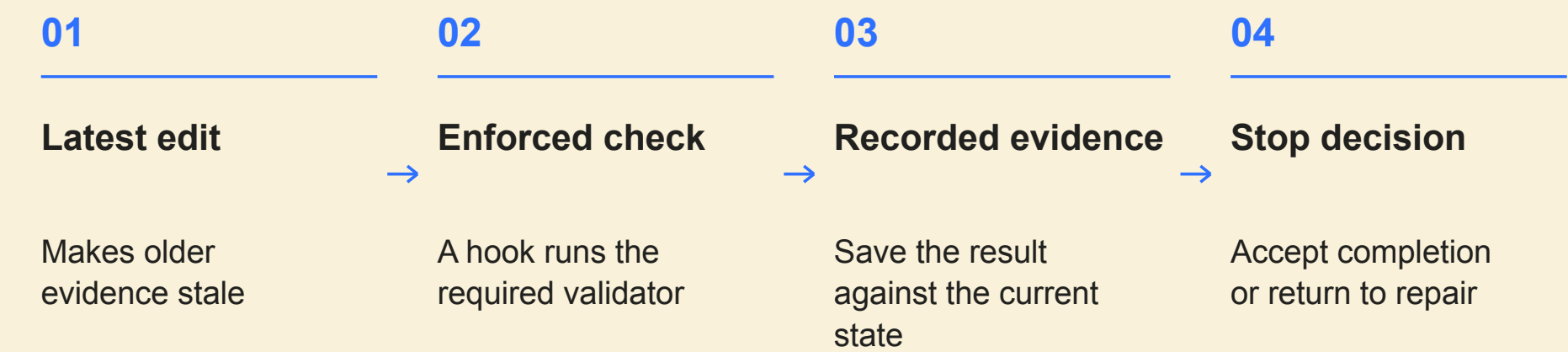
</>
Scrutiny Validator
Runs tests, typechecking, linting. Spawns code review agents for each completed feature.

</>
User-Testing Validator
Acts like a QA engineer. Launches the app, navigates via computer-use & verifies flows end-to-end.

Neither validator has ever seen the code. Validation is **adversarial by design**.

Fresh evidence gates completion

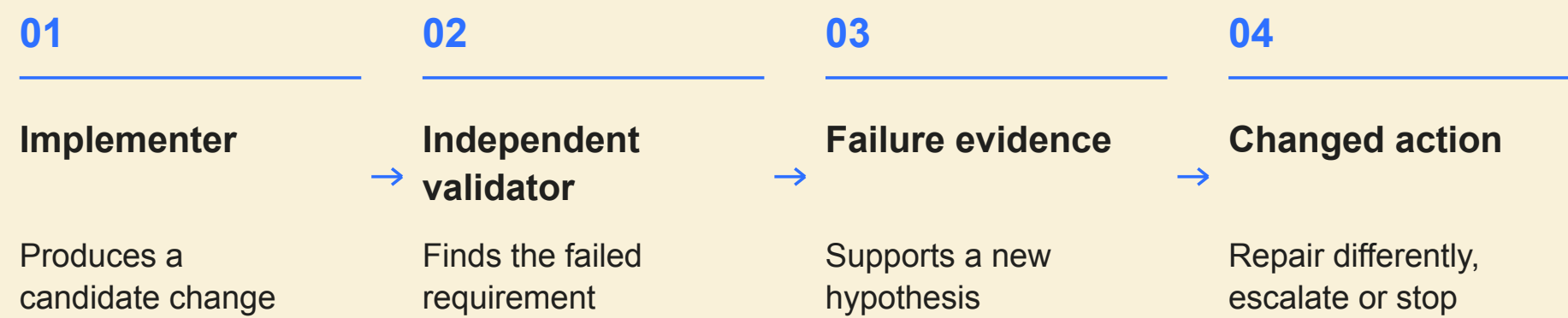
A hook runs the required check after the latest edit. The harness accepts completion only from the current result.



The harness accepts completion only from evidence produced after the latest relevant edit.

A failed check should change the next attempt

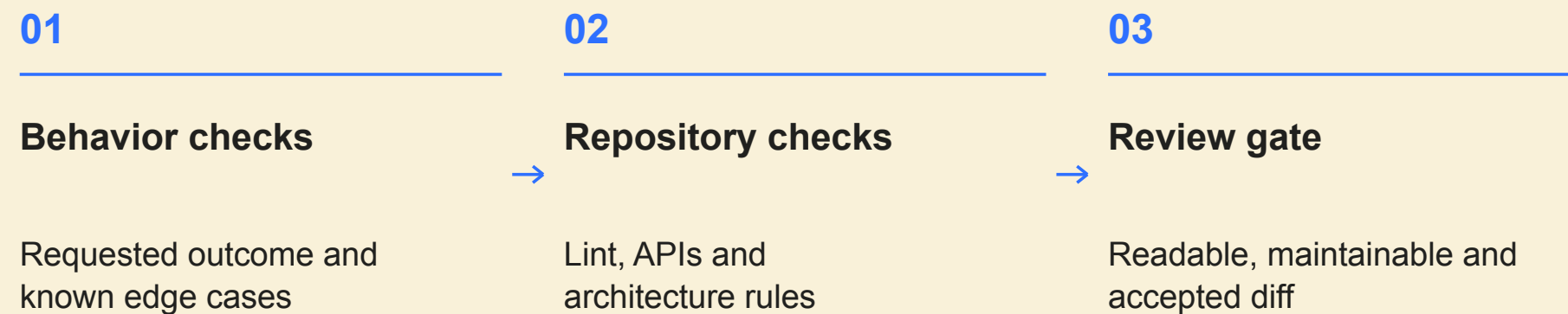
The validator identifies the failed requirement.
The harness uses that evidence to form a new hypothesis, choose a different action, or escalate.



A failed check earns another attempt only when it changes what happens next.

A passing test is not enough to ship code

Executable repository rules catch structural failures. Human review protects qualities that remain difficult to encode.



A passing agent-written test is only the first gate.

The evidence and remaining budget determine the next move

Ask a volunteer to choose a branch. Another attempt is justified only by a new hypothesis and enough budget to test it.

Continue with a changed plan

- Name the failed requirement
- Form a new, testable hypothesis
- Use the remaining budget to repair, retrieve, or change model

Escalate or stop incomplete

- No credible next hypothesis
- The same failure keeps repeating
- Budget or authority has ended

Ask the room: which current evidence makes another attempt worth its cost?

Workshop 6: Multi-agent systems

06

Should another agent own part of this work?

Another agent adds capacity and a coordination bill

Parallel work can finish sooner. The parent still pays to assign the work, pass context, combine the results, and verify the whole.

What another agent adds

What the parent still pays for

A separate context window

Selecting and describing the assignment

Parallel tool use

Passing the relevant context

An independent investigation

Combining returned results

A second attempt or perspective

Verifying the complete outcome

Parallel work helps only when the independent work exceeds the coordination cost.

What failed here: subagents or the handoff?

←

Post

boris

✓

@boristane

if you're building sub-agents you should stop now

our main agent used to be a workflow with sub-agents, each with a specific role (triaging, investigating, etc.)

we moved to a single agent and latency dropped, quality improved and cost was slashed

polylane

Sub-agents are just wrong

Aleksandr Diamond and Boris Tane · September 14, 2026

Sub-agents are just wrong · Polylane

From polylane.com

1:03 PM · Sep 14, 2026 · 14.1K Views

30

21

231

230

←

Post

boris

✓

@boristane · 1h

a lot of new techniques for compaction, that's the way

1 2 299

Show replies

Navlio

✓

@Navlio · 1h

the part that broke for us was the handoff. every sub-agent passes a summary up, so whoever makes the call is reading a report about the work instead of the work

one agent keeps the raw output in context, and that's most of the quality difference

1 1 408

boris

✓

@boristane · 1h

precisely

329

Nandan Priyadarshi

✓

@nandanpri · 1h

Had the same multi-agent spaghetti last quarter — triage bot, fixer bot, reviewer bot. Collapsed it to one agent with a clearer system prompt and a status file; latency dropped and I stopped babysitting handoffs.

1 2 292

boris

✓

@boristane · 1h

let's goooo

Which task would you delegate?

A. Pagination repair: workers would edit the same code.

B. Five repository investigations: each worker can inspect a separate repository.

Vote for one agent, a worker plus validator, or parallel investigators.

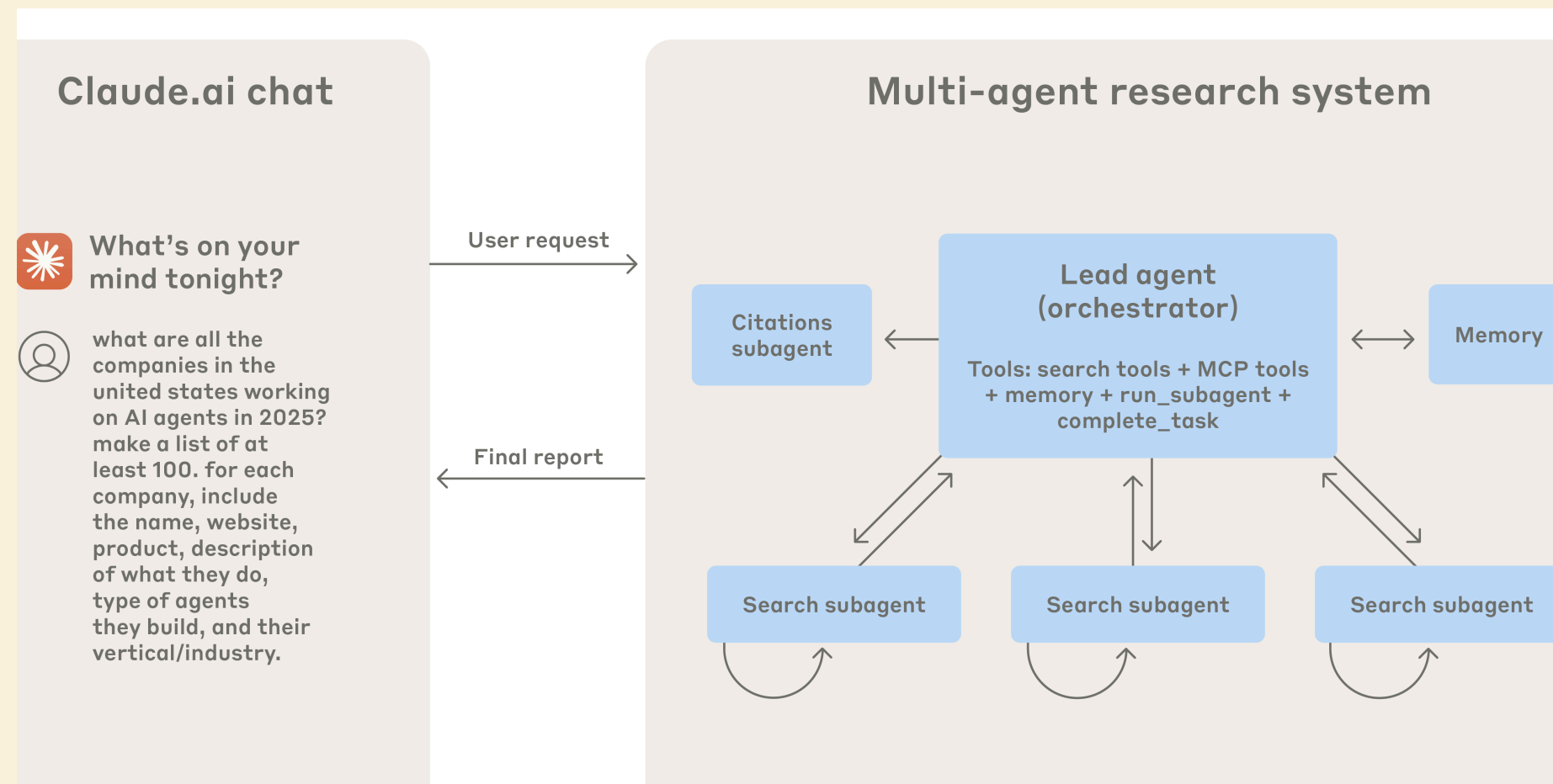
Keep the total budget the same.

Parallelize independent investigations; keep shared edits with one owner

		The split works when each result can be checked independently. Shared changing files create conflicts and serial dependencies.
Independent investigations	Shared implementation	
Separate repository questions	Same changing files	Sources: Anthropic · LangChain · Cognition
Read-only evidence gathering	Edits depend on each other	
Root combines the findings	Keep one implementation owner	

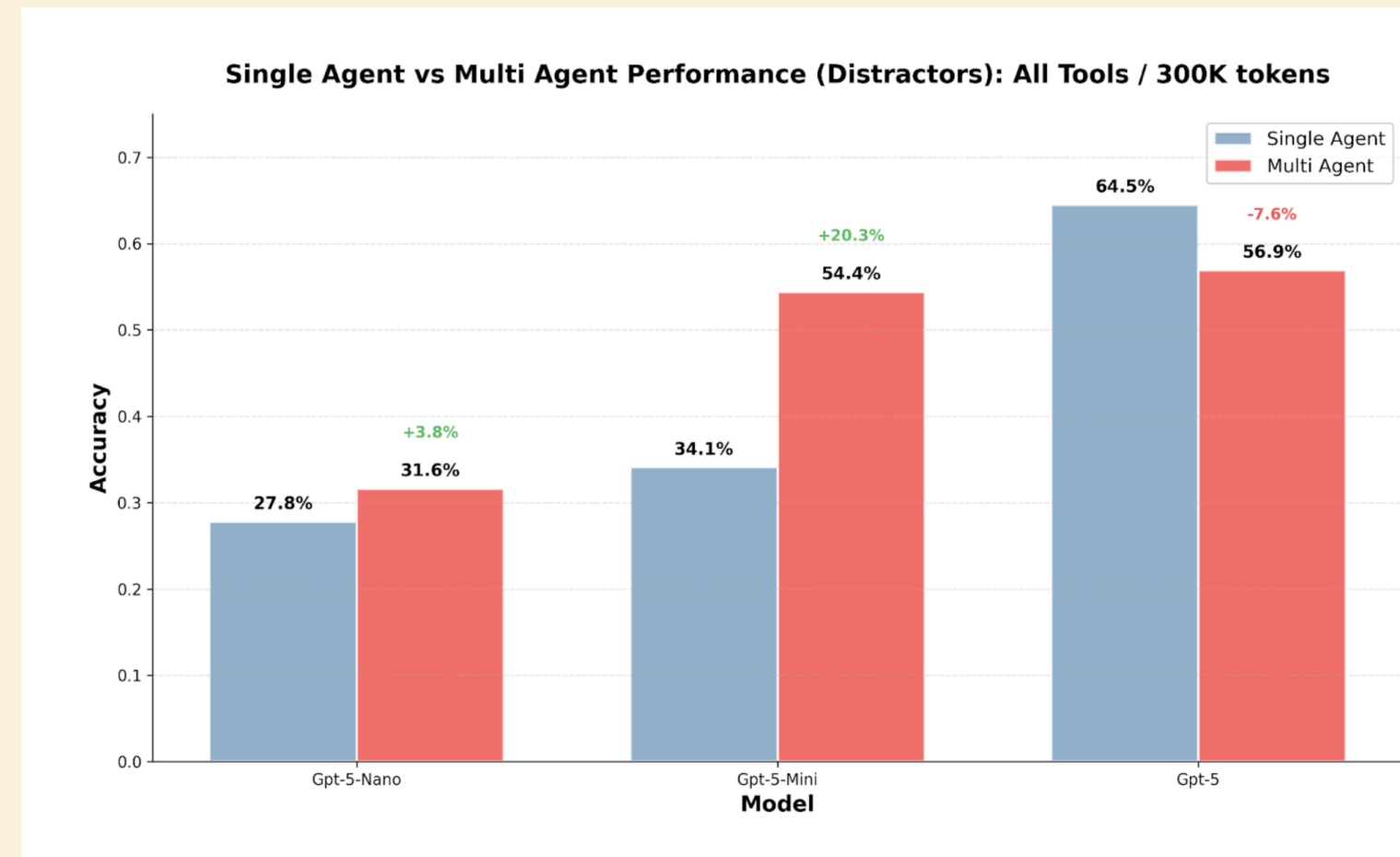
Compare setup, handoffs and synthesis with one agent using the same total budget.

Parallel research improved 90.2% when the searches were independent



Anthropic compared an Opus 4 lead with Sonnet 4 workers against one Opus 4 agent on an internal research evaluation. The compute was unequal. Its separate 15× token figure compares multi-agent research with chat, not with the single-agent research baseline.

Splitting a large tool catalog helped smaller models but hurt GPT-5



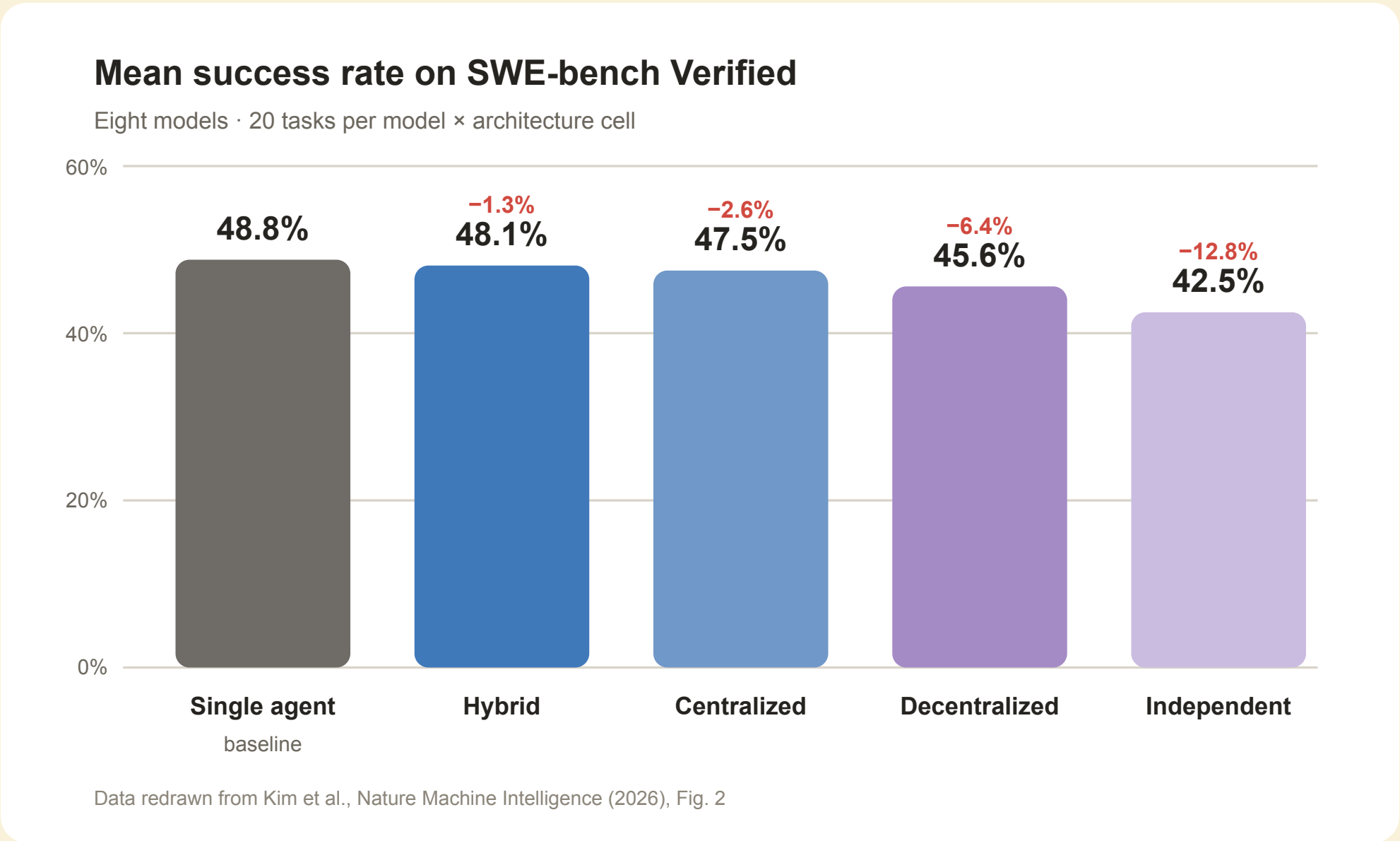
In Snorkel's All Tools condition, multi-agent accuracy rose 3.8 points for GPT-5 Nano and 20.3 points for GPT-5 Mini, then fell 7.6 points for GPT-5. The experiment used a limited sample and retained API-error-free runs.

With an equal reasoning budget, one agent usually matched or beat the group

Data	Model	100 thinking tokens							500 thinking tokens						
		SAS	SAS-L	Seq	Sub	Roles	Deb	Ens	SAS	SAS-L	Seq	Sub	Roles	Deb	Ens
FRAMES	Qwen3-30B	0.191	0.195	0.198	0.155	0.207	0.204	0.146	0.240	0.220	0.223	0.187	0.223	0.206	0.19
MuSiQue	Qwen3-30B	0.200	0.210	0.220	0.174	0.202	0.225	0.149	0.250	0.213	0.226	0.187	0.204	0.229	0.18
FRAMES	DeepSeek-R1-70B	0.365	0.374	0.387	0.385	0.427	0.400	0.365	0.427	0.432	0.396	0.402	0.407	0.392	0.40
MuSiQue	DeepSeek-R1-70B	0.294	0.316	0.328	0.309	0.316	0.308	0.316	0.383	0.375	0.332	0.303	0.329	0.320	0.31
FRAMES	G2.5-F	0.333	0.427	0.462	0.365	0.441	0.445	0.284	0.487	0.459	0.494	0.442	0.451	0.470	0.38
MuSiQue	G2.5-F	0.263	0.352	0.272	0.264	0.300	0.318	0.243	0.340	0.352	0.287	0.291	0.311	0.329	0.27
FRAMES	G2.5-P	0.368	0.451	0.654	0.530	0.609	0.649	0.407	0.600	0.450	0.660	0.562	0.598	0.660	0.40
MuSiQue	G2.5-P	0.308	0.373	0.393	0.364	0.400	0.412	0.302	0.391	0.423	0.391	0.363	0.397	0.435	0.33
A		0.300	0.337	0.364	0.333	0.368	0.378	0.300	0.300	0.366	0.376	0.348	0.365	0.300	0.31

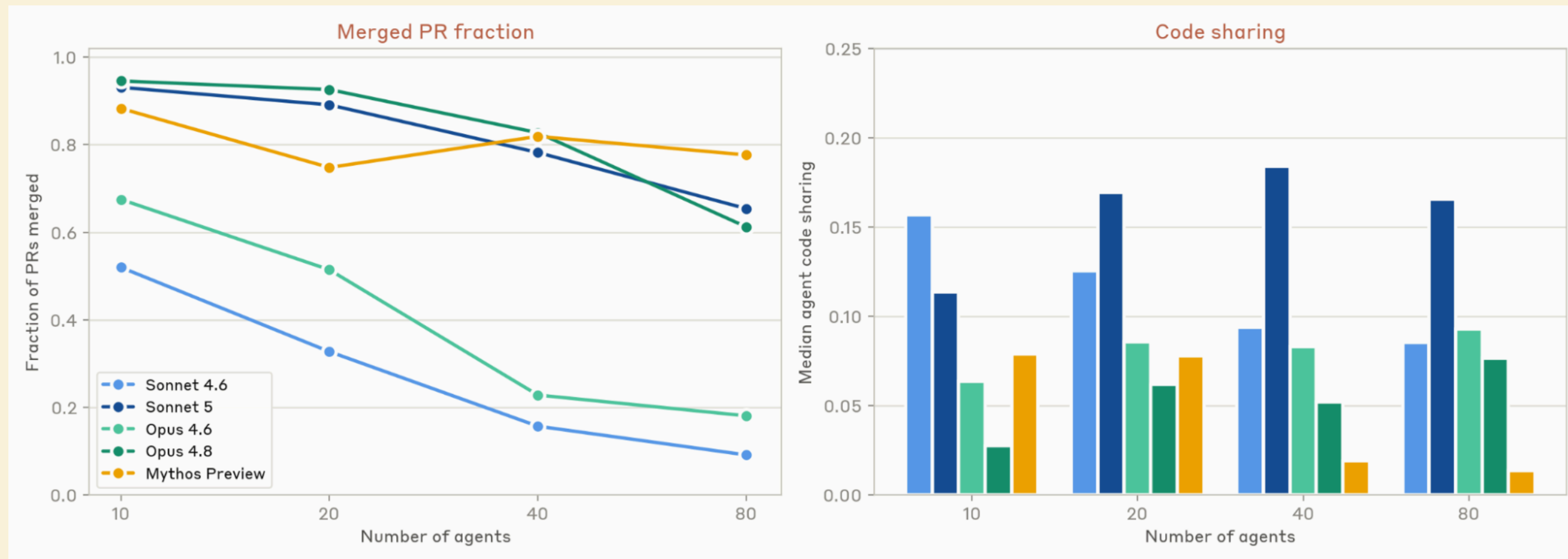
At the requested 500-token budget, a single-agent variant was best or statistically tied in six of eight model and dataset comparisons. Gemini did not always use the requested number of reasoning tokens, so the authors also report realized usage.

Every tested multi-agent architecture made SWE-bench results worse



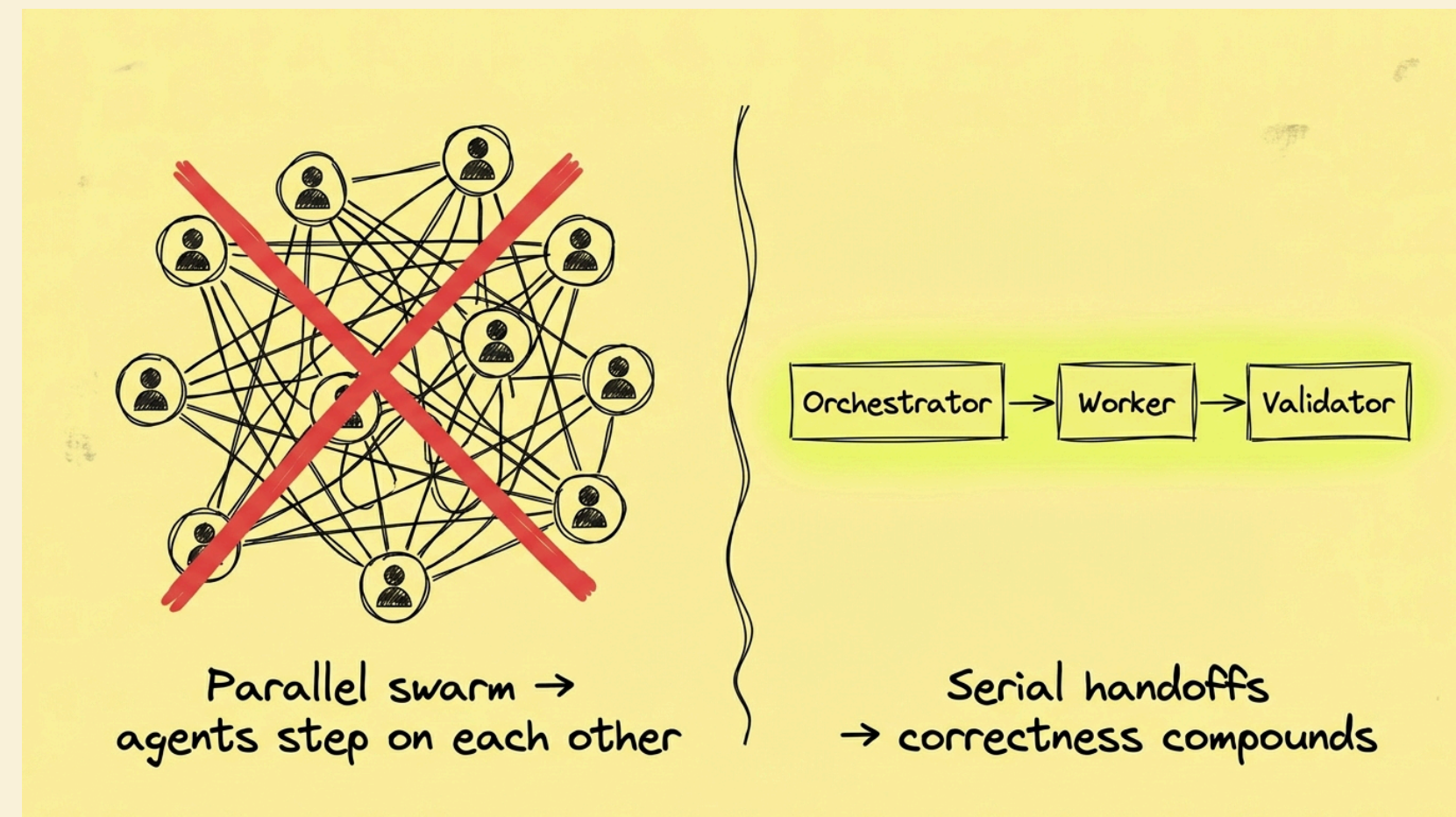
Across eight models, all four multi-agent architectures scored below the single-agent baseline: from -1.3% for hybrid to -12.8% for independent. Each configuration used 20 SWE-bench Verified tasks, so the exact per-cell gaps are imprecise.

Shared code became harder to merge as more agents joined



In Anthropic's 12-hour shared-repository experiment, earlier Claude models opened many conflicting pull requests and merged fewer as the group grew. Newer models often avoided conflicts by keeping stronger file ownership.

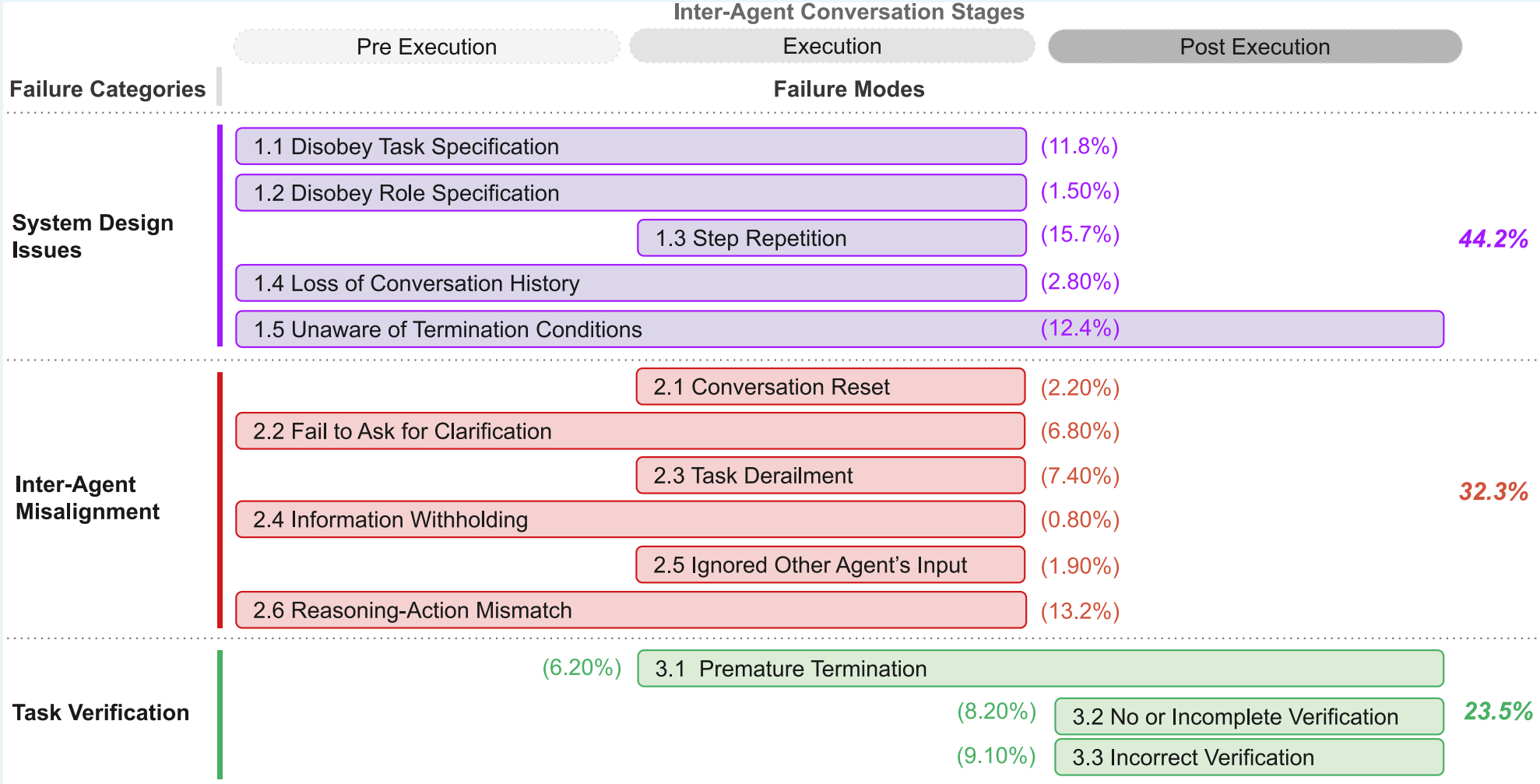
Factory serializes code changes and parallelizes read-only work



Factory reports using one code-changing worker or validator at a time. Code search, API research, and independent review can still run in parallel because they do not compete to modify the same state.

Design the handoff around three failure surfaces

Across 1,642 traces from seven systems, MAST grouped 14 failure modes into system design, inter-agent misalignment, and task verification. A delegation contract should defend all three.



Six decisions shape how the agent works

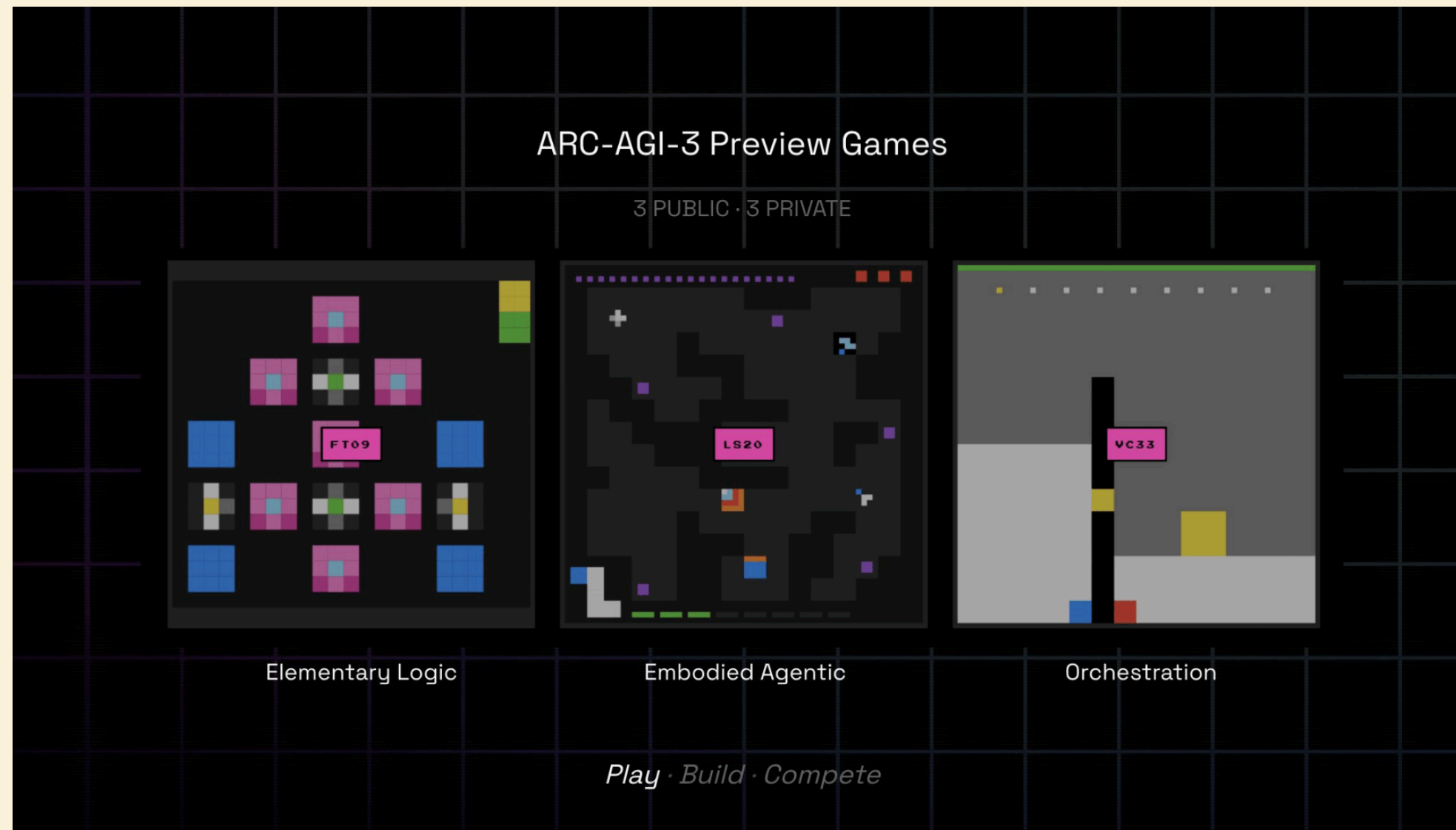
The model chooses actions inside the environment created by these harness decisions.

Workshop	Harness decision
Harness choice	Which behavior and defaults do we inherit?
Tools	Which actions and evidence can the model request?
Context	What enters the next model call, and what persists?
Model choice	Which model or attempt gets the remaining budget?
Goals and validation	Which evidence permits another attempt or completion?
Multiple agents	Which work separates cleanly, and who owns synthesis?

Measure each change against the same task, environment, and acceptance checks.

Two harness settings raised ARC-AGI-3 from 13.3% to 38.3%

The model stayed fixed. Retaining reasoning and replacing rolling truncation with compaction tripled the reported RHAEE score and used six times fewer output tokens.



A learned harness raised Opus 4.6 from 74.7% to 76.4%

Meta-Harness generated and selected agent configurations for TerminalBench 2.0 instead of relying on one fixed, hand-written setup.

TerminalBench 2.0 harness	Pass rate
Claude Code	58.0%
Terminus-KIRA	74.7%
Meta-Harness	76.4%

Claude Opus 4.6 · 89 tasks · five independent trials per task. Source: Meta-Harness project page.

Harness-R1 gained 9.3 points while the task model stayed frozen

A separate 9B harness engineer converted batches of failures into validated runtime patches for a frozen Qwen3.5-9B task model.

Frozen Qwen3.5-9B condition	Success
Vanilla task harness	44.3%
Harness-R1 runtime patches	53.6%

Aggregate result across WebShop, ALFWorld, and DBBench · +9.3 percentage points. Source: Harness-R1 preprint.

The harness and runtime accumulate technical debt around the model

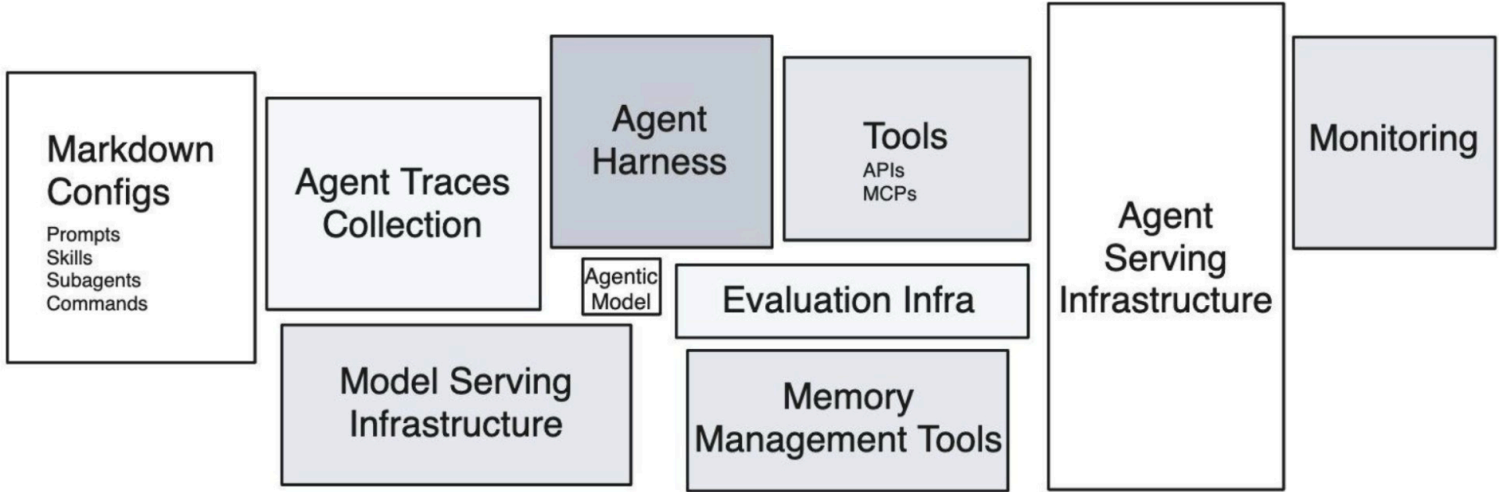
Han, Not Solo

BlogsPublicationsTalksAbout

Hidden Technical Debt of AI Systems: Agent Runtime

Apr 24, 2026 • Han Lee | 18 min read (3306 words)  

Eleven years ago, Sculley et al. drew **the diagram** everyone in MLOps has seen: a tiny black box labeled “ML Code” surrounded by a sprawl of much larger boxes — data collection, feature extraction, configuration, monitoring, serving infrastructure. The point of the diagram was that the model code is the smallest piece of a real ML system, and that everything else is where the technical debt accumulates.



The diagram illustrates the components of an Agent Runtime system. It features a central 'Agentic Model' box, which is the smallest. Surrounding it are several larger boxes representing infrastructure and tools. To the left is 'Markdown Configs' (Prompts, Skills, Subagents, Commands). To the right is 'Agent Serving Infrastructure' (the largest box), which is connected to 'Monitoring'. Above the 'Agentic Model' is 'Agent Harness' and 'Tools' (APIs, MCPs). Below it are 'Agent Traces Collection', 'Model Serving Infrastructure', 'Evaluation Infra', and 'Memory Management Tools'.

The same diagram is being redrawn for agents. The agentic model call is the small box. The largest box to the right that is currently driving most of the spend and influencing how system architecture will be done in the future incidents, is the **agent runtime**, or agent serving infrastructure.

OpenHands and OpenCode each merged about 5,700 PRs in one year

Production harnesses require ongoing engineering as models, tools, providers, and workloads change.



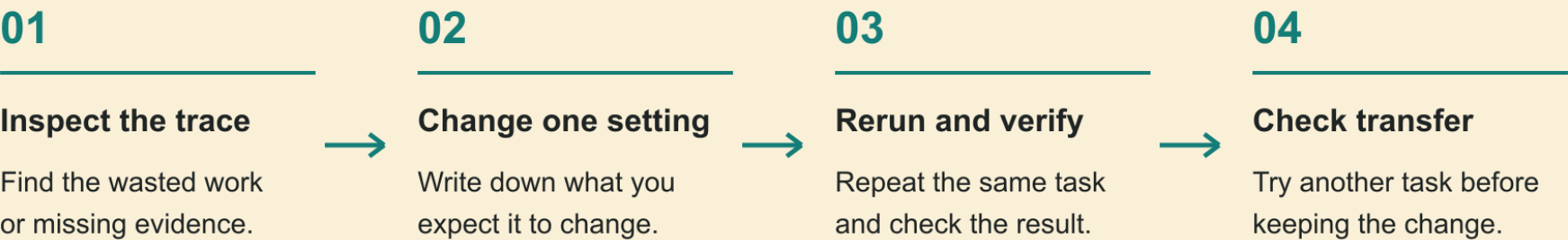
Model changes can make harness workarounds obsolete

A workaround that helps one model can hinder the next. Preserve stable repository facts, but retest model-specific prompts, tools, and routing rules after a model change.

One harness change, measured against the baseline

A harness change needs a repeatable comparison

Use Agent Canvas to locate the failure, then check the effect of one change.



HOLD FIXED Task, model, starting files, environment, verifier

MEASURE Verified outcome, total cost, time, retries, human intervention

Use repeated trials to distinguish the intended effect from a different model trajectory.

Price fresh input, cached input, and output across the full task. Keep the change only if verified quality holds and total cost falls.

Workshop: github.com/rajshah4/harness-engineering